

openEuler 24.03 LTS SP4 Technical White Paper

1 Introduction

The openEuler open source community is incubated and operated by the OpenAtom Foundation.

openEuler is a digital infrastructure OS that fits into any server, cloud computing, edge computing, and embedded deployment. This secure, stable, and easy-to-use open source OS is compatible with multiple computing architectures. openEuler suits operational technology (OT) applications and enables the convergence of OT and information and communications technology (ICT).

The openEuler open source community is a portal available to global developers, with the goal of building an open, diversified, and architecture-inclusive software ecosystem for all digital infrastructure scenarios. It has a rich history of helping enterprises develop their software, hardware, and applications.

The openEuler open source community was officially established on December 31, 2019, with the original focus of innovating diversified computing architectures.

On March 30, 2020, the Long Term Support (LTS) version openEuler 20.03 was officially released, which was a new Linux distribution with independent technology evolution.

On September 30, 2020, the innovative openEuler 20.09 version was released through the collaboration efforts of multiple companies, teams, and independent developers in the openEuler community. The release of openEuler 20.09 marked a milestone not only in the growth of the openEuler community, but also in the history of open sourced software in China.

On March 31, 2021, the innovative kernel version openEuler 21.03 was released. This version is enhanced in line with Linux kernel 5.10 and also incorporates multiple new features, such as live kernel upgrade and tiered memory expansion. These features improve multi-core performance and deliver the computing power of one thousand cores.

On September 30, 2021, openEuler 21.09 was released. This premium version is designed to supercharge all scenarios, including edge and embedded devices. It enhances server and cloud computing features, and incorporates key technologies including cloud-native CPU scheduling algorithms for hybrid service deployments and KubeOS for containers.

On March 30, 2022, openEuler 22.03 LTS was released based on Linux kernel 5.10. Designed to meet all server, cloud, edge computing, and embedded workloads, openEuler 22.03 LTS is an all-scenario digital infrastructure OS that unleashes premium computing power and resource utilization.

On September 30, 2022, openEuler 22.09 was released to further enhance all-scenario innovations.

On December 30, 2022, openEuler 22.03 LTS SP1 was released, which is designed for hitless porting with best-of-breed tools.

On March 30, 2023, openEuler 23.03 was released. Running on Linux kernel 6.1, it streamlines technical readiness for Linux kernel 6.x and facilitates innovations for hardware adaptation and other technologies.

On June 30, 2023, openEuler 22.03 LTS SP2 was released, which enhances scenario-specific features and increases system performance to a higher level.

On September 30, 2023, the openEuler 23.09 innovation version was released based on Linux kernel 6.4. It provides a series of brand-new features to further enhance developer and user experience.

On November 30, 2023, openEuler 20.03 LTS SP4 was released, an enhanced extension of openEuler 20.03 LTS. openEuler 20.03 LTS SP4 provides excellent support for server, cloud native, and edge computing scenarios.

On December 30, 2023, openEuler 22.03 LTS SP3 was released. Designed to improve developer efficiency, it enhances features for server, cloud native, edge computing, and embedded scenarios.

On May 30, 2024, openEuler 24.03 LTS was released. This version is built on Linux kernel 6.6 and brings new features for server, cloud, edge computing, AI, and embedded deployments to deliver enhanced developer and user experience.

On June 30, 2024, openEuler 22.03 LTS SP4 was released. Designed to improve developer efficiency, it further extends features for server, cloud native, edge computing, and embedded scenarios.

On September 30, 2024, openEuler 24.09 was released. It is an innovation version built based on Linux kernel 6.6 and brings more advanced features and functions.

On December 30, 2024, openEuler 24.03 LTS SP1 was released. This enhanced and extended version of openEuler 24.03 LTS is developed on Linux kernel 6.6 and designed for server, cloud, edge computing, and embedded deployments. It offers new features and enhanced functions that streamline processes across a range of domains.

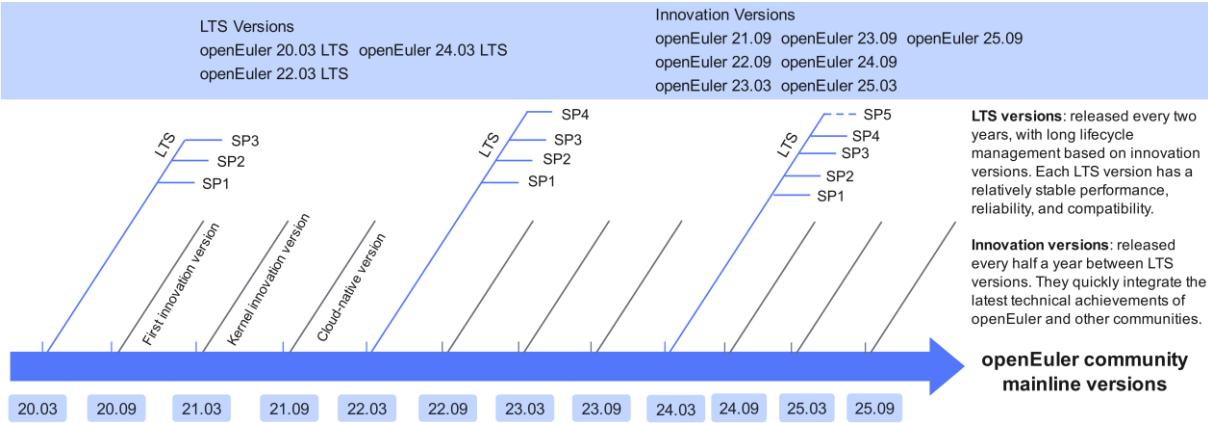
On March 30, 2025, openEuler 25.03 was released. It is an innovation version designed based on Linux kernel 6.6 and is suited for server, cloud, edge, and embedded scenarios. It provides a variety of new features and functions and brings brand-new experience to developers and users in diverse industries.

On June 30, 2025, openEuler 24.03 LTS SP2 was released. It is an enhanced version of 24.03 LTS based on the Linux 6.6 kernel. It is designed for server, cloud, AI, and embedded scenarios, introducing new functions and features covering kernel optimization, heterogeneous collaborative inference, high-density many-core computing, confidential containers, and multi-core and multi-instance hybrid deployment. It offers enhanced experience for developers and users across various industries.

On September 30, 2025, openEuler 25.09 was released. It is an innovation version based on the Linux 6.6 kernel. It offers new features and enhanced functionality for server, cloud, AI, and embedded scenarios. Key improvements include kernel optimization, heterogeneous collaborative inference, high-density many-core computing, confidential containers, confidential VMs, multi-core and multi-instance hybrid deployment, compilers, trusted computing, and cryptographic acceleration. This latest release enhances the experience for developers and users across a wide range of fields.

More recently, on December 30, 2025, the first openEuler release supporting SuperPoDs was officially launched. openEuler 24.03 LTS SP3, an enhanced and extended version of openEuler 24.03 LTS based on the Linux 6.6 kernel, is designed for server, cloud, and AI scenarios. It introduces a wide range of new features and capabilities, including kernel optimization, heterogeneous collaborative inference, intelligent diagnosis, confidential virtual machines (cVMs), compiler enhancements, RISC-V architecture optimizations, an intelligent developer desktop, security hardening, UnifiedBus-powered SuperPoDs, identity authentication, and virtualization. This release delivers a significantly improved experience for developers and users across a broader range of industries.

On June 30, 2026, openEuler 24.03 LTS SP4 was officially released as an enhanced and extended update to openEuler 24.03 LTS, built on Linux kernel 6.6. Designed for server, cloud, and AI scenarios, this release continues to deliver a wide range of new features and capability enhancements. These include kernel optimizations; improved reliability and usability of UnifiedBus SuperPoDs; NPU resource partitioning; rapid recovery for inference services; E2B sandbox support; intelligent diagnostics, tuning, and operations and maintenance (O&M); compiler advancements; and confidential virtual machines. Together, these innovations provide a refreshed experience for developers and users, enabling openEuler to support a broader range of applications and serve an expanding user base.

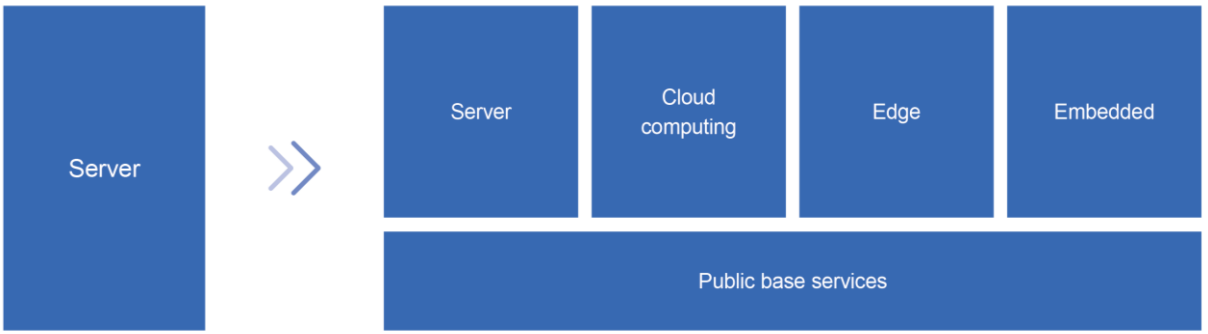


openEuler releases LTS versions to provide enhanced specifications and a secure, stable, and reliable OS for enterprise users.

openEuler is built on tried-and-tested technologies. Its innovation versions, released every half year, quickly integrate the latest technical achievements of openEuler and other communities. The innovative tech is first verified in the openEuler open source community as a single open source project, and then these features are added to each new release, enabling community developers to obtain the source code.

Technical capabilities are first tested in the open source community, and continuously incorporated into each openEuler release. In addition, each release is built on feedback given by community users to bridge the gap between innovation and the community, as well as improve existing technologies. openEuler is both a release platform and incubator of new technologies, working in a symbiotic relationship that drives the evolution of new versions.

Innovative Platform for All Scenarios



openEuler supports multiple processor architectures (x86, Arm, RISC-V, and LoongArch), as part of a focus to continuously improve the ecosystem of diversified computing power.

The openEuler community is home to an increasing number of special interest groups (SIGs), which are dedicated teams that help extend the OS features from server to cloud computing, edge computing, and embedded scenarios. openEuler is built to be used in any scenario.

The OS is a perfect choice for ecosystem partners, users, and developers who plan to enhance scenario-specific capabilities. By creating a unified OS that supports multiple devices, openEuler hopes to enable a single application development for all scenarios.

Open and Transparent Software Supply Chain

The process of building an open source OS relies on supply chain aggregation and optimization. To ensure reliable open source software or a large-scale commercial OS, openEuler comprises a complete lifecycle management that covers building, verification, and distribution. The brand regularly reviews its software dependencies based on user scenarios, organizes the upstream community addresses of all the software packages, and verifies its source code by comparing it to that of the upstream communities. The build, runtime dependencies, and upstream communities of the open source software form a closed loop, realizing a complete, transparent software supply chain management.

2 Platform Architecture

System Framework

openEuler is an innovative open source OS platform built on kernel innovations and a solid cloud base to cover all scenarios. It is built on the latest trends of interconnect buses and storage media, and offers a distributed, real-time acceleration engine and base services. It provides competitive advantages in edge and embedded scenarios, and is the first step to building an all-scenario digital infrastructure OS.

openEuler 24.03 LTS SP4 runs on Linux kernel 6.6 and provides POSIX-compliant APIs and OS releases for server, cloud native, and edge environments. It is a solid foundation for intelligent collaboration across hybrid and heterogeneous deployments. openEuler 24.03 LTS SP4 is equipped with a distributed soft bus and K3s edge-cloud collaboration framework, among other premium features, making it a perfect choice for collaboration over digital infrastructure and everything connected models.

In the future, the openEuler open source community will continue to innovate, aiming to promote the ecosystem and consolidate the digital infrastructure.

Cloud base:

- **KubeOS for containers:** In cloud native scenarios, the OS is deployed and maintained in containers, allowing the OS to be managed based on Kubernetes, just as service containers.
- **Secure containers:** Compared with the traditional Docker+QEMU solution, the iSulad+shimv2+StratoVirt secure container solution reduces the memory overhead and boot time by 40%.
- **Confidential containers:** The iSulad+Kuasar+secGear confidential container solution offers privacy protection while remaining compatible with the cloud-native ecosystem.
- **Confidential computing:** Based on the Arm Confidential Compute Architecture (Arm CCA) specification, this solution introduces support for direct device passthrough, including NICs, NVMe drives, and Chinese cryptography accelerator cards, into VMs. It ensures the confidentiality and integrity of data and code within confidential VMs, providing robust

protection even in the presence of privileged infrastructure software or cloud service providers.

All scenarios:

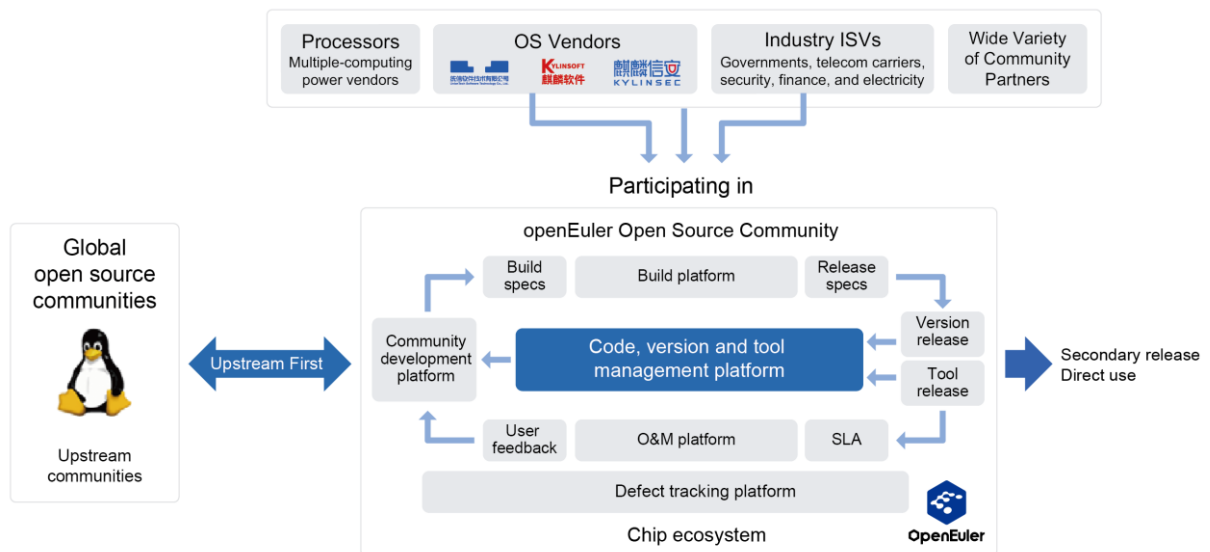
- **Edge computing:** openEuler 24.03 LTS SP4 Edge is released for edge computing scenarios. It integrates the Kubernetes edge-cloud collaboration framework to provide unified management, provisioning of edge and cloud applications, and other capabilities.
- **Embedded:** openEuler 24.03 LTS SP4 Embedded features an image smaller than 5 MB and a boot time of under 5 seconds. Its elastic virtualization base and mixed-criticality (MICA) deployment framework support concurrent deployment of multiple cores and instances. Cross-OS communication provides an efficient shared-memory mechanism for communication between different OSs.
- **AI-native OS:** The OS enables AI software stacks with out-of-the-box availability. Heterogeneous convergence of memory, scheduling, and training/inference resources reduces AI development costs and improves efficiency. The intelligent interaction platform of the OS streamlines development and administration.
- **SuperPoD OS:** The OS extends the existing framework to support UnifiedBus SuperPoDs by building a hierarchical SuperPoD system software stack. It provides SuperPoD capabilities and standardized interfaces, enabling ecosystem development and value realization.

Flourishing community ecosystem:

- **Desktop environments:** UKUI, DDE, and Kiran-desktop.
- **openEuler DevKit:** Supports OS migration, compatibility assessment, and various development tools such as secPaver which simplifies security configuration.
- **DevStation:** It offers an intelligent developer workstation that delivers an out-of-the-box, efficient, and secure development environment, streamlining the entire process from coding and deployment to building, packaging, and release. Additionally, it integrates a Model Context Protocol (MCP) AI engine to quickly invoke the community toolchain, improving efficiency from infrastructure setup to application development. It also integrates an intelligent CVE remediation system that significantly improves the response speed and efficiency of security vulnerability fixes.

Platform Framework

The openEuler open source community partners with upstream and downstream communities to advance the evolution of openEuler versions.



Hardware Support

The openEuler open source community works with multiple vendors to build a vibrant southbound ecosystem. With participation of major chip vendors including Intel and AMD, all openEuler versions support x86, Arm, LoongArch, and RISC-V CPU architectures, and a wide range of CPU chips, such as Loongson 3 series, Zhaoxin KaiXian and KaiSheng, Intel Sierra Forest and Granite Rapids, and AMD EPYC 4/5. openEuler can run on servers from multiple hardware vendors and is compatible with NIC, RAID, Fibre Channel, GPU & AI, DPU, SSD, and security cards.

openEuler supports the following CPU architectures:

Hardware Type	x86_64	AArch64	LoongArch	RISC-V
CPU	Intel, AMD, Hygon, Zhaoxin	Kunpeng, Phytium	Loongson	Sophgo, THead, etc.

Visit <https://www.openeuler.org/en/compatibility/> to see the full hardware list.

3 Operating Environments

Servers

To install openEuler on a physical machine, check that the physical machine meets the compatibility and hardware requirements.

For a full list, visit <https://openeuler.org/en/compatibility/>.

Item	Configuration Requirement
Architecture	AArch64, x86_64, RISC-V, LoongArch64
Memory	≥ 4 GB
Drive	≥ 20 GB

VMs

Verify VM compatibility when installing openEuler.

Hosts running on openEuler 24.03 LTS SP4 support the following software packages:

- `libvirt-9.10.0-24.oe2403sp334.oe2403sp4`
- `libvirt-client-9.10.0-24.oe2403sp334.oe2403sp4`
- `libvirt-daemon-9.10.0-24.oe2403sp334.oe2403sp4`
- `qemu-8.2.0-57.oe2403sp378.oe2403sp4`
- `qemu-img-8.2.0-57.oe2403sp378.oe2403sp4`

openEuler 24.03 LTS SP4 is compatible with the following guest OSs for VMs:

Guest OS	Architecture
CentOS 6	x86_64
CentOS 7	AArch64
CentOS 7	x86_64
CentOS 8	AArch64
CentOS 8	x86_64
Windows Server 2016	x86_64
Windows Server 2019	x86_64
Item	Configuration Requirement
Architecture	AArch64, x86_64
CPU	≥ 2 CPUs
Memory	≥ 4 GB
Drive	≥ 20 GB

Edge Devices

To install openEuler on an edge device, check that the edge device meets the compatibility and minimum hardware requirements.

Item	Configuration Requirement
Architecture	AArch64, x86_64
Memory	≥ 4 GB
Drive	≥ 20 GB

4 Scenario-specific Innovations

AI

AI is redefining OSs by powering intelligent development, deployment, and O&M. openEuler supports general-purpose architectures like Arm, x86, and RISC-V, and next-gen AI processors like NVIDIA and Ascend. Further, openEuler is equipped with extensive AI capabilities that have made it a preferred choice for diversified computing power.

OS for AI

Ready-to-Use Availability

openEuler offers an efficient development and runtime environment that containerizes software stacks of AI platforms with out-of-the-box availability. It also provides various AI frameworks to facilitate AI development.

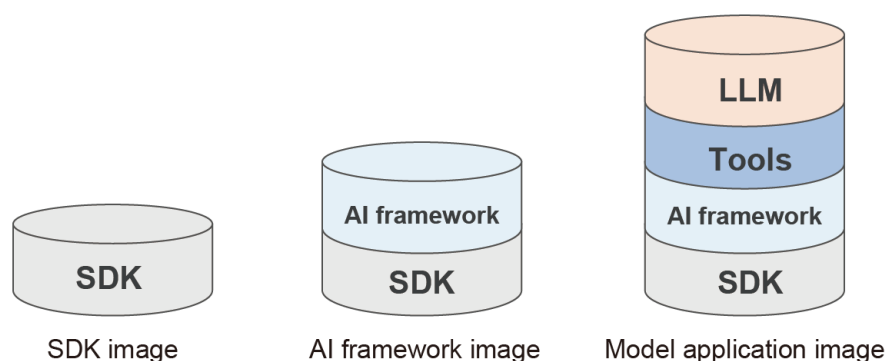
Feature Description

openEuler supports TensorFlow, PyTorch, and MindSpore frameworks and software development kits (SDKs) of major computing architectures, such as Compute Architecture for Neural Networks (CANN) and Compute Unified Architecture (CUDA), to make it easy to develop and run AI applications.

The updated version adds compatibility testing for the following software. All packages can be installed directly using **pip**:

- **Machine learning and deep learning frameworks and libraries:** FastAI, JAX, PyMC, Stable Baselines3, scikit-image
- **Natural language processing (NLP):** spaCy, TextBlob, Gensim, Pattern (requires source build)
- **Large language model (LLM) toolchain:** LlamaIndex, OpenAI Python, Diffusers
- **Computer vision and data augmentation:** Imgaug, Albumentations, Kornia, Detectron2
- **Automated machine learning (AutoML) and hyperparameter optimization:** Optuna
- **Experiment tracking:** Weights & Biases
- **Similarity and nearest neighbor search:** Annoy, NMSLIB, hnswlib
- **Streaming and online learning:** River
- **Parallel computing:** Dask
- **Mathematics and symbolic computation:** SymPy
- **Linear programming and optimization:** PuLP

Environment setup is further simplified by containerizing software stacks. openEuler provides three types of container images:



- **SDK images:** Use openEuler as the base image and install the SDK of a computing architecture, for example, Ascend CANN and NVIDIA CUDA.
- **AI framework images:** Use an SDK image as the base and install AI framework software, such as PyTorch and TensorFlow. Users can use an AI framework image to quickly build a distributed AI framework, such as Ray.
- **Model application images:** Provide a complete set of toolchains and model applications.

For details, see the [openEuler AI Container Image User Guide](#).

Application Scenarios

openEuler uses AI container images to simplify deployment of runtime environments. Users can select the container image that best suits their requirements and complete the deployment in a few simple steps.

- **SDK images:** Users can develop and debug Ascend CANN or NVIDIA CUDA applications using an SDK image, which provides a compute acceleration toolkit and a development environment. These containers offer an easy way to perform high-performance computing (HPC) tasks, such as large-scale data processing and parallel computing.
- **AI framework images:** This type of containers is designed to support AI model development, training, and inference.
- **Model application images:** Such an image contains a complete AI software stack and purpose-built models for model inference and fine-tuning.

XPU Turbo

Feature Description

The XPU Turbo large language model (LLM) heterogeneous acceleration runtime enhances model inference performance in single-server, multi-XPU setups by optimizing Kunpeng + XPU (GPU/NPU) resource synergy.

- **CPU inference acceleration:** Improves CPU throughput via NUMA-aware scheduling, parallelized matrix operations, and SVE-optimized inference operators.
- **Heterogeneous converged scheduling:** Supports two scheduling modes, PD separation and AF separation. In PD separation mode, XPU Turbo supports executing the prefill phase of an inference request on GPUs and offloading the decode phase to CPUs when GPUs are fully utilized. In AF separation mode, XPU Turbo offloads the feed-forward network (FFN) computation to the CPU.

Application Scenarios

XPU Turbo is used to optimize Transformer models, including DeepSeek and Qwen. Its CPU inference acceleration capability has been adapted for DeepSeek 7B/14B/32B and Qwen2.5/Qwen3/Qwen3-MoE series models on mainstream hardware platforms such as NVIDIA and Ascend. In addition, with AF separation enabled, XPU Turbo supports running inference for the Qwen3-235B-A22B model on a single NPU. XPU Turbo fits into the following application scenario:

- **Data centers:** XPU Turbo assigns inference tasks to CPUs to fully utilize CPU resources and increase the concurrency and throughput of LLMs.

For details about how to use XPU Turbo, see the [XPU Turbo Deployment Guide](#).

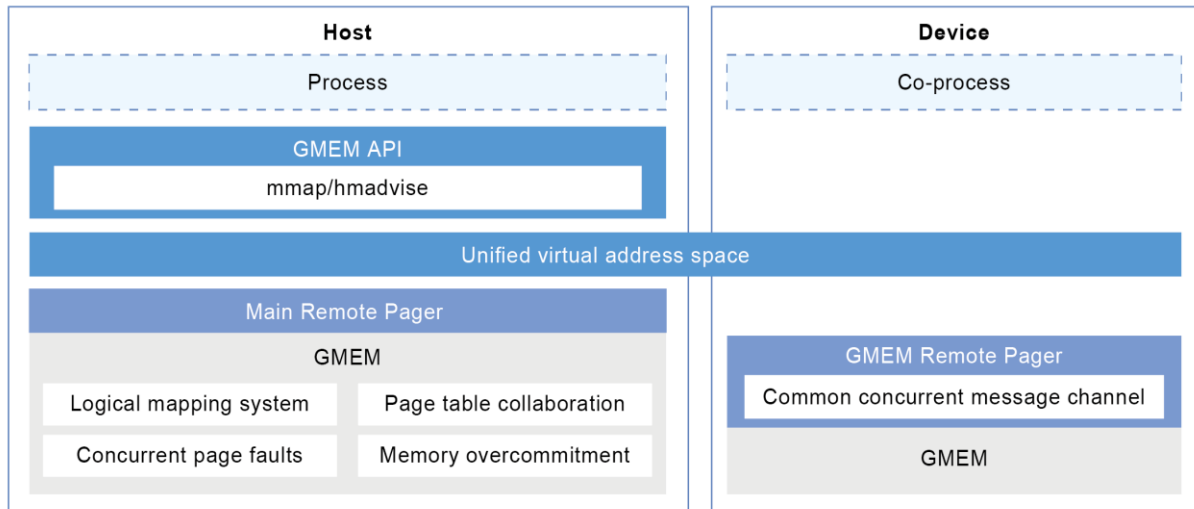
GMEM

In the post-Moore era, there have been breakthroughs in GPUs, TPUs, FPGAs, and other dedicated heterogeneous accelerators. Similar to CPUs, these devices increase computing speeds by storing data in local memory (such as LPDDR SDRAM or HBM), but such design catalyzes more complicated memory systems. Modern memory systems have the following defects:

- Memory management is split between CPUs and accelerators. Explicit data migration makes it difficult to balance the usability and performance of accelerators' memory.
- The high bandwidth memory (HBM) available on accelerators is often insufficient for foundation models. Manual swapping is only feasible in limited scenarios and typically results in significant performance degradation.
- A large number of invalid data migrations occur in search & recommendation and big data scenarios, and no efficient memory pooling solution is available.

Heterogeneous Memory Management (HMM) is a Linux feature that is plagued by issues of poor programming, performance, and portability, while also relying heavily on manual tuning. As such, it is unfavored by most OS communities, and has fueled demand for an efficient solution for heterogeneous accelerators. Generalized Memory Management (GMEM) is one new option, which offers a centralized management mechanism for heterogeneous memory connections. GMEM APIs are compatible with native Linux APIs, and feature high usability, performance, and portability. After an accelerator calls GMEM APIs to connect its memory to the unified address space, the accelerator automatically obtains the programming optimization capability for heterogeneous memory, and does not need to execute the memory management framework multiple times. This greatly reduces development and maintenance costs. Developers can apply for and release a unified set of APIs to achieve heterogeneous memory programming without memory migrations. If the HBM of an accelerator is insufficient, GMEM can use the CPU memory as the accelerator cache to transparently over-allocate the HBM without manual swapping. GMEM offers an efficient memory pooling solution thanks to a shared memory pool that eliminates the need for duplicate migrations.

Feature Description



GMEM enhances memory management in the Linux kernel. Its logical mapping system masks the differences between the ways how the CPU and accelerator access memory addresses. The Remote Pager memory message interaction framework provides the device access abstraction layer. In the unified address space, GMEM automatically migrates data to the OS or accelerator when data is to be accessed or paged.

- **GMEM**

GMEM leverages the computing power of both CPUs and accelerators. It combines the two independent address spaces of the OS and accelerators into a unified virtual address space, to realize unified memory management and transparent memory access.

GMEM uses a collection of new logical page tables to manage the unified virtual address space, ensuring data consistency between the page tables of different CPUs and micro architectures. Based on the memory access consistency mechanism of the logical page tables, the target memory space can be migrated between the host and accelerator using a kernel page fault handling procedure. If the accelerator's memory space is insufficient, the accelerator can borrow memory from the host and reclaim its cold memory to achieve memory overcommitment. This practice removes issues in which model parameters are restricted by the accelerator's memory, and reduces the cost of foundation model training.

GMEM high-level APIs in the kernel allow the accelerator driver to directly use memory management functions by registering the MMU functions defined in the GMEM specification. With memory management functions, the accelerator can create logical page tables and perform memory overcommitment. The logical page tables decouple the high-layer logic of memory management from the CPU's hardware layer, so as to abstract the high-layer memory management logic that can be reused by different accelerators. This design only requires the accelerator to register bottom-layer functions, and does not need high-layer logic for the unified address space. The OS automatically manages data migration across different storage media, such as DRAM and HBM.

- **Remote Pager**

Remote Pager is a memory message interaction framework that adopts message channel, process management, memory swap, and memory prefetch modules for the collaboration between the host and accelerator. It is enabled by the independent driver **remote_pager.ko**. The Remote Pager abstraction layer simplifies device adaptation by enabling third-party accelerators to easily access the GMEM system.

- User APIs

To allocate the unified virtual memory, users can directly use the mmap system call. GMEM adds the flag (**MMAP_PEER_SHARED**) of allocating the unified virtual memory to mmap.

The libgmem user-space library provides the madvise API of memory prefetch semantics that helps optimize the accelerator memory access. For details, visit <https://atomgit.com/openeuler/libgmem/blob/master/README.md>.

- Constraints

Hardware: Ascend NPUs are required.

Software: GMEM supports only 2 MB huge pages. Therefore, transparent OS huge pages must be enabled on hosts and NPUs to use GMEM. The heterogeneous memory obtained using **MAP_PEER_SHARED** cannot be inherited during fork.

For details about how to use GMEM, visit <https://atomgit.com/openeuler/docs-centralized/tree/master/docs/en/docs/GMEM>.

Application Scenarios

- Heterogeneous unified memory programming

To simplify heterogeneous memory programming, GMEM can allocate a unified virtual memory to CPUs and accelerators, which then share the same pointer. For example, an NPU memory management system can be connected to GMEM with just 100 lines of modified code in the NPU driver, a huge reduction on the original 4,000 lines in the memory management framework.

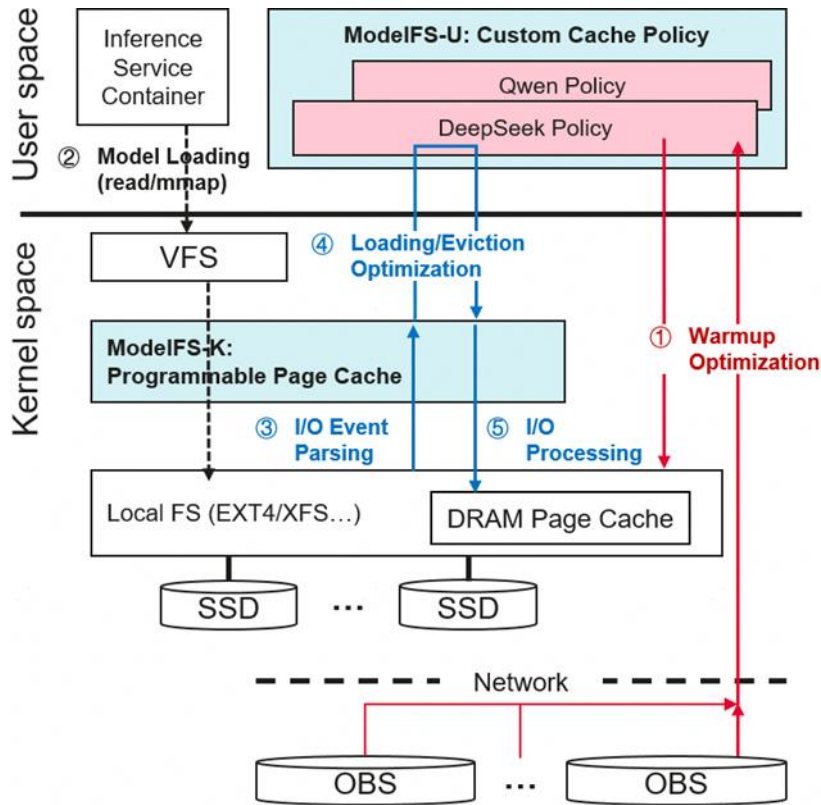
- Automatic memory overcommitment of an accelerator

When a GMEM API is used to allocate memory to an application, the application is not limited by the physical memory capacity of the accelerator. Instead, the application can transparently over-allocate memory until the CPU's DRAM capacity is exhausted. GMEM swaps cold device memory pages out to the CPU memory to achieve high-performance and easy-to-start training and inference. This design makes GMEM deliver 1.6 times the performance of NVIDIA-UVM in ultra-large model training when the overcommitment ratio is 200%. (Test results compared the Ascend 910 NPU and NVIDIA A100 GPU under the same HBM conditions.)

ModelFS

ModelFS addresses the model loading bottleneck during the startup phase of large language model (LLM) inference. While existing approaches often sacrifice compatibility for performance, real-world industrial deployment demands seamless integration with existing ecosystems. ModelFS bridges this gap by achieving state-of-the-art loading performance through optimized file system cache policies, without compromising compatibility. Specifically, it introduces a non-intrusive, flexible, and lightweight programmable page cache framework in the kernel, enabling users to customize file system page cache policies. Building on this framework, we provide a reference implementation optimized to accelerate model loading.

Feature Description



- **ModelFS-K**

The kernel module of ModelFS provides a programmable framework for the file system page cache. Its core design is a stacked file system that can be mounted on top of an existing file system. ModelFS-K redirects the logic of the underlying file system to user-space **prefetch()** and **evict()** functions via user-space procedure calls (UPCs).

- **ModelFS-U**

The user-space module of ModelFS provides the runtime environment for cache policies. It offers a VFS-like programming framework in user space, allowing model implementers and I/O optimizers to customize cache policies based on the I/O characteristics of model loading. Users need to implement the **init()**, **exit()**, **prefetch()**, and **evict()** functions. ModelFS-U registers these functions with ModelFS-K and is responsible for parsing I/O events as well as performing asynchronous data prefetching and eviction at runtime.

Application Scenarios

ModelFS can accelerate I/O operations across multiple types of storage backends, including local file systems, distributed file systems, and object storage service (OBS). The following are two typical use cases.

- **Remote OBS weight prefetching**

When model weights are stored in remote OBS, the inference system typically prefetches hotspot weights to local storage in advance due to the low bandwidth of OBS. ModelFS can accelerate weight prefetching by implementing a cache policy that aggregates I/Os across multiple OBS buckets.

- **Local SSD weight loading acceleration**

When model weights are stored on local SSDs, the performance bottleneck often arises from low bandwidth utilization. ModelFS addresses this issue with an I/O template-based mechanism. During the first run, it traces the I/O loading behavior of each inference service. In subsequent inference startups, it accurately predicts future I/O behavior and fully leverages SSD bandwidth and XPU affinity to accelerate prefetching and eviction.

NPU Compute Partitioning

Computing power is a key driver of AI advancement. To accelerate AI workloads, vendors have introduced various AI accelerators. However, most AI cards lack advanced scheduling capabilities such as priority control, fairness, preemption, and compute bandwidth management. As a result, they cannot effectively support mixed-workload scenarios for small and medium-sized AI models, such as online-online or online-offline inference co-location. Dedicated deployment of such models leads to low utilization of AI cards and significant resource waste.

Feature Description

xSched is a general-purpose scheduling framework designed for small and medium-sized models. It provides fundamental scheduling mechanisms for NPUs, including task preemption, time slicing, gang scheduling, compute bandwidth control, and memory capacity management. It also implements multiple scheduling policies, such as fair scheduling and real-time scheduling, to meet the requirements of different AI workload scenarios.

Application Scenarios

- In multi-inference workload co-location scenarios (for example, co-located online-online inference), xSched enables on-demand allocation of compute resources within a single NPU, addressing the over-provisioning issue caused by coarse-grained, card-level resource allocation.
- In training-inference co-location scenarios or online-offline inference co-location scenarios (where AI workloads are co-located with both online and offline workloads), xSched enables inference tasks to quickly preempt offline inference or training tasks. This improves NPU utilization while ensuring that the performance of high-priority tasks is not degraded.

Fast Fault Recovery

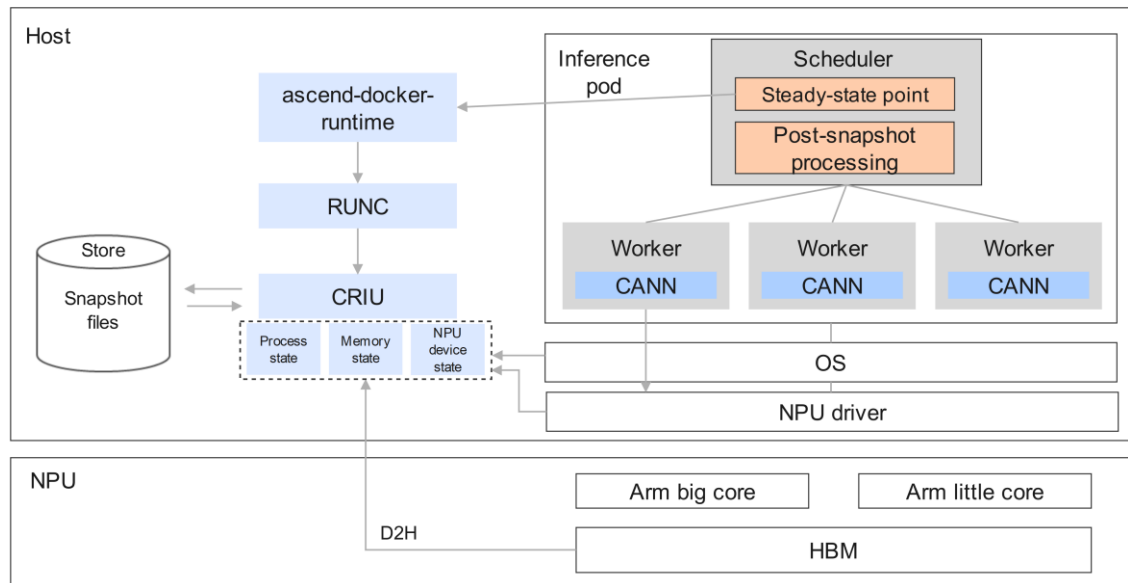
In today's AI inference production environment, an inference service is usually deployed in a containerized form and runs for a long time. However, a fault in a running process is inevitable. A conventional fault recovery path usually takes several minutes or even longer, resulting in a long service interruption and a waste of resources.

Feature Description

This feature enables second-level rapid fault recovery for AI inference containers, reducing recovery time from minutes to seconds. It is built on components that support process state dump and restore (for example, CRIU), with extended support for importing and exporting NPU device state, and is integrated with the Kubernetes ecosystem to achieve fast recovery of inference containers. The overall architecture is illustrated below.

- **Snapshot workflow:** The inference framework within the inference pod initiates a snapshot. It first invokes the NPU driver snapshot interface to save NPU memory and device states, which are then transferred to the host via device-to-host (D2H) transfer. It then invokes the snapshot interface, and ultimately invokes CRIU to create a snapshot and persist the complete host-side state of the inference pod.

- **Restore workflow:** Kubernetes triggers pod recovery. CRIU first restores the inference pod along with its host-side state, after which the NPU driver restore interface is invoked to recover the NPU memory and device state.



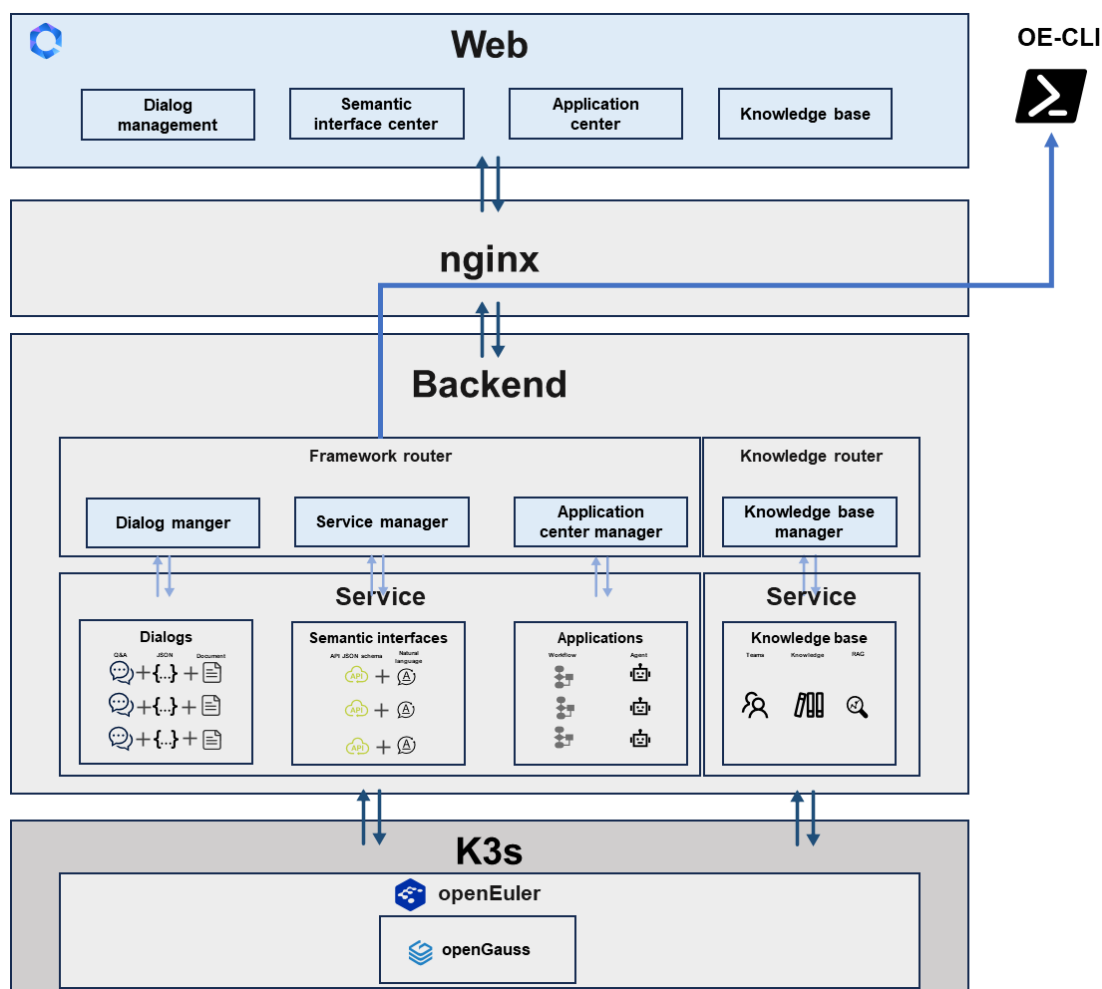
Application Scenarios

Container snapshots can be used for rapid fault recovery and elastic scaling of AI inference containers.

- **Rapid fault recovery:** In inference scenarios, container snapshots enable fast restoration of inference containers, minimizing service disruption time.
- **Elastic scaling:** Leveraging ultra-fast snapshot startup and integration with Kubernetes, inference containers can be scaled elastically.

AI for OS

AI makes openEuler more intelligent. Witty is an AI-powered Q&A platform built on openEuler data. It enables workflow orchestration through semantic interfaces and agent building through MCP. Additionally, it integrates some system services to further improve the intelligence of openEuler.



Intelligent Q&A

Feature Description

Witty is accessible via web or shell.

- **Web:** Through the visual web entry, users can easily build and use the knowledge base, run automated tests on the knowledge base, and view test evaluation results. They can also register OpenAPI APIs, build and use workflow applications based on them, register and activate MCP, and build and use MCP-based agent applications. The web entry helps beginners access openEuler knowledge and leverage openEuler's AI capabilities effortlessly.
- **Shell:** Through the shell entry, users can trigger intelligent Q&A or pre-integrated agent applications within the agent framework. The shell entry allows O&M personnel to interact with openEuler in an OS-affinity and intelligent manner, reducing O&M costs.

Intelligent Planning, Scheduling, and Recommendation

- **Intelligent planning:** The agent applications of Witty can plan steps in real time based on user input and available tools, continuing until the user's objective is achieved or the maximum number of steps is reached.

- **Intelligent scheduling:** Witty allows users to define multiple workflows within a workflow application. When a query request is made, Witty automatically extracts the relevant parameters and selects the most suitable workflow to execute the query task.
- **Intelligent recommendation:** Based on users' query requests and workflow execution results, Witty recommends workflows that may be useful in future tasks, increasing the likelihood of task completion and making applications easier to use.

Workflow Applications

- **Semantic interfaces:** A semantic interface contains natural language comments. Witty supports two methods for registering semantic interfaces. The first method allows users to register APIs with Witty as OpenAPI (3.0 or later) YAML files. When writing an OpenAPI YAML file, users can add natural language comments to the APIs. When these APIs are called, the foundation model selects the right APIs and sets their parameters based on the comments. The second method allows users to register functions with Witty as Python code, and this method provides a more user-friendly option for using Witty. The semantic interfaces generated in both methods can be orchestrated into workflows in a visualized manner.
- **Workflow orchestration and invocation:** Witty enables users to visually connect built-in semantic interfaces and user-registered interfaces to create workflows. Users can debug these workflows, and then release and use them as applications. When workflows are debugged and executed, intermediate results are displayed to help lower debugging costs and enhance the overall user experience.

Agent Applications

- **MCP registration, installation, and activation:** MCP is a mainstream AI-related protocol. It uses SDKs to encapsulate complex and diverse services with natural semantic information, allowing AI to easily invoke tools and services reconstructed based on MCP.
- **Agent building and use:** Witty allows building agents based on MCP and various foundation models. These agents can decompose a user's objective into phased tasks using the configured model information and the user-provided objective. MCP tools are then used to complete each task until the user's objective is achieved.

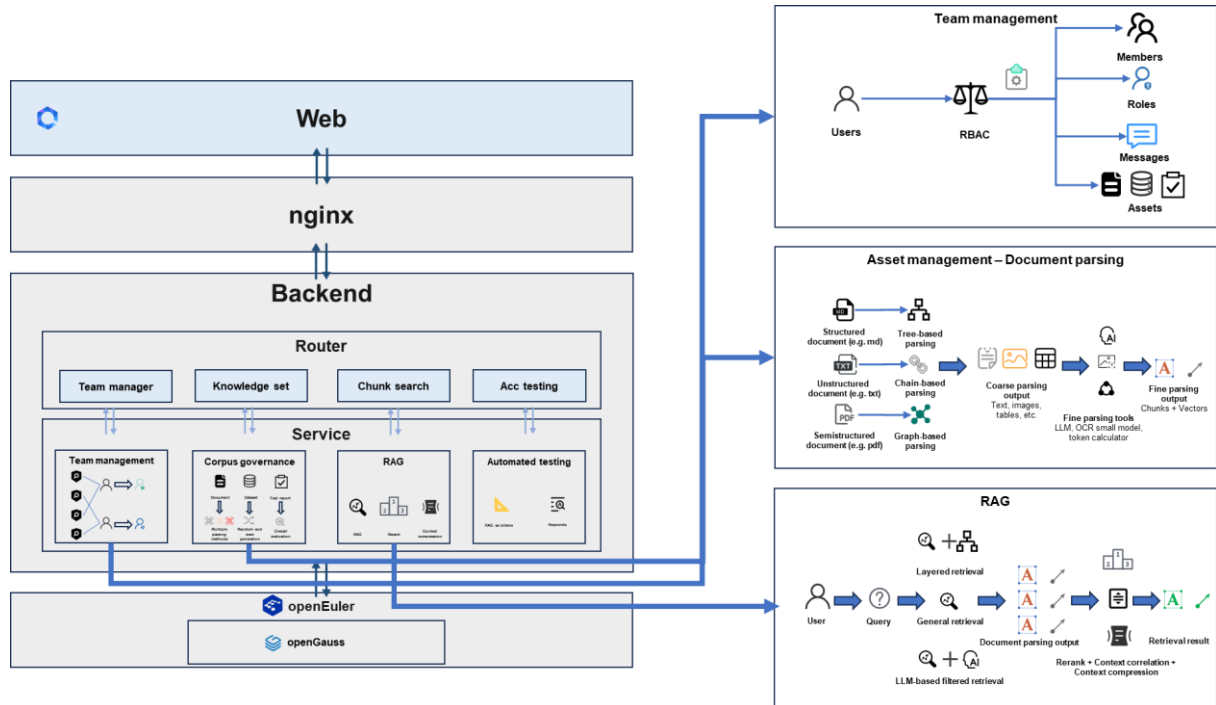
RAG

Retrieval-augmented generation (RAG) is a technique for enhancing the long-term memory capability of large language models (LLMs). Used by Witty, RAG is essential to reducing model training costs and has the following highlights:

- **Pre-processing for retrieval:** When a user's query request lacks sufficient information, it can be rewritten based on historical context and inferred intent to enhance retrieval accuracy.
- **Knowledge indexing:** For documents of various formats and content types, Witty offers document parsing capabilities such as summarization, text feature extraction, tree-structured parsing, and optical character recognition (OCR). These capabilities help generate segment-level feature indexes to increase hit rates.
- **Retrieval enhancement:** Witty supports eight retrieval methods. Among them, the methods using dynamic keyword weighting and LLM-based filtering significantly boost out-of-the-box accuracy.
- **Post-processing for retrieval:** For fragments with missing context, a uniform randomized context completion method is provided to enhance the completeness of fragment information and reduce hallucination in model fitting. For fragments (sets) with the maximum number of tokens, various methods are provided to enable intelligent Q&A for scenarios with limited resources. These methods include the reranking method based on the Jaccard distance,

token compression method that removes common words and applies random discarding, and token truncation method based on binary search.

These capabilities enable RAG for Witty to accommodate to more document formats and content types, and enhance Q&A services with minimal impact on system load.



Team Management

Team management is a fundamental capability of the RAG technology in Witty. It leverages role-based access control (RBAC) to manage team members' access to roles, users, and assets, thereby enhancing the overall usability of the knowledge base.

- **Roles:** This includes editing the atomic permissions of roles, adding, deleting, and modifying roles, and assigning roles to members. Atomizing these capabilities is central to instantiating team permissions.
- **Members:** This includes processing invitations and requests, as well as adding, deleting, modifying, and querying members. Atomizing these capabilities is central to managing team members and assigning permissions.
- **Assets:** This includes adding, deleting, modifying, and querying the asset library, as well as the documents, datasets, and testing results within it. Atomizing these capabilities is central to enabling on-demand access to asset data.

Corpus Governance

Corpus governance is one of the basic RAG capabilities in Witty. It imports corpuses into the knowledge base in a supported format using context location extraction, text summarization, and OCR, increasing the retrieval hit rate.

- **Context location extraction:** The relative location relationship within a document is retained. Specifically, the global and local relative offsets of each segment are recorded to support context completion.

- **Text summarization:** Sliding windows and LLMs are used to generate text summaries for complex documents or segments, supplying foundational data for multi-level retrieval.
- **OCR augmentation:** For documents containing both images and text, summaries are generated based on image text and surrounding context to provide foundational data for answering image-related questions.
- **Document source tracing:** During answer generation, Witty displays a note in the lower right corner of the corresponding sentence based on the returned segment and related document information. Users can click the note to view the document name and abstract, enhancing the reliability of question answering.

Automated Testing

Automated testing is a core RAG capability in Witty. It detects shortcomings in the knowledge base and retrieval augmentation algorithms through automated dataset generation and evaluation.

- **Dataset generation:** After a user selects a parsed document, Witty randomly samples a portion of the parsing results and sends them to an LLM. The LLM then generates and filters Q&A pairs to create a high-quality test dataset that contains queries, reference answers, and original segments.
- **Test evaluation:** Automated tests are conducted using the user-generated dataset and the predefined parameters such as the retrieval augmentation algorithm, LLM, and top-k fragments. Tests score are calculated based on the queries, reference answers, original fragments, associated fragments, and model fitting results in the test dataset. After that, the test results are evaluated using metrics such as the precision rate, recall rate, fidelity value, explainability, longest common substring score, edit distance score, and Jaccard similarity coefficient.

Application Scenarios

- Common openEuler users can build customized smart assistants using local files or documents from the openEuler community, such as white papers.
- openEuler developers can build custom workflows or agent applications using the web client, reducing repetitive work in certain scenarios, such as code audits.
- openEuler O&M personnel can interact with the OS using natural language through the shell, reducing the effort required to construct and modify complex commands.

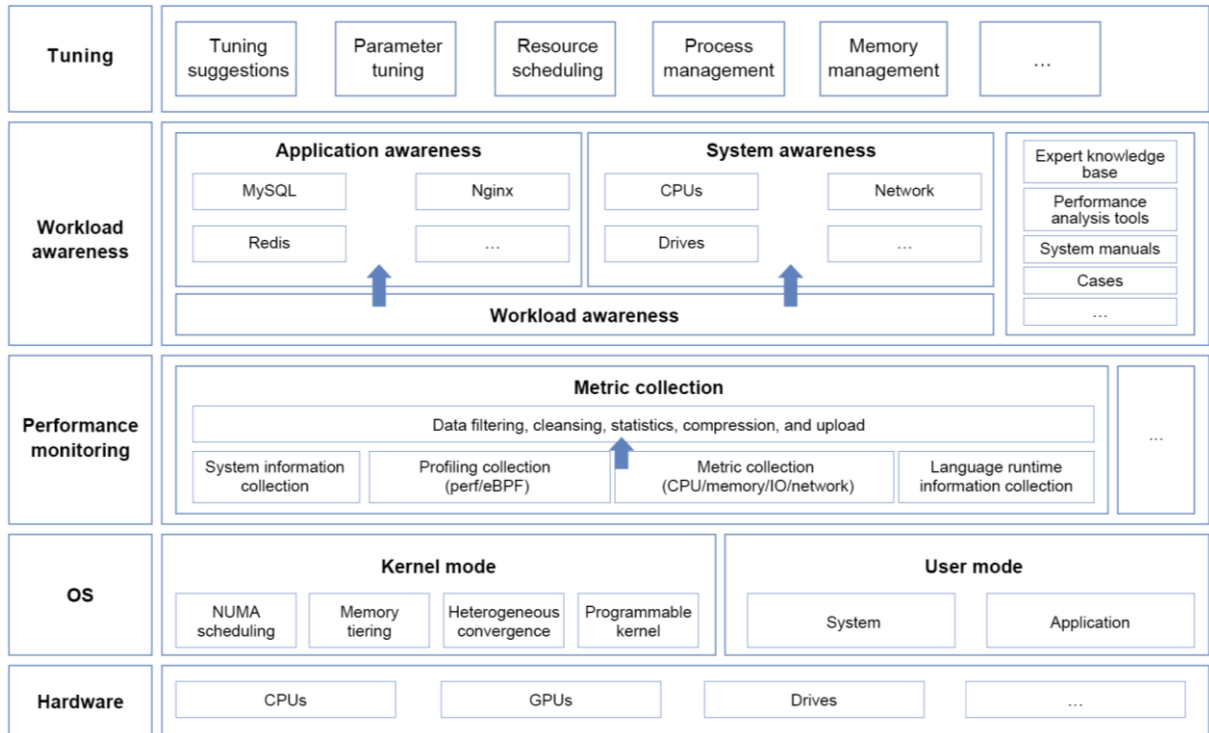
For details, see [Witty](#).

Intelligent Tuning

Feature Description

LLM-based System Parameter Optimization

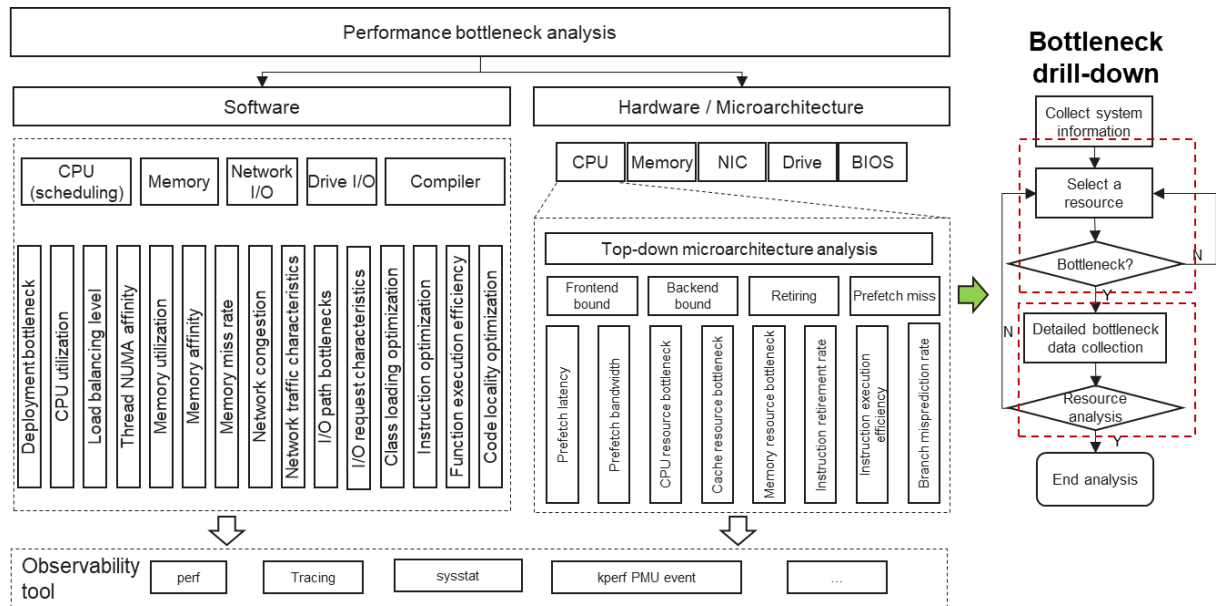
Witty supports the intelligent shell entry. Through this entry, users can interact with Witty using a natural language and perform heuristic tuning operations such as performance data collection, system performance analysis, and system performance tuning. The MCP protocol can be used to detect tuning intents.



SKILL-based Top-Down Performance Bottleneck Identification

This capability performs top-down root-cause analysis of performance bottlenecks using an evidence-chain approach: starting from coarse-grained bottleneck metrics, analyzing multiple potential causes, progressively collecting fine-grained data, and validating hypotheses to construct a complete bottleneck chain in the context of the actual system state. Guided by key metrics defined in SKILL, the system covers a wide range of common OS-level performance bottlenecks and provides targeted optimization recommendations.

It integrates with multiple native OS-level performance analysis interfaces and tools, enabling multi-layer data collection and top-down bottleneck localization. It further supports drill-down analysis across five major categories: scheduling, locking, I/O, memory, and networking.

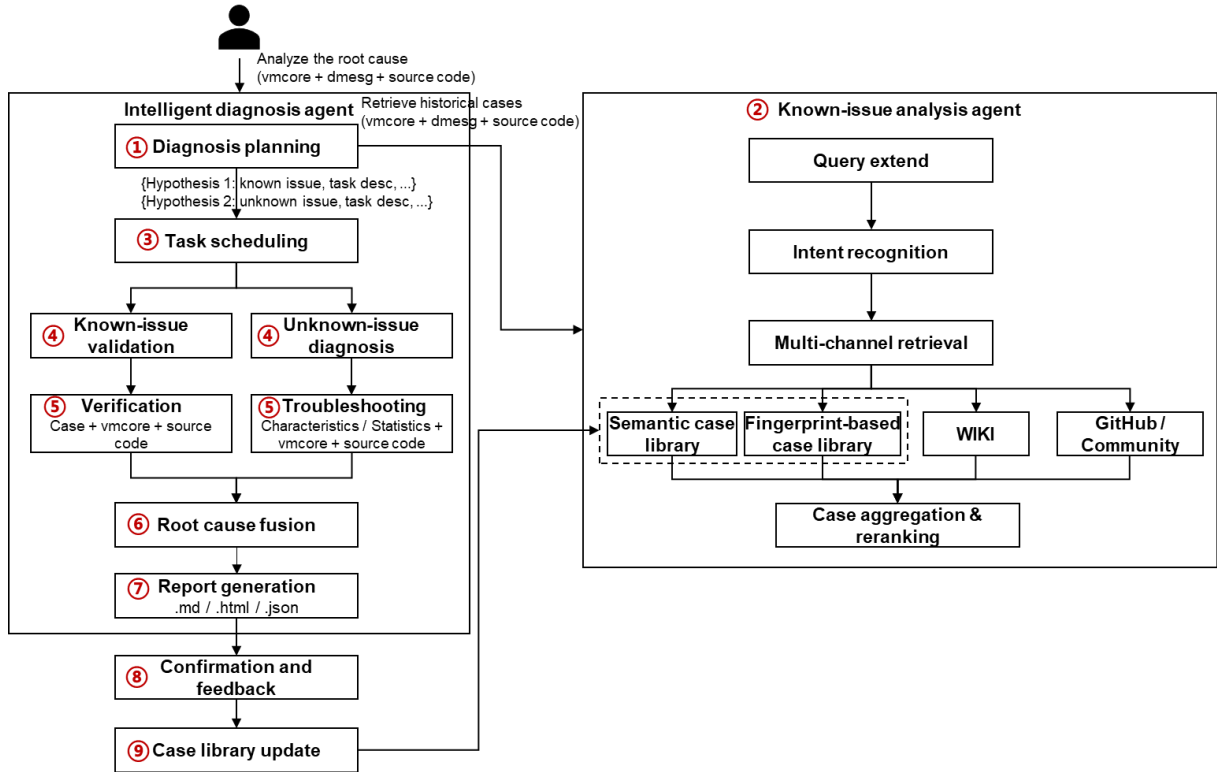


Application Scenarios

- **Gaining insights from key performance metrics:** Users can learn about the system performance status based on collected performance metrics like CPU, I/O, drive, network, microarchitecture, and application.
- **Analyzing system performance:** Performance analysis reports are generated, making it easier to locate performance bottlenecks across the entire system and in individual applications.
- **Receiving performance tuning suggestions:** Based on current system performance, Witty can recommend appropriate application or system parameters, which are automatically applied. It can also generate a one-click performance tuning script to optimize both the system and specific applications.

Intelligent Diagnosis

Intelligent System Crash Diagnosis



Feature Description

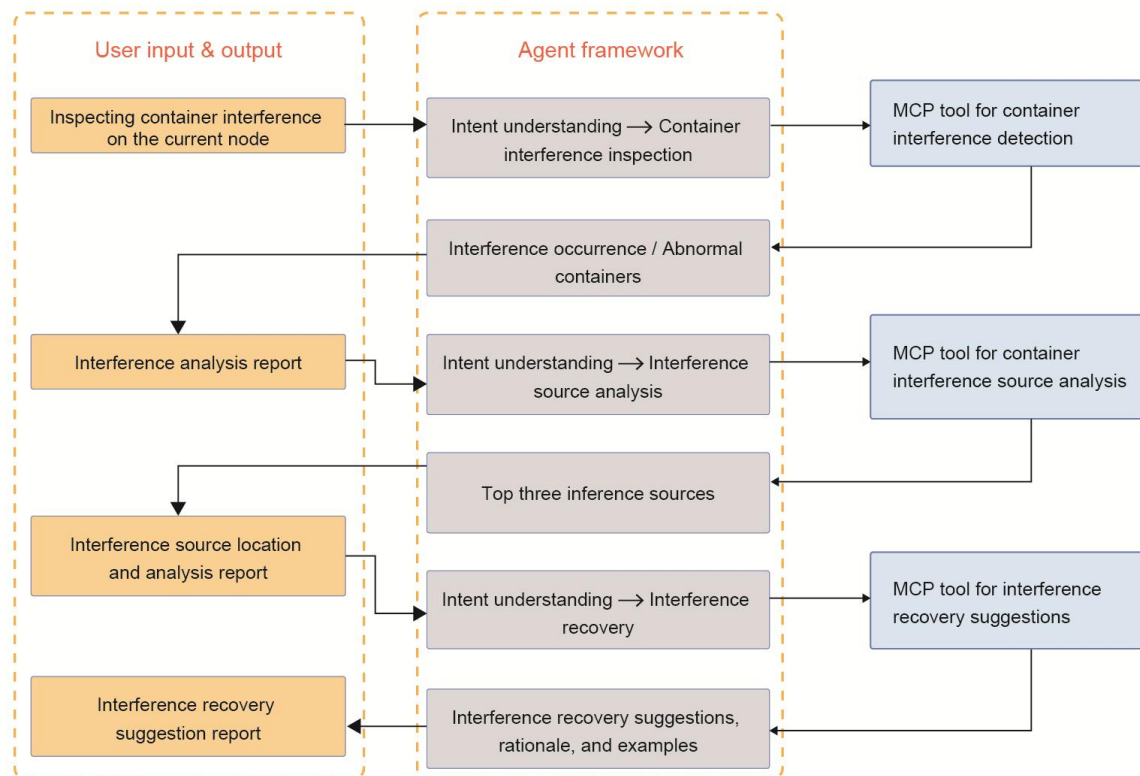
- **Intelligent diagnosis agent:** The intelligent diagnosis agent adopts a hypothesis generation and parallel validation paradigm. It takes multi-dimensional inputs, including dmesg logs, kernel source code, and fault descriptions, and leverages a known-issue analysis agent to retrieve historical cases. It concurrently generates and evaluates two categories of hypotheses: known faults and unknown faults. For known fault hypotheses, it validates matches by comparing the current runtime data with historical cases in terms of analysis workflow and diagnostic signatures. For unknown fault hypotheses, it automatically invokes the crash tool and performs source code-level analysis to infer the underlying root cause. The agent then consolidates results from both paths and automatically generates diagnostic reports in HTML, Markdown, and JSON formats. These reports comprehensively include root cause conclusions, fault timelines, fault propagation chains, causal relationships, complete troubleshooting procedures, and remediation recommendations.
- **Known-issue analysis agent:** This agent features a built-in intelligent log parsing engine that automatically extracts critical anomaly signals from logs, including error codes, warning messages, kernel stack traces, and register states. It integrates lightweight retrieval-augmented generation (RAG) and hybrid search strategies to perform precise matching across multiple sources, including a semantic case base, fault fingerprint database, knowledge base wikis, and the GitHub open source community. This enables rapid retrieval of similar historical incidents, providing strong prior evidence for efficient end-to-end troubleshooting.
- **Case accumulation and continuous learning:** Verified and user-confirmed diagnostic reports, along with original fault descriptions and raw runtime data, are automatically

standardized and transformed into structured cases for ingestion into the semantic case base. By continuously enriching high-quality fault samples, the system builds a self-improving data flywheel, progressively enhancing retrieval accuracy and hypothesis generation quality. This results in a positive feedback loop: the more the system is used, the faster and more accurate fault diagnosis becomes.

Application Scenarios

When a kernel crash occurs on a production server—such as OOM, null pointer dereference, deadlock, soft lockup, or hardware exception—and a vmcore dump is generated, O&M engineers no longer need to manually use the crash tool or analyze massive logs line by line. By uploading the dump file, system logs, and fault symptoms, the combination of the intelligent diagnosis agent and known-issue analysis agent can automatically perform end-to-end diagnosis. This replaces hours of manual debugging and enables rapid incident response and mitigation.

Container Interference Detection



Feature Description

- **Interference detection:** The container interference detection tool monitors nodes over a specified period, identifies interference, and reports the affected containers along with the corresponding metrics.
- **Interference source analysis:** The container interference source analysis tool identifies all affected containers on a node and reports the top three suspected interference source containers, along with the associated metrics (such as CPU run delay) for each affected container.

- **Interference recovery suggestion:** Based on the results of interference source analysis, the interference recovery suggestion tool analyzes the interference and generates a report with recommended recovery actions, including the rationale and example commands for each suggestion.

Application Scenarios

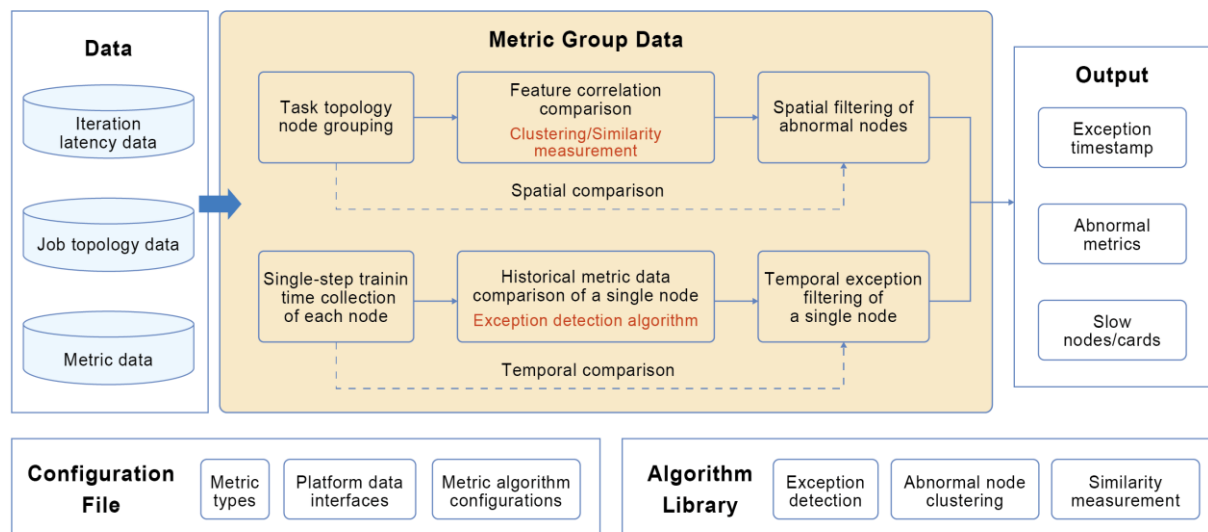
The intelligent diagnosis interface in openEuler 24.03 LTS SP4 provides three key functions: container interference detection, interference source analysis, and interference recovery suggestion generation.

- Container interference detection identifies interference in monitored container metrics on a single node and reports the affected containers.
- Interference source analysis determines the root causes based on detection results and node monitoring metrics, outputting the top three contributing metrics for each affected container.
- Interference recovery suggestion generation produces a report with recommended recovery actions based on the results of interference source analysis.

AI Cluster Slow-Node Demarcation

Slow node detection in AI clusters requires both real-time responsiveness and intelligent analysis. The agent leverages time-series analysis algorithms provided by sysTrace to identify slow nodes and underperforming cards in AI training workloads. Compared with traditional approaches, it delivers higher accuracy and broader applicability. In addition, users can interact with the agent using natural language to automatically diagnose and analyze a specific cluster or a subset of nodes, enabling efficient and precise identification of slow nodes.

Feature Description



The core capability of sysTrace is time series analysis, which provides the following functions:

- **Configuration file:** Contains the types of metrics to be observed, configuration parameters for the metric algorithms, and data interfaces, which are used to initialize the slow node detection algorithms.

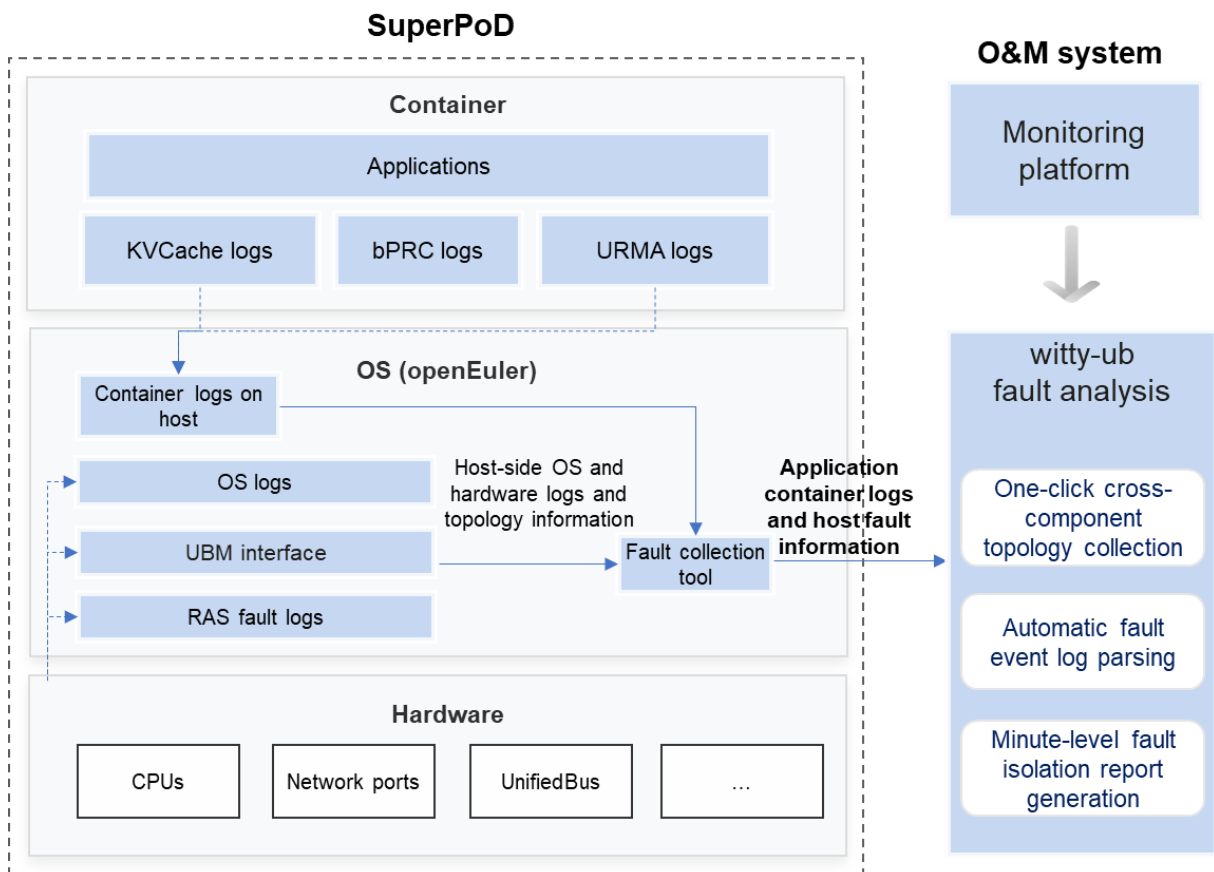
- **Algorithm library:** Includes common time series exception detection algorithms, such as dbscan, k-sigma, abnormal node clustering, and similarity measurement.
- **Data:** Metric data collected from each node includes Python stack traces, I/O-related information, and HCCL operator activity traces.
- **Grouped metric comparison:** Supports spatial filtering of abnormal nodes and temporal exception filtering of a single node. Spatial filtering identifies abnormal nodes based on the exception clustering algorithm, while temporal exception filtering determines whether a node is abnormal based on the historical data of the node.

Application Scenarios

sysTrace detects slow-card and slow-node anomalies based on XPU-collected data and raises alerts.

AI model training: This feature quickly detects slow nodes for distributed training jobs in large-scale AI clusters, facilitating system self-healing and fault rectification by O&M personnel.

SuperPoD Connectivity and Latency Issue Isolation



Feature Description

After a UnifiedBus SuperPoD fault occurs, the system analyzes observability data such as logs and metrics to infer the faulty component. This is critical for identifying responsible components and accelerating fault recovery. This feature targets fault isolation scenarios for communication

latency and connectivity issues in SuperPoD KVCache and URMA, and provides the following capabilities:

- **One-click cross-component topology collection:** Based on information from UBM diagnostic interfaces and component logs, the system collects communication resource topologies with a single click and aggregates global topology information. This supports fault impact analysis and enables efficient fault isolation.
- **Automatic fault event log parsing:** Communication fault logs are parsed in an event-centric manner. The system automatically extracts fault metadata and correlates upstream and downstream component logs, significantly reducing the volume of logs to analyze and focusing on key fault signals.
- **Minute-level fault isolation report generation:** Based on topology collection and log parsing results, the system identifies communication latency and connectivity issues. By leveraging a SuperPoD fault case library and component knowledge, it automatically generates fault isolation reports within minutes, enabling rapid fault isolation.

Application Scenarios

This capability targets SuperPoD KVCache and URMA communication scenarios, providing SuperPoD system O&M personnel with latency and connectivity issue isolation capabilities. It supports two usage modes: standalone deployment and integration with external O&M systems.

- **Standalone usage:** Provides a complete and user-friendly GUI. After starting frontend and backend services, users can perform data upload, query, and analysis directly through the interface.
- **Integration with external O&M systems:** Provides RESTful northbound APIs. After starting backend services, users can invoke the APIs to access core functionalities and integrate them into external O&M systems.

Intelligent Container Image Pulling

Feature Description

Witty can invoke environment resources through a natural language, assist in pulling container images for local physical resources, and establish a development environment suitable for debugging on existing compute devices.

This system supports three types of containers, and container images have been released on Docker Hub. Users can manually pull and run these container images.

- **SDK layer:** encapsulates only the component libraries that enable AI hardware resources, such as CUDA and CANN.
- **SDKs + training/inference frameworks:** accommodates TensorFlow, PyTorch, and other frameworks (for example, tensorflow2.15.0-cuda12.2.0 and pytorch2.1.0.a1-cann7.0.RC1) in addition to the SDK layer.
- **SDKs + training/inference frameworks + LLMs:** encapsulates several models (for example, llama2-7b and chatglm2-13b) based on the second type of containers.

The following table lists the container images supported by Witty:

Registry	Repository	Image Name	Tag
docker.io	openeuler	cann	8.0.RC1-oe2203sp4
			cann7.0.RC1.alpha002-oe2203sp2

Registry	Repository	Image Name	Tag
docker.io	openeuler	oneapi-runtime	2024.2.0-oe2403lts
docker.io	openeuler	oneapi-basekit	2024.2.0-oe2403lts
docker.io	openeuler	llm-server	1.0.0-oe2203sp3
docker.io	openeuler	mlflow	2.11.1-oe2203sp3
			2.13.1-oe2203sp3
docker.io	openeuler	llm	chatglm2_6b-pytorch2.1.0.a1-cann7.0.RC1.alpha002-oe2203sp2
			llama2-7b-q8_0-oe2203sp2
			chatglm2-6b-q8_0-oe2203sp2
			fastchat-pytorch2.1.0.a1-cann7.0.RC1.alpha002-oe2203sp2
docker.io	openeuler	tensorflow	tensorflow2.15.0-oe2203sp2
			tensorflow2.15.0-cuda12.2.0-devel-cudnn8.9.5.30-oe2203sp2
docker.io	openeuler	pytorch	pytorch2.1.0-oe2203sp2
			pytorch2.1.0-cuda12.2.0-devel-cudnn8.9.5.30-oe2203sp2
			pytorch2.1.0.a1-cann7.0.RC1.alpha002-oe2203sp2
docker.io	openeuler	cuda	cuda12.2.0-devel-cudnn8.9.5.30-oe2203sp2

Application Scenarios

- **Common openEuler operations:** Simplify the process of building a deep learning development environment while saving physical resources. For example, set up an Ascend development environment on openEuler.
- **openEuler development:** Developers familiarize themselves with the openEuler AI software stack to reduce the trial-and-error cost of installing components.

SuperPoDs

The exponential growth in computing power requirements, coupled with breakthroughs in high-speed interconnect technologies, is driving the evolution of hardware from standalone nodes to SuperPoDs. Going forward, the core requirements for data, resources, and services will exhibit the following characteristics:

- **Resource pooling:** Compute, memory, interconnect, and storage resources can be pooled to support many-to-many collaboration.

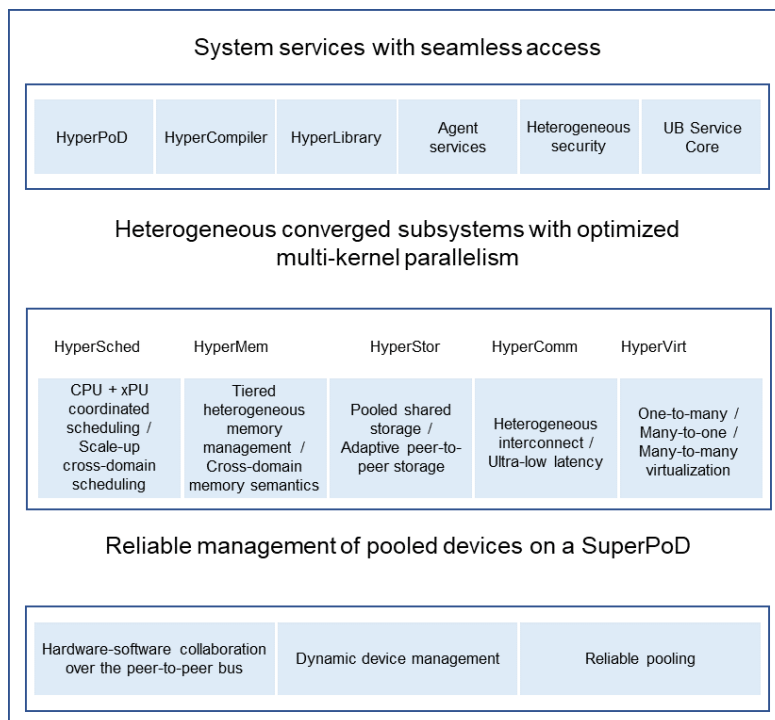
- **Scale expansion:** On-demand, flexible expansion is supported, ensuring high utilization. A unified addressing mechanism is used across the entire system, reducing static transfer and dynamic latency.
- **Long-term stability and reliability:** The system's long-term stable runtime is improved tenfold. It supports automatic recovery, is easy to deploy, and adapts to various equipment room environments. The system can also be expanded from modules to full cabinets.

To meet the requirements for SuperPoD scheduling, pooling, communication, and virtualization, openEuler has been upgraded to support SuperPoDs, unleashing heterogeneous computing.

SuperPoD OS Architecture

During the implementation of SuperPoD applications, the industry faces three core challenges: streamlining the development process and enabling smooth migration of services with zero modifications, accurately matching diversified computing power supply and achieving on-demand, efficient resource release, and managing increasing system complexity while ensuring high application availability. The openEuler heterogeneous convergence system addresses the preceding challenges as follows:

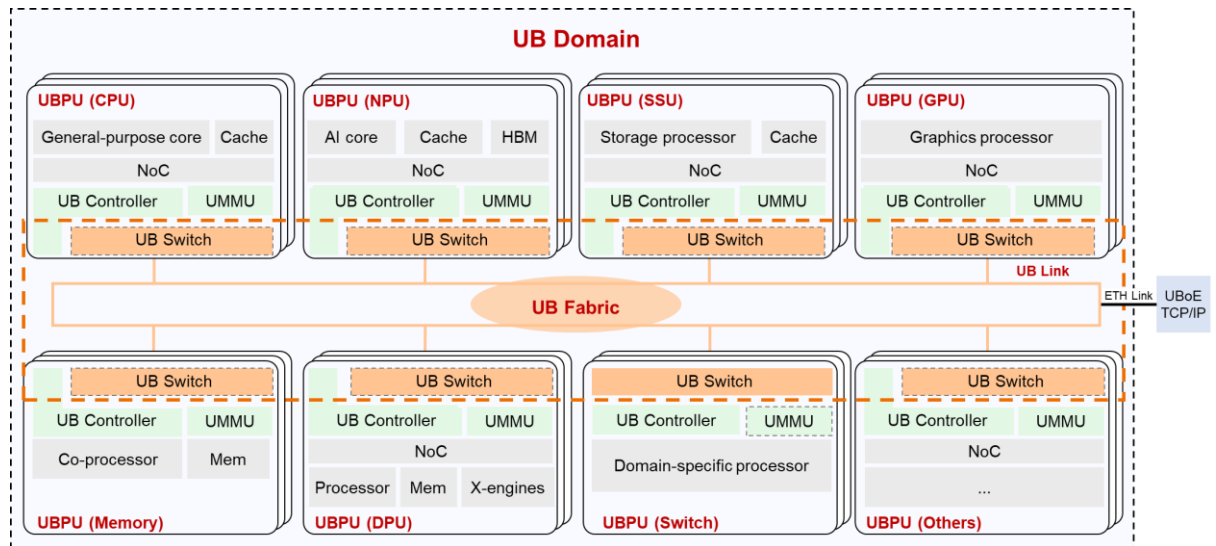
- It introduces advanced system services to streamline service migration, enabling zero-modification deployment and greatly reducing development and migration costs.
- It strengthens the core subsystems for heterogeneous convergence, providing communication and virtualization capabilities that accurately match computing power requirements and maximize resource utilization.
- It enhances device pooling to support flexible SuperPoD expansion. Reliable pooling prevents fault propagation and ensures stable service operation.



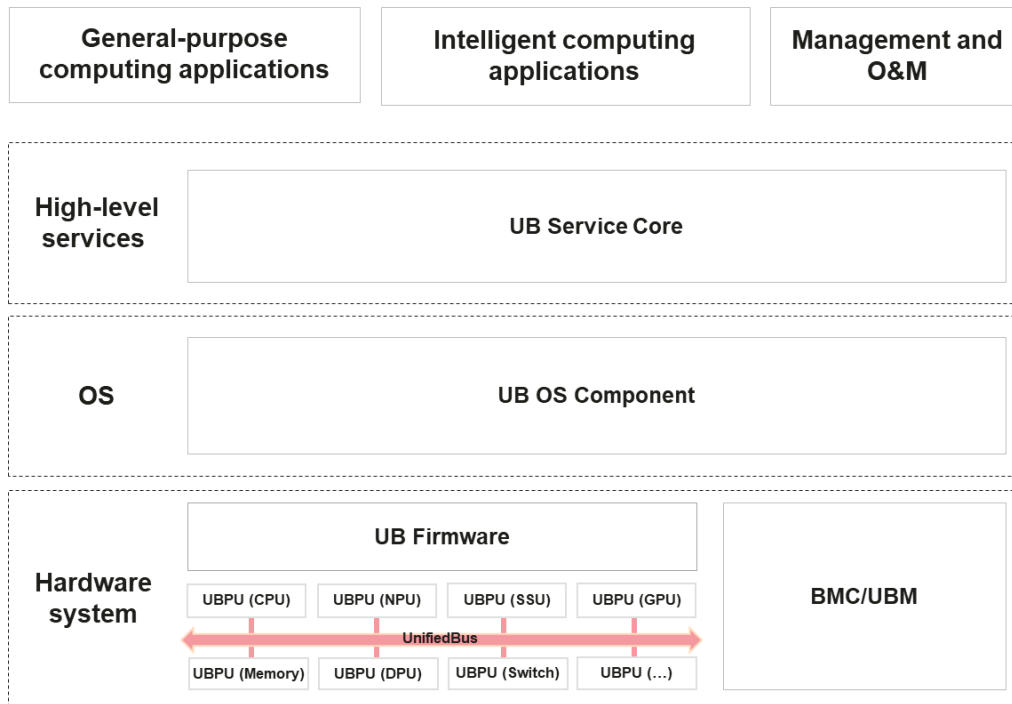
OS Implementation for SuperPoDs via UnifiedBus

UnifiedBus Architecture

SuperPoDs are the core of UnifiedBus-powered computing systems. They redefine computing systems, break the boundaries of computing hosts, and enable the expansion of intelligent and general-purpose computing capabilities, thereby achieving substantial performance gains. UnifiedBus represents the target computing system architecture for the future AI era. The following figure shows the UnifiedBus architecture.



Within a SuperPoD, UnifiedBus provides an ultra-low-latency, unified multi-protocol interconnect. It fully pools and connects computing resources via peer-to-peer links, enabling flexible resource composition, ultra-large-scale networking, and high system availability. The following figure shows the overall reference architecture of a UnifiedBus-powered computing system.



This reference consists of four layers:

- **The UnifiedBus hardware implements composable computing for SuperPoDs.**
 - **UnifiedBus:** It forms the foundation of a UnifiedBus-powered SuperPoD computing system. UnifiedBus provides bus-level unified interconnect, peer-to-peer collaboration, full resource pooling, large-scale networking, and high availability for computing clusters.
 - **UB Fabric Manager (UBFM):** It completes the UnifiedBus interconnect configuration, enables flexible and efficient aggregation of computing resources, optimizes computing interconnect SLAs (including latency, bandwidth, and reliability), and delivers flexible SuperPoD computing power.
- **UB OS Component enables UnifiedBus and devices, and provides unified abstraction and management.**
 - UB OS Component extends the Linux kernel to natively support SuperPoDs, providing unified abstraction and management of UnifiedBus devices.
 - It also maintains compatibility with POSIX interfaces to enable fast migration of existing applications.
- **UB Service Core enables peak SuperPoD performance.**
 - It provides unified management and scheduling of heterogeneous computing resources such as memory, SSUs, and NPUs, enabling new scale-up capabilities while simplifying resource management and scheduling complexity. It also delivers capabilities for scenarios including memory pooling, communication, I/O, and virtualization, supports multiple modes of ecosystem integration, and fully unlocks the advantages of the SuperPoD architecture.

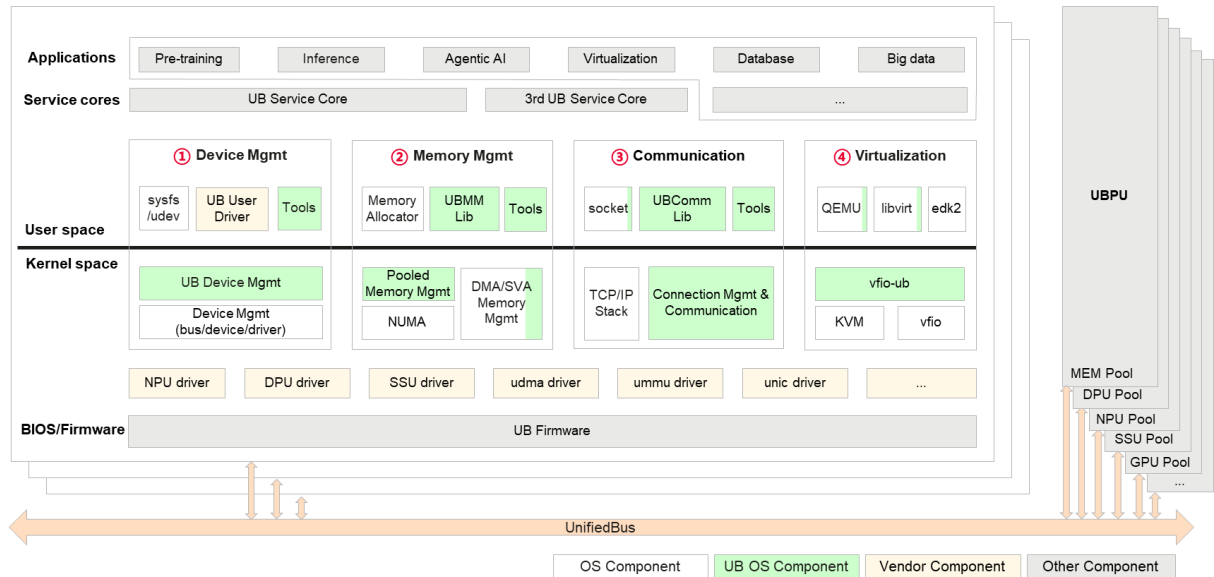
Application Scenarios

SuperPoDs, built on UnifiedBus, support 10 core service scenarios for AI and general-purpose computing. These include foundation model pre-training, central inference, post-training and reinforcement learning, multi-modal content understanding and generation, agentic AI,

virtualization, big data, databases, software-defined storage, and HPC. It provides leading system capabilities, improving computing service performance and resource utilization.

For details, see the *UnifiedBus™ (UB) SuperPoD Reference Architecture White Paper* at the [UnifiedBus community](#).

UB OS Component



UB OS Component enhances the original OS framework of memory management, communication, device management, and virtualization to provide improved support for UB. The feature enhancements include:

- Device Mgmt:** It provides UB device management capabilities, allowing hot plugging and configuration of UB devices on compute nodes. It consists of the following modules:
 - UB Device Mgmt:** Based on Linux's device management model (bus/device/driver), this module provides extended management for UB devices. It manages UB drivers, UB firmware interactions, and UB device hot plugging. In addition, it provides device driver interfaces for developing UB device drivers.
 - sysfs:** UB Device Mgmt generates UB device, driver, and bus information in sysfs. Users can view and use the information through sysfs.
 - udev:** UB Device Mgmt generates uevents. Users can use the udev tool to obtain the hot plug status of UB devices and manage these devices accordingly.
 - UB User Driver:** It is the user-space driver for UB devices. It exposes UB devices to user space, enabling applications and virtualization software to use these devices.
- Memory Mgmt:** It enables memory-semantic access for memory borrowing and sharing across compute nodes and devices within UB Domain. It consists of the following modules:
 - DMA/SVA Memory Mgmt:** Based on Linux's direct memory access (DMA) and shared virtual addressing (SVA) features, this module manages UB Memory Management Unit (UMMU). Drivers and other components can register host memory with UB devices using this module.
 - Pooled Memory Mgmt:** Its functions include exporting local memory, importing remote memory, and maintaining shared memory consistency. Based on Linux's NUMA memory management framework, Pooled Memory Mgmt can bind remote memory to NUMA

nodes. When it instead binds remote memory to memory devices (character devices created under */dev*), the remote memory can be accessed through mmap calls.

- **UBMM Lib:** It encapsulates the interfaces provided by Pooled Memory Mgmt in user space. UB Pooled Resource Manager (UBPRM, see Note 1) can call these interfaces to manage memory on both the Home and User sides.
- **Memory Allocator:** It is compatible with common memory allocators including glibc, jemalloc, tcmalloc, and libnuma. After UBPRM borrows memory from other compute nodes and binds it to NUMA nodes through UBMM Lib, applications can allocate or release the borrowed memory using standard memory management interfaces such as malloc, free, and numa_alloc_onnode provided by these common allocators, without requiring any application modifications.
- **Communication:** It implements communication and remote procedure calls (RPCs) across compute nodes and UB devices. It consists of the following modules:
 - **Connection Mgmt & Communication:** They manage asynchronous communication connections and provide kernel-space asynchronous communication interfaces for other modules.
 - **UBComm Lib:** It provides interfaces for enabling asynchronous communication and Unified Remote Procedure Call (URPC) within UB. Applications can use these interfaces to perform inter-node communication and function invocation within a SuperPoD.
 - **Socket:** It is compatible with glibc and other libraries that provide socket interfaces. Applications on different nodes can communicate with each other through socket interfaces without modification or with minimal modification (see Note 2).
- **Virtualization:** It enables UB devices to be passed through to VMs. It consists of the following modules:
 - **QEMU:** It extends QEMU functions to virtualize UB Controller, UB devices, and UMMU.
 - **libvirt:** It extends libvirt functions to support UB device creation and deletion.
 - **vfio-ub:** It supports UB device virtualization based on Linux's Virtual Function I/O (VFIO) framework.

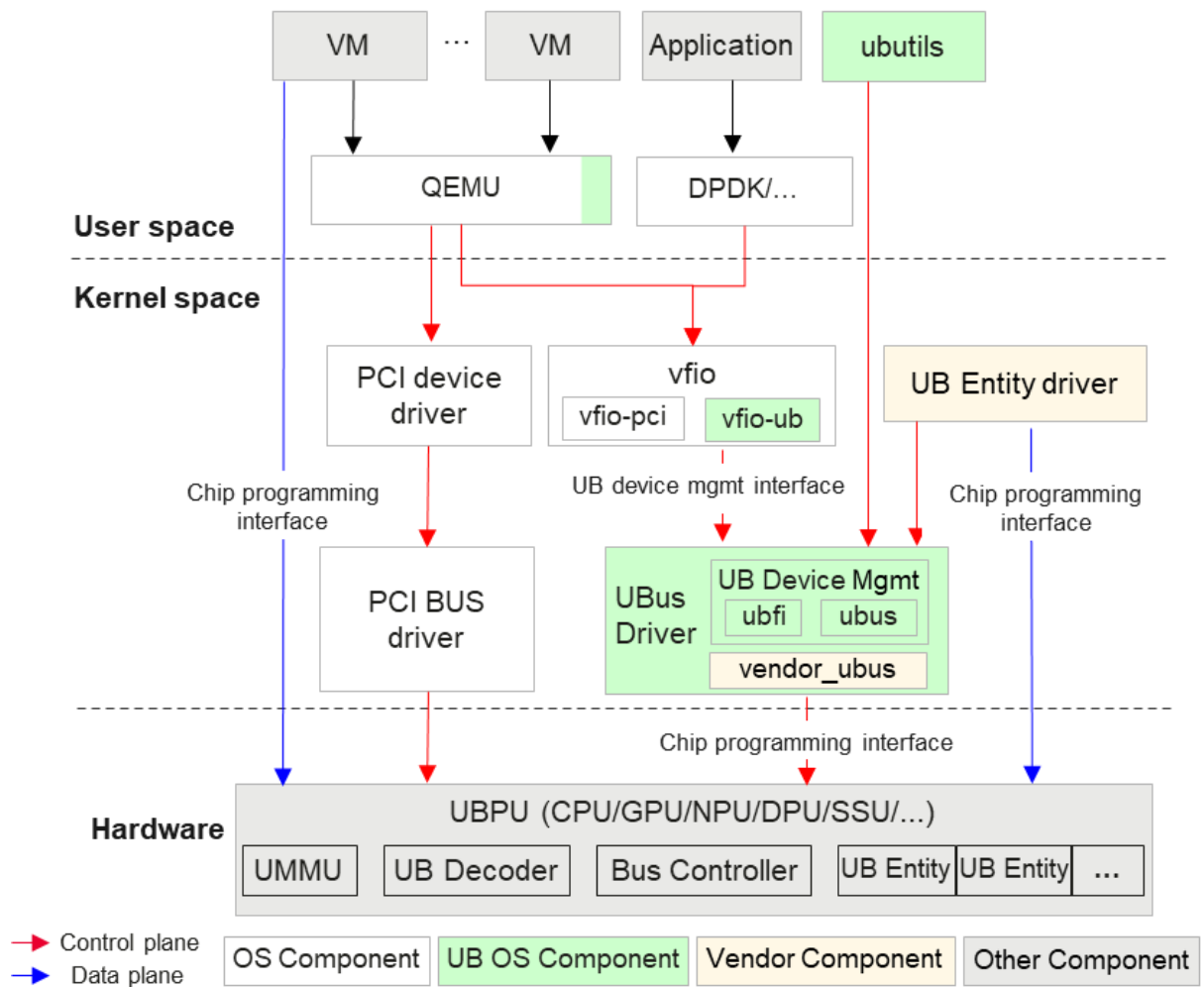
For details about the functions and code of the UnifiedBus components, see the *UnifiedBus™ (UB) Software Reference Design for Operating Systems* at the [UnifiedBus community](#).

Device Mgmt

UB device management involves UBus Driver, vfio-ub, and ubutils, which together provide a set of UB device service management interfaces. These services include device discovery, device registration, and interrupt enabling for UB device drivers; user-space passthrough to UB devices; and querying UB device information and configurations. Various UB devices can be registered with UB to perform their functions.

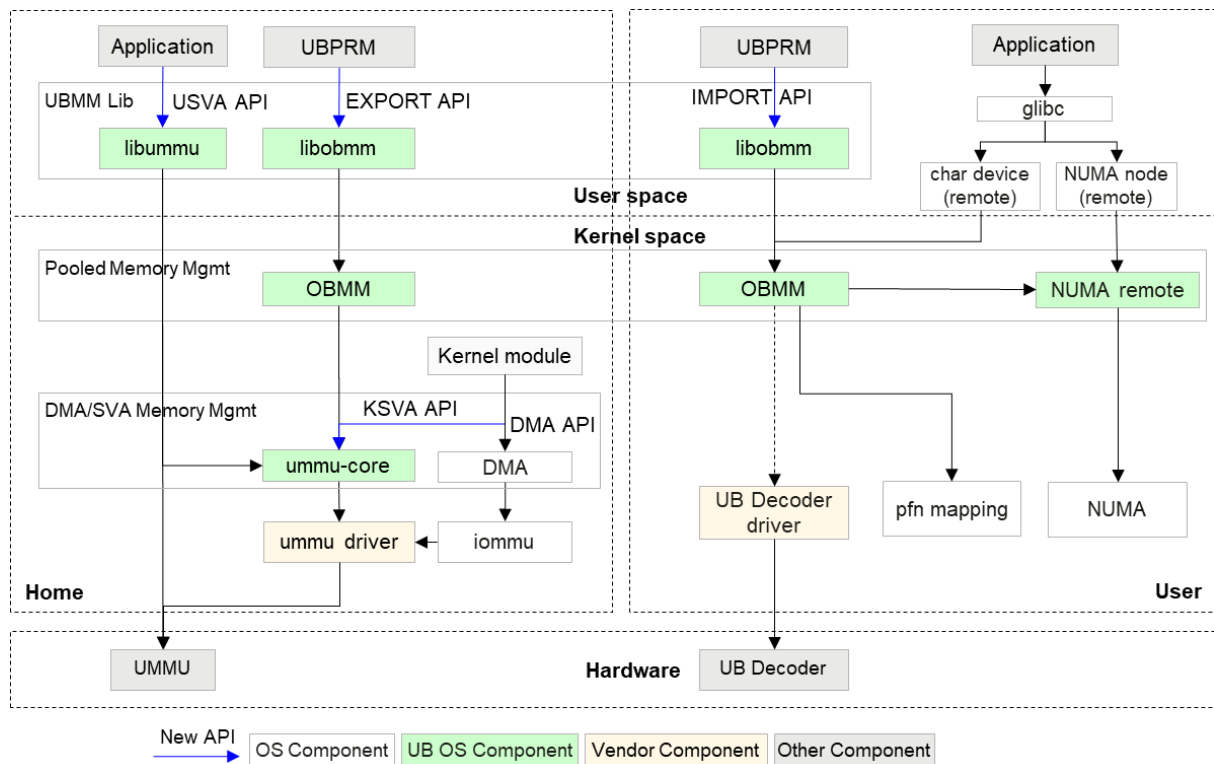
In the OS, UB device management functions, such as device discovery, registration, and driver loading, can be used in both single-server and cluster deployments.

- **Single-server scenarios:** Each server operates independently and exclusively manages its local UB devices. UBus Driver discovers and enables these devices, while vfio-ub enables user-space passthrough for direct device access.
- **Cluster scenarios:** In a SuperPoD, UBus Driver discovers and enables UB Entities allocated to compute nodes, supports the addition and removal of pooled devices, and works with vfio-ub to enable user-space passthrough for direct device access.



Memory Mgmt

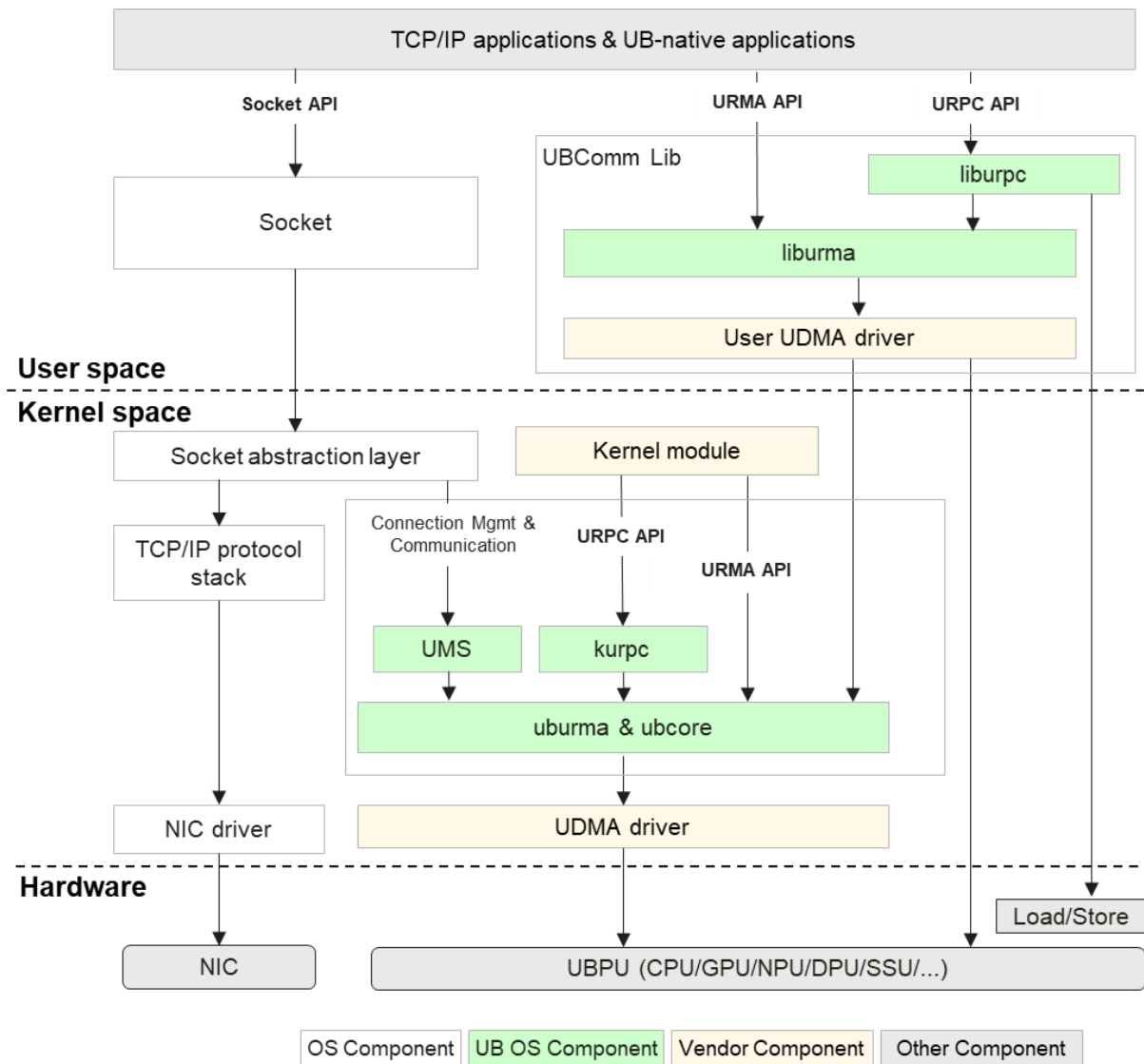
In general, existing bus architectures follow a host-device model, where the host (CPU) manages each device and three independent memory access paths exist: host-to-device, device-to-host, and device-to-device. UB streamlines this model. It categorizes UB nodes as Users and Homes in the memory access model. Users can access the memory resources of Homes either synchronously (via load/store) or asynchronously (via DMA).



Based on UnifiedBus capabilities, the Ownership-Based Memory Management (OBMM) module provides inter-node data path configuration and cross-node data consistency maintenance. After the configuration is complete, users can access the memory of another node from within a UnifiedBus cluster, enabling cross-node memory sharing, memory pooling, and inter-node memory borrowing.

Communication

UnifiedBus Memory Development Kit (UMDK) is a distributed communication software library that provides high-performance communication interfaces for inter-card communication within data center networks, SuperPoD environments, and servers, fully utilizing UB hardware capabilities.



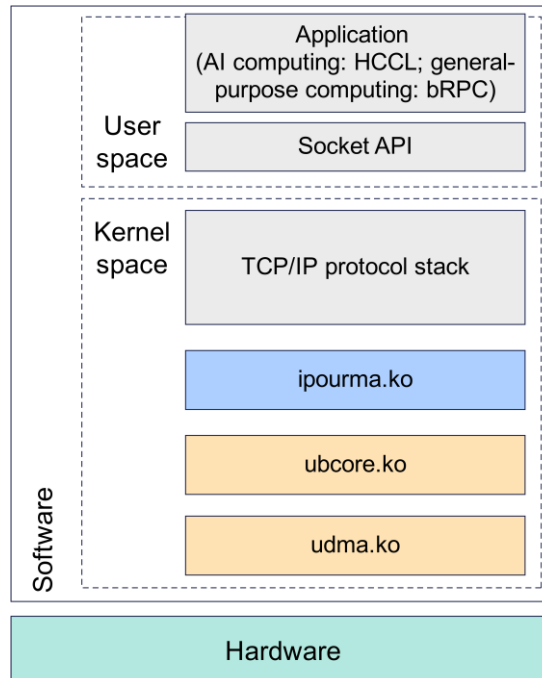
- **UB Memory-based Socket (UMS):** It can connect to the Socket abstraction layer. It is a kernel network protocol stack compatible with the Socket programming interface in the northbound direction and transmits data over the UnifiedBus network in the southbound direction. It transparently accelerates TCP applications to improve performance.
- **Unified Remote Memory Access (URMA):** It is a software library for communication over UnifiedBus. It masks hardware differences and provides remote memory access semantics, such as read/write, send/receive, and atomic operations. URMA serves as the foundation for UnifiedBus communication applications.
- **Unified Remote Procedure Call (URPC):** It is the native RPC of UnifiedBus. It supports high-performance communication between hosts and devices, accelerating remote procedure calls.

IP over URMA

Feature Description

IP over URMA provides a standardized approach for transmitting IP packets over hardware based on the UnifiedBus protocol. Implemented as a kernel module, IP over URMA creates netdev

devices based on UB entities and registers fundamental netdev callback functions, thereby offering an abstraction layer that enables the kernel network protocol stack to control UnifiedBus hardware (Currently, IP over URMA supports IPv6 only). In addition, IP over URMA implements observability and diagnostics tools such as ethtool, allowing the collection and analysis of network packet metrics based on UnifiedBus hardware.



Application Scenarios

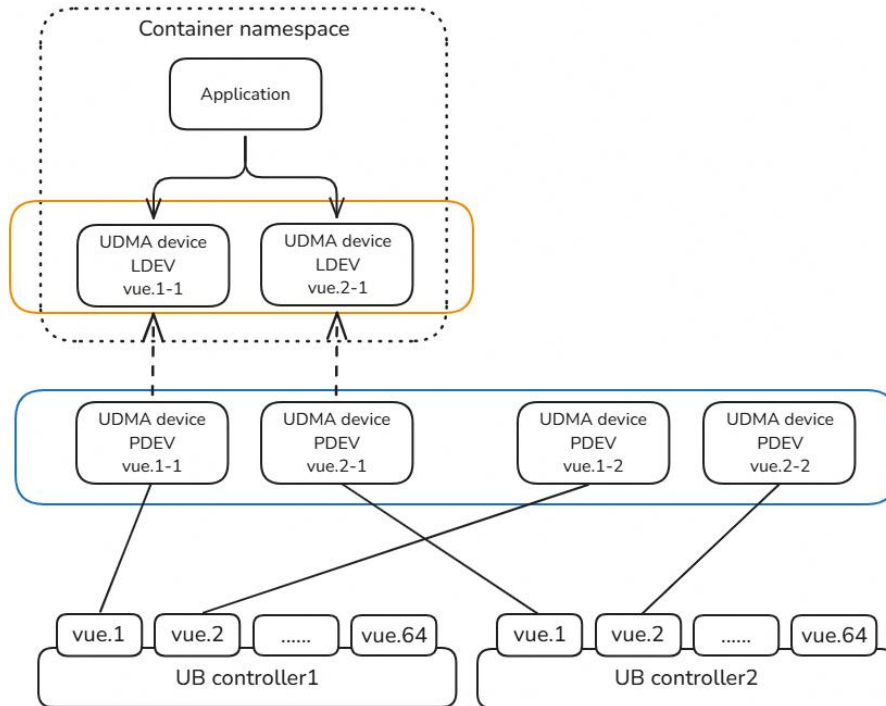
IP over URMA enables TCP/IP communication over the UnifiedBus fabric, combining the high-performance characteristics of UnifiedBus with the broad compatibility of Ethernet. It can replace Ethernet NICs in providing the standard TCP protocol stack and socket APIs, thereby improving UnifiedBus usability. This capability is valuable in the following scenarios:

- **AI SuperPoDs and cluster training:** IP over URMA enables standard TCP communication. In Ascend AI computing scenarios, it is used for collective communication and related workloads, interfacing with the TCP/IP protocol stack for control-plane communication, thereby reducing the need for Ethernet NIC hardware.
- **General-purpose networking and communication:** In general-purpose computing scenarios, IP over URMA enables TCP communication and can be combined with TCP and standard TLS authentication mechanisms to achieve secure communication. Similarly, this approach helps reduce Ethernet NIC hardware costs.

URMA Container Communication

Feature Description

The URMA container solution establishes remote direct memory access capabilities with device-level sharing and EID isolation in containerized environments. Leveraging hardware virtualization and kernel-mediated technologies, it enables a single physical URMA device to be simultaneously exposed to multiple designated container network namespaces, allowing efficient consolidation and reuse of hardware resources. At the same time, each EID of the URMA device is bound to a single namespace, ensuring strict isolation so that URMA resources across tenants or applications cannot be accessed across boundaries.



Application Scenarios

In multi-tenant environments, this URMA container solution flexibly allocates devices and EID resources based on actual resource availability and isolation requirements. When sufficient physical devices are available, each tenant can be assigned a dedicated URMA device, achieving complete hardware-level isolation with no interference between tenants. When device resources are limited, multiple tenants or multiple services within a single tenant can securely share the same URMA device. Although the underlying hardware is shared, each container namespace is assigned its own independent EID, ensuring strict isolation of memory access permissions so that tenant A cannot perceive or access the resources of tenant B.

URMA High-Performance Distribution Communication Acceleration

When applications use URMA for communication, a one-to-one mapping between Jetty/JFS and JFR for logical connections causes JFR receive buffer consumption to grow linearly with the number of connections. For example, with communication memory managed in 8 KB chunks and a JFR depth of 1024, each logical connection requires 8 MB of pre-allocated receive buffer. When the number of logical connections reaches 1,000 to 10,000, the reserved receive buffer alone can reach 8 GB to 80 GB. To reduce memory pre-allocation, UMQs introduce a URMA-based shared JFR communication model. In this mode, multiple logical connections within an application share a single JFR for data reception, and received data is efficiently dispatched to the correct logical connection using service-associated labels.

Feature Description

UMQs provide a URMA-based shared JFR communication model. They support configurable mapping between receive queue elements (RQEs) received from a shared JFR and logical connection handles, enabling efficient RQE distribution. In addition, UMQs implement a traffic control mechanism based on the real-time depth of the shared JFR receive buffer to prevent traffic oversubscription. Key capabilities include:

- **Shared JFR enablement:** When creating a group of UMQs, the same UMQ can be specified as the shared JFR source. All UMQs in the group then use a common underlying JFR, and applications can monitor the shared UMQ to receive packet events across the group.
- **High-performance distribution:** UMQs can be configured with labels associated with application logical connections during creation. These labels are carried with received data, allowing applications to efficiently dispatch data to the corresponding logical connections.
- **Adaptive flow control:** The receive side maintains a signaling pool associated with the depth of the shared JFR receive queue. The transmit side must acquire sufficient signaling credits before sending data. When credits are insufficient, the transmit side adaptively adjusts the number of requested credits based on historical consumption. Idle credits caused by timeouts are reclaimed to maintain efficiency.

Application Scenarios

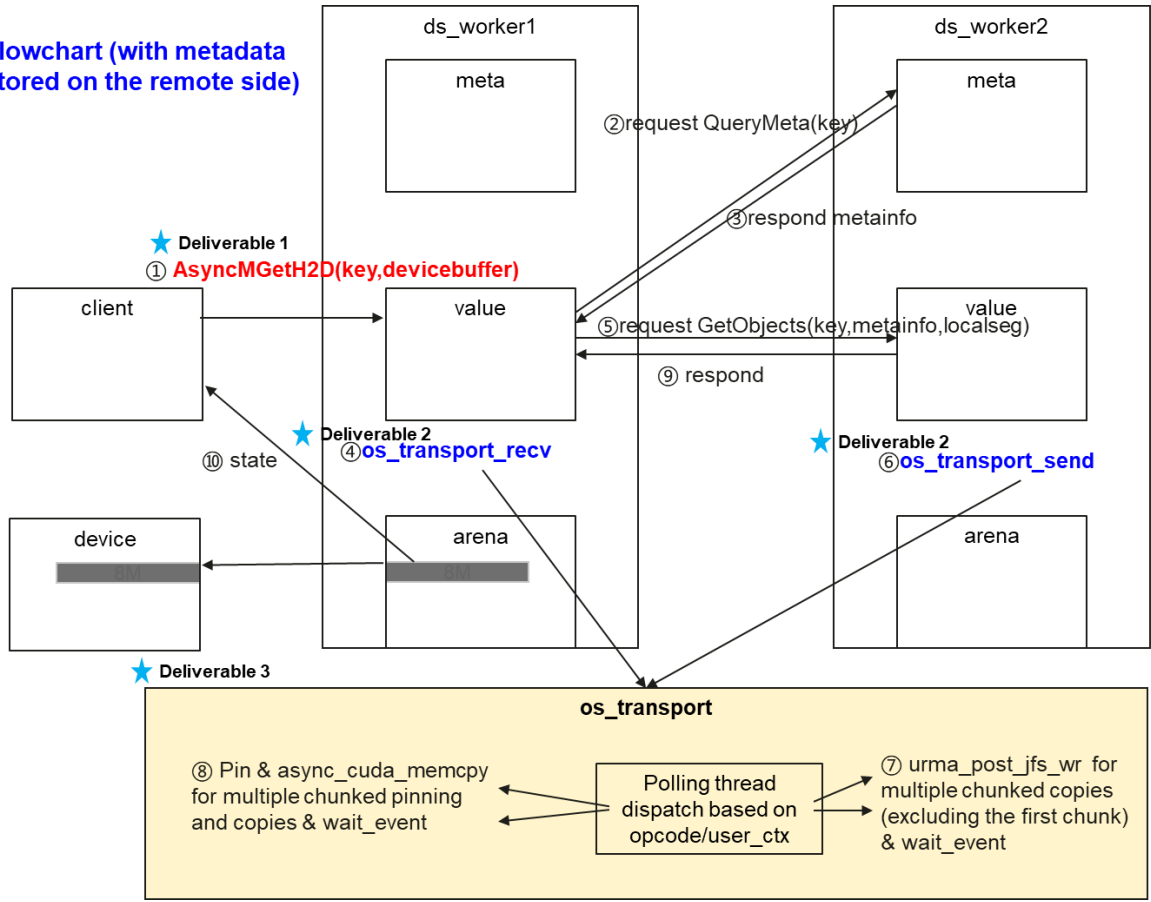
This solution is well suited for environments with large numbers of nodes and connections. Applications can plan communication memory based on concurrency levels and active connections, significantly reducing memory overhead along the communication path. For example, in search and recommendation services, RPC is used for global task scheduling across fully meshed nodes, while the number of active connections remains relatively low (on the order of hundreds). By adopting this solution, communication memory can be reduced to around 1 GB at a scale of 1,000 connections, while preserving the performance advantages of URMA-based communication.

RH2D Multi-Level Cache Passthrough Acceleration

Feature Description

RH2D targets heterogeneous converged OSs and distributed caching scenarios, providing the capability to asynchronously transfer KV cache data from a remote host to a local device. Its core objective is to reduce the serial waiting overhead of traditional workflows, where data is first read from a remote host into local memory and then copied from the client to the GPU during large-object access such as KV cache retrieval. Built on the high-speed URMA transport channel, RH2D partitions remote objects into fixed-size chunks and employs an asynchronous pipeline to enable parallel progression of remote data transfer and local GPU writes. For large data blocks, RH2D supports concurrent receiving, dispatching, and copying, thereby reducing end-to-end latency for KV cache retrieval and significantly improving the efficiency of writing KV cache data from the remote host to the GPU.

Flowchart (with metadata stored on the remote side)



Application Scenarios

RH2D is well suited for scenarios such as AI inference, training, distributed caching, and heterogeneous computing, where large volumes of data must be frequently fetched from remote nodes and written to GPUs. It is particularly effective for workloads characterized by large value sizes, high concurrency, and multi-threaded access. By minimizing serial waiting during data retrieval, RH2D significantly improves efficiency in use cases such as KV cache fetching in large model inference and cross-node sample cache reads.

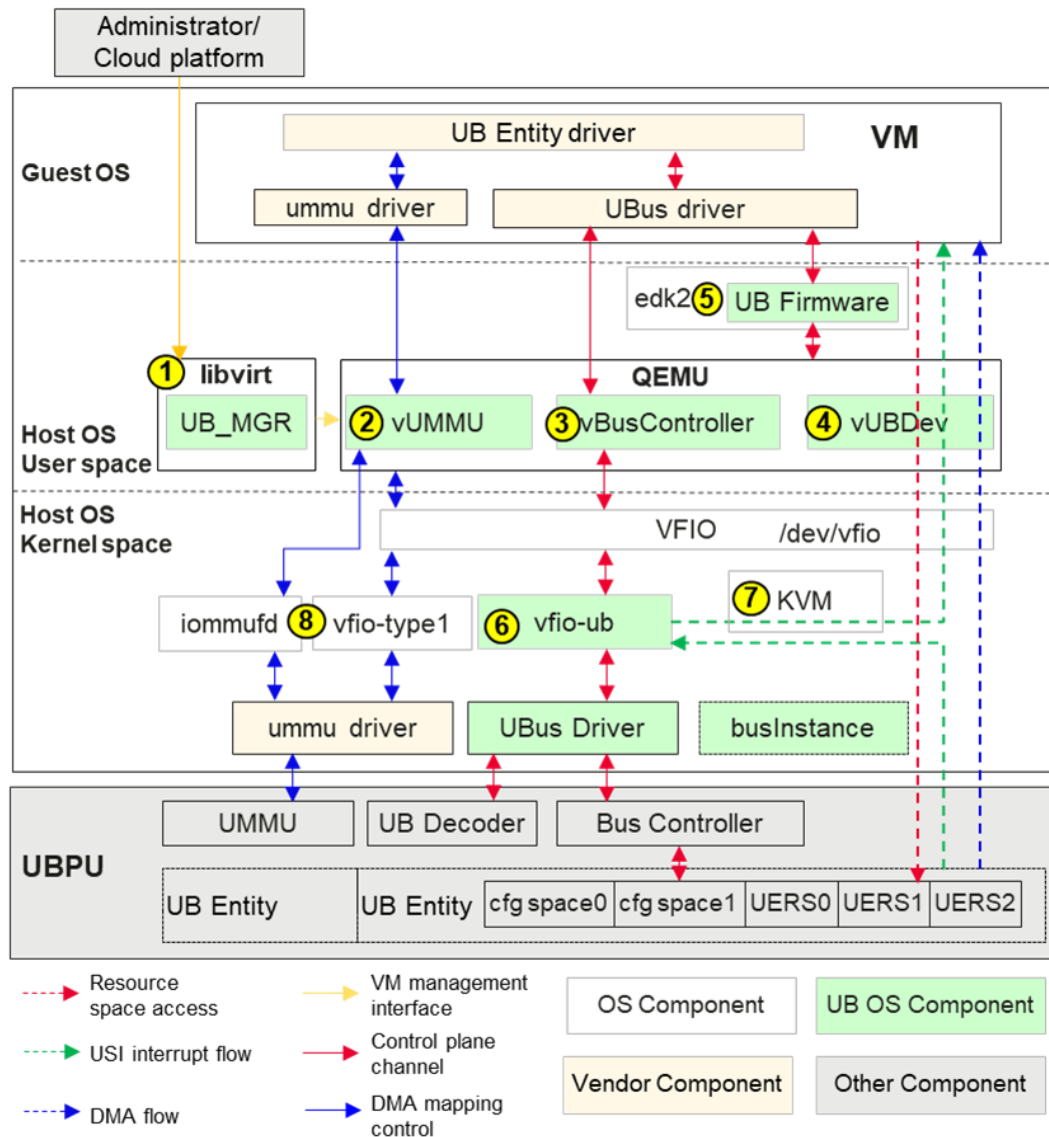
Virtualization

In cloud computing scenarios, hardware resources on a physical machine are isolated and provisioned to VMs through virtualization, significantly enhancing resource utilization. As a next-generation high-speed interconnect, UB natively supports virtualization.

Mainstream device virtualization technologies include I/O full virtualization, I/O paravirtualization, and hardware-assisted I/O virtualization. Under traditional architectures, each server connects to the Top of Rack (TOR) switch through its own NIC, meaning its maximum supported bandwidth is fixed. This can lead to several issues in real-world applications, as illustrated below.

A UB Entity acts as an isolated functional unit within a UB device. Virtualization in UB supports direct device assignment to VMs, delivering native-level performance. Compared to traditional hardware-assisted I/O virtualization, UB device virtualization not only supports passthrough to local UB devices but also to pooled UB devices. The preceding problems can be solved by pooling NIC and DPU resources. All device resources are aggregated in a shared device pool, allowing servers to dynamically request NIC resources based on their actual network load. This

approach improves overall resource utilization and helps prevent network bandwidth bottlenecks. In addition, UnifiedBus provides unique memory management and high-bandwidth communication features, enabling flexible memory overcommitment and fast live migration for VMs.



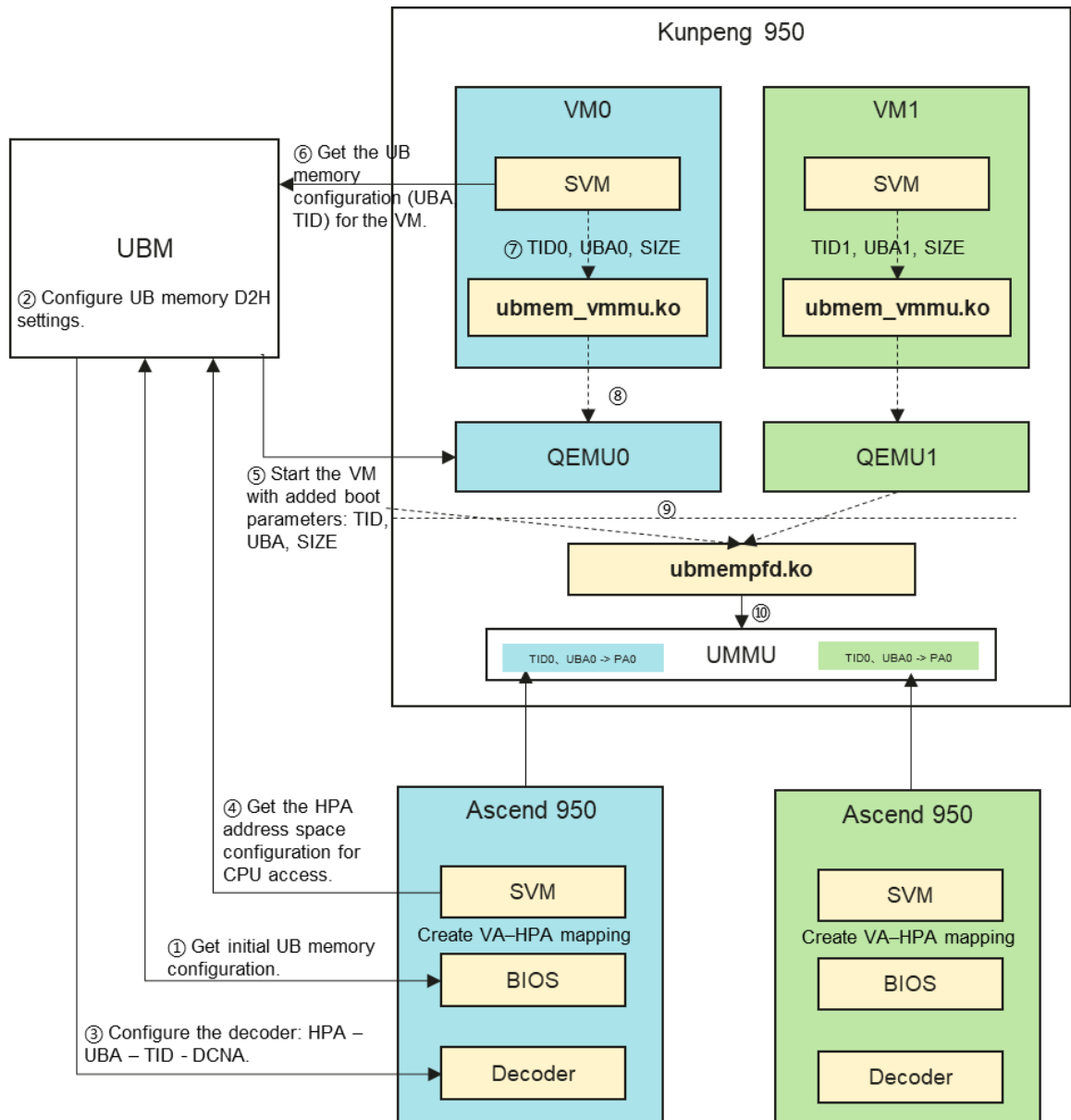
- UBNative:** It is a high-performance device virtualization solution built on the UnifiedBus protocol. It simulates the UnifiedBus device model to enable seamless VM access to UnifiedBus devices. Based on Virtual Function I/O (VFIO) technology, UnifiedBus devices are passed through to VMs, ensuring native device performance in virtual environments. This achieves hardware-level efficiency and resource utilization, providing stable support for high-throughput, latency-sensitive scenarios.
- Memory overcommitment:** As host memory capacity increases, memory accounts for a larger share of the total cost of ownership (TCO). Improving memory utilization in cloud scenarios is therefore becoming increasingly important. The memory overcommitment function is implemented based on the UnifiedBus-powered SuperPoD architecture. A memory reclamation solution for hugepage and passthrough VMs fully reclaims unused VM memory, improving virtual memory utilization and reducing cloud vendors' TCO.

- **Ultra-fast live migration:** URMA memory read/write semantics are used with the UnifiedBus high-performance network to migrate memory data. This reduces downtime and resource overhead, and improves the success rate of VMs under high memory pressure.

Device Passthrough for UnifiedBus VMs

QEMU supports NPU D2H transfers.

Feature Description



This feature is part of the UB Memory subsystem, which sits between the guest OS service drivers and the host UMMU. It provides interfaces for invocation by service driver subsystems or modules within the guest OS, with the SVM module serving as the primary upper-layer component.

On the southbound side, the UB Memory subsystem leverages VUMMU hardware emulated by QEMU. QEMU simulates the VUMMU device and, through the interfaces provided by the UB

Memory Mapping Driver, transparently propagates map and unmap operations from the guest to corresponding memory mapping operations on the host.

Usage:

Add the `ubmem_vmmu` device configuration to the VM XML file.

```
<qemu:commandline>
```

```
<qemu:arg value='-device'/>
```

```
<qemu:arg value='ubmem_vmmu,tid=1,uba=0,size=204800000'/>
```

```
</qemu:commandline>
```

Parameter description:

- **ubmem_vmmu**: Fixed device type, which does not need to be changed.
- **tid**: Token ID assigned by the UB control plane. Each VM is allocated a unique token ID, which is used by NPUs to identify the target VM when sending UnifiedBus packets. Specifying the token ID via the command line also enables validity checks, helping prevent potential compromise of drivers within the Guest OS.
- **uba** and **size**: Range of UB memory addresses, which are specified by the UB control plane and represent the destination address range for UB packets initiated by NPUs. Specifying the parameters via the command line enables validity checks, helping prevent potential compromise of drivers within the Guest OS.

Application Scenarios

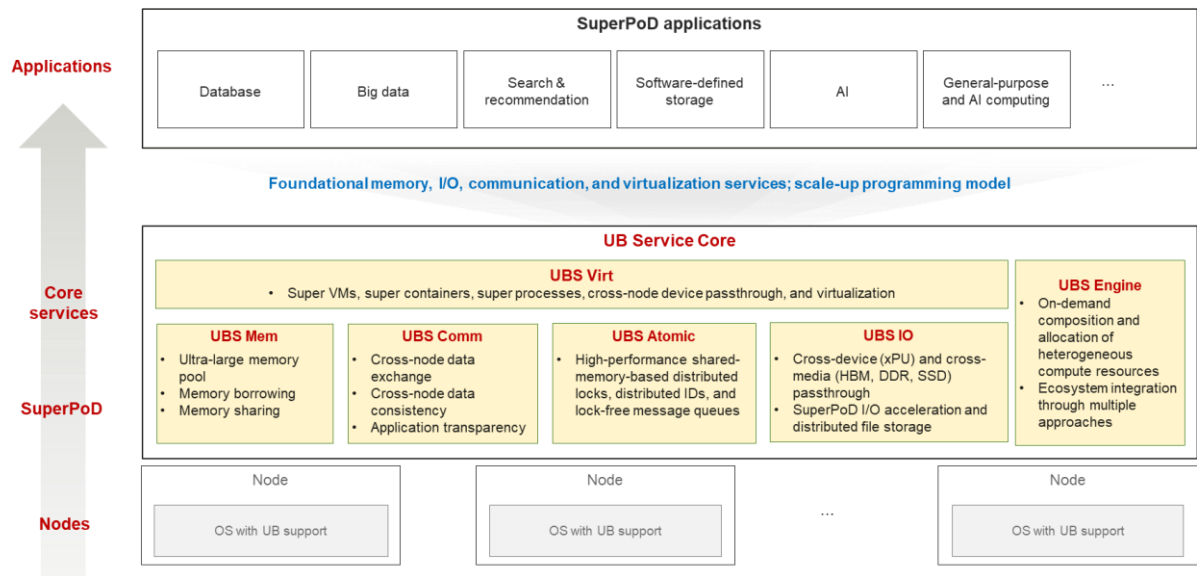
In virtualization scenarios where NPUs are connected via the PCIe bus, NPUs are typically passed through directly to VM tenants to balance security isolation and performance. In this setup, tenant KV cache data resides in the VM memory, and tenants remain unaware of the host. The NPUs can directly access the KV cache in the VM through the PCIe bus and SMMU hardware, thereby avoiding the performance overhead associated with transferring KV cache data from the guest to the NPUs.

In virtualization scenarios where NPUs are connected via UnifiedBus, the high-performance UnifiedBus interconnect replaces PCIe for communication between CPUs and NPUs. However, UnifiedBus does not currently support full virtualization for NPU access to VM memory, such as through SMMU-like page tables. To address this, a paravirtualization approach is adopted: the UB driver within the VM proactively shares KV cache metadata, such as GPA and size, with the host. The host then establishes the corresponding UMMU page tables, enabling NPUs to access the KV cache memory over UnifiedBus.

UB Service Core

For SuperPoDs, UnifiedBus builds the UB Service Core to encapsulate underlying capabilities and cluster topologies. This allows applications to use SuperPoD resources like local resources, streamlining development and maintaining compatibility with the existing ecosystem.

UB Service Core enables six cluster-level system services, leveraging the advantages of the peer-to-peer interconnect and accelerating applications by 30% to 50%. The following figure illustrates the reference architecture of UB Service Core.



UB Service Core consists of six components:

- **UB Service Core Engine (UBS Engine):** It provides resource pooling and dynamic scheduling for memory, SSUs, and NPUs, supports distributed automatic primary node election, and implements N-1 high availability. It serves as the core control-plane reference implementation for a UnifiedBus-powered computing system.
- **UB Service Core Memory (UBS Mem):** It supports unified memory programming and enables memory sharing and pooling within a UnifiedBus-powered SuperPoD.
- **UB Service Core Communication (UBS Comm):** It is a high-performance, highly reliable communication protocol with ecosystem compatibility (user-space Socket/Verbs over UnifiedBus) for SuperPoDs.
- **UB Service Core IO (UBS IO):** It provides high-level I/O services for the global data read/write cache system for SuperPoDs.
- **UB Service Core Virt (UBS Virt):** It supports virtualization pooling, live migration policy decision-making, quick disaster recovery, and efficient VM/container communication, improving virtualization performance.
- **UB Service Core Atomic (UBS Atomic):** It leverages UB atomic operation instructions and cross-node shared memory capabilities to provide distributed cluster locking, global transaction ID management, and cluster queue communication functionality.

For details, see the *UnifiedBus™ (UB) Service Core Software Architecture Reference Design* at the [UnifiedBus community](#).

UnifiedBus Reliability

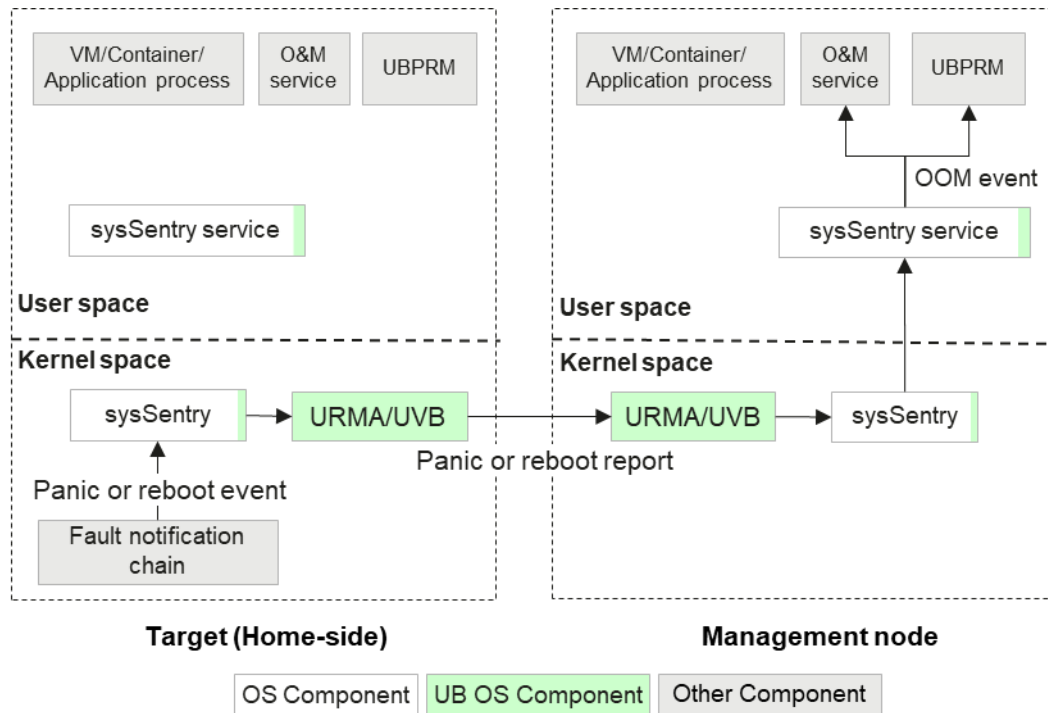
Reliability Plugin

Feature Description

When critical events such as OOM, panic, or reboot occur, sysSentry blocks these events and reports them to UBPRM to prevent data loss or service interruption. It also provides centralized management for UB event reports.

Node Fault Detection and Recovery

When a node on the home side experiences a panic or reboot, the memory it has lent becomes inaccessible, which directly impacts the service processes on the user side. To mitigate this, the data stored in the memory on the home side needs to be migrated.



The report process is as follows:

When a home-side node encounters a kernel failure (such as a panic, reboot, or shutdown), it notifies sysSentry of the impending reset.

1. sysSentry initiates a fault event broadcast via the URMA/UVB channels.
2. The fault event is sent from the URMA/UVB channel of the home-side node to the URMA/UVB channel of the management node.
3. sysSentry on the management node receives the event and notifies UBPRM via sysSentry Service.
4. UBPRM uses **sentryctl** commands to configure sysSentry to intercept panic and reboot events and set required parameters for cross-node communication. By subscribing to OS panic and reboot events, sysSentry can monitor panic and reboot occurrences.

Once UBPRM receives the fault report from sysSentry Service, it initiates specific isolation and recovery measures. For example, it can migrate the home-side node's memory to a healthy node and terminate the borrowing relationship with the faulty node, ensuring that services utilizing pooled memory remain unaffected.

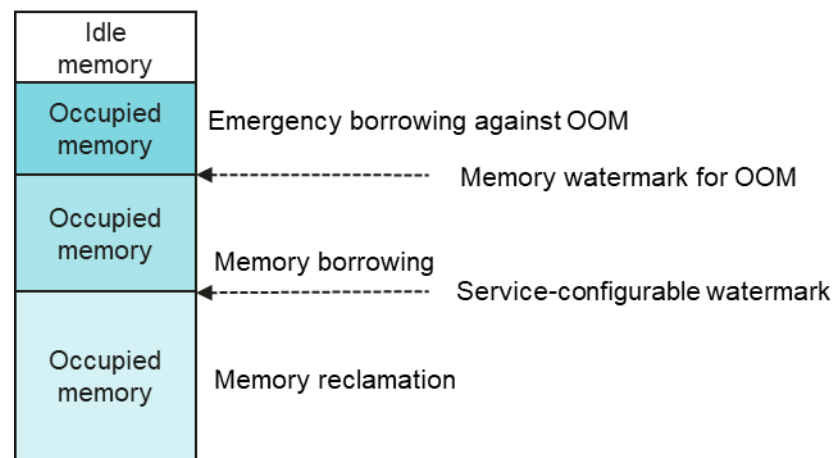
After the fault is rectified, the fault handling results need to be returned. The system performs the recovery according to the following process:

1. UBPRM notifies sysSentry Service of the fault handling results.
2. The fault handling results are sent via sysSentry and the URMA/UVB channels back to the home-side node.

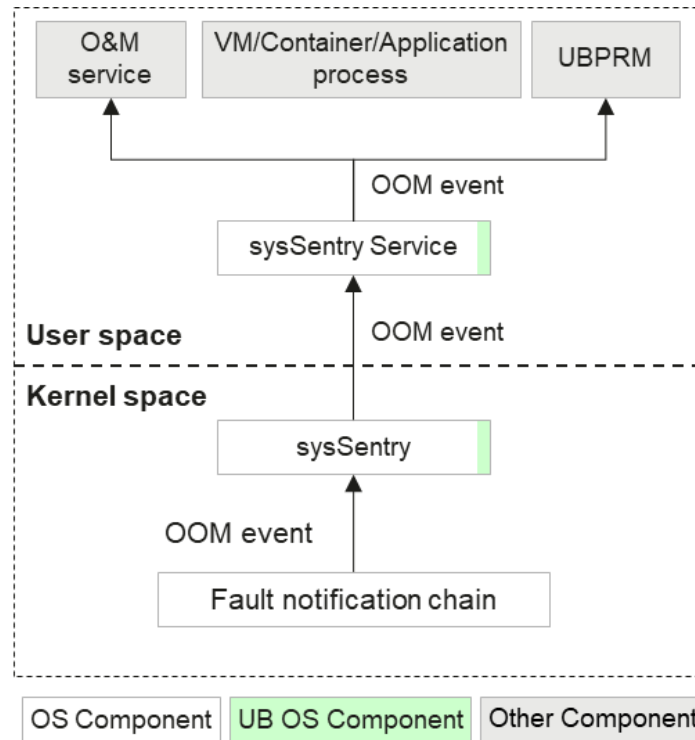
3. sysSentry on the home-side node retrieves the handling results from its local URMA/UVB channel.
4. The home-side node returns to its kernel failure process to complete the reset and reboot.
5. If the home-side node does not receive a notification from UBPRM within a certain period, it will proceed with the reset and reboot.

OOM Detection, Prevention, and Recovery

In memory borrowing scenarios, UBPRM monitors memory usage watermarks at fixed intervals. These watermarks are configurable, enabling the system to dynamically apply memory return or borrowing policy based on actual demand. When a large volume of memory is consumed in a short period, the standard periodic detection mechanism may fail to respond in time. This can lead to the OOM killer terminating critical processes or causing node reboots, resulting in service interruptions. To mitigate this risk, the OOM prevention mechanism is implemented to report impending OOM events to UBPRM and trigger the emergency borrowing policy.



When OOM-triggered emergency borrowing occurs, the report process is as follows.



1. The node experiencing an OOM condition reports an OOM event to sysSentry through the OOM fault report chain.
2. sysSentry reports the OOM event to sysSentry Service.
3. sysSentry Service reports the OOM event to UBPRM or the O&M service.

UBPRM can subscribe to OOM events from sysSentry. When an OOM event occurs on a node, UBPRM receives a notification via sysSentry Service and initiates the following recommended recovery process:

1. UBPRM borrows available memory resources from other nodes and provisions them to the OS of the service node.
2. The OS attempts to fulfill the pending memory request within the OOM context using the newly provisioned resources. If the request is successful, the OS allocates the memory to the application. Otherwise, the OS kills the application. If OOM occurs while kernel space is requesting memory, the panic process is triggered to perform a full hardware reset.

Application Scenarios

In the context of UnifiedBus memory pooling, when a node experiences a critical failure that threatens the availability of borrowed memory, sysSentry hijacks the fault event and notifies the management plane to initiate emergency memory migration, preventing service interruptions.

Memory Pooling Fault Takeover and Notification

Feature Description

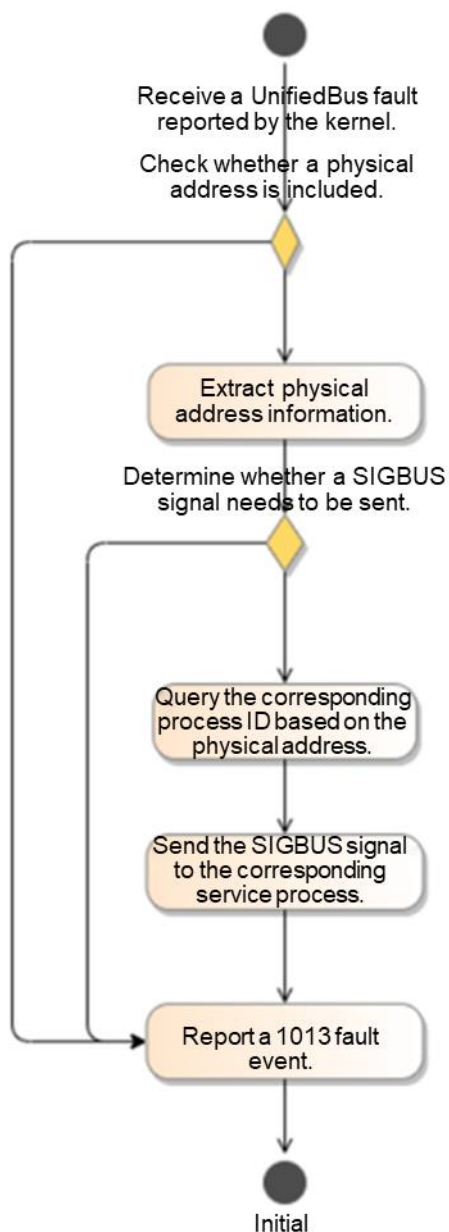
The OS defines a standard mechanism for handling memory access faults. Such faults are reported as SEA or SEI in accordance with hardware specifications, and are handled by the kernel following the defined protocol. The handling actions may include triggering a kernel panic,

or isolating the affected memory region and delivering a SIGBUS signal to user-space threads that access the corresponding memory. This fault-handling flow applies to memory managed within the kernel's memory management subsystem.

SEA/SEI events report synchronous or asynchronous fault conditions triggered by proactive memory access operations (LD/ST instructions). In addition, UnifiedBus introduces another class of fault events with a broader impact scope: link-down failures. Since all cross-node memory accesses on UnifiedBus must traverse the UB link, a link failure may cause all remote memory accesses to fail. To address this, UnifiedBus provides proactive reporting of link-down events. When a link-down fault is detected, the hardware reports the event to the kernel UBUS component via the BIOS. sysSentry subscribes to these events and forwards them to user space. Upon receiving such events, user-space applications can perform customized isolation or recovery operations.

SuperPoD memory pooling provides two usage modes: memory borrowing and memory sharing. In the memory borrowing mode, memory is exposed to the OS and managed by the memory management subsystem. The fault handling flow for borrowed memory is similar to that for local system memory. Borrowed memory is brought online via remote NUMA and allocated through the memory management subsystem. Applications can request memory allocation based on specific memory access policies. When a memory fault occurs, it is reported to the OS using the existing SEA/SEI mechanism. The OS then handles the fault through the standard memory failure workflow, including memory isolation and process termination. In the memory sharing mode, memory is exposed through a dedicated memory device, bypassing the kernel memory management subsystem, and is directly mapped into user space via mmap as a device-backed region. Memory faults in this mode require additional handling mechanisms. For UnifiedBus SuperPoDs, an additional UBEvent asynchronous event mechanism is provided to report memory access fault information. These events are handled directly by the UBUS driver, which exposes a hook interface for external processing. sysSentry registers this hook to handle UBEvent-based memory fault notifications. Upon receiving a fault event, sysSentry processes the reported faulting memory address and error information, and performs a reverse lookup to identify the corresponding memory device file. In user space, it traverses all processes that have mapped the device and sends SIGBUS signals to notify affected threads of the memory fault.

This feature complements the application-level notification capability for memory faults in shared memory scenarios. Once a fault is detected, applications are notified and can perform custom handling based on their own logic.



Application Scenarios

Take a database service using UnifiedBus shared memory as an example. In this scenario, multiple database instances across nodes use shared memory as a global buffer pool. When a memory fault occurs, the database process receives a SIGBUS signal. Instead of being terminated, the database can intercept the signal and handle it gracefully. It rolls back only the transactions affected by the memory fault, isolates the corresponding memory region, and recovers data through mechanisms such as log replay, thereby minimizing the overall impact on the service.

In online-offline co-location scenarios, memory borrowing is used. Online applications are given higher priority and use local memory, while offline applications use borrowed memory to avoid performance interference. Offline applications can be configured, via NUMA affinity, to use remote memory nodes. When a memory fault occurs, it is handled through the standard kernel memory

failure workflow, including memory isolation and delivery of SIGBUS signals to affected processes.

Fault Containment for Borrowed-Memory File Cache

Feature Description

The page cache is an OS-provided mechanism for sharing memory across processes. A UnifiedBus SuperPoD may include NUMA nodes that are not associated with any CPUs, referred to as CPU-less nodes. Memory allocated from these CPU-less nodes may be physically located on remote hosts. Due to the shared nature of the page cache, a fault in a page cache page can impact all processes that map or access it.

A new sysctl parameter, **vm.filemap_alloc_local**, is introduced to control the page cache allocation policy. When this parameter is set to **1**, the page cache is not allocated to CPU-less NUMA nodes. When it is set to **0**, the default allocation policy is retained.

mempolicy support:

If this parameter is set to **1**, CPU-less NUMA nodes are excluded from the set of NUMA nodes defined by the process mempolicy during page cache allocation, while the policy mode itself remains unchanged. If the mempolicy contains only CPU-less NUMA nodes, the system falls back to the process `mems_allowed` mask, removes any CPU-less NUMA nodes from it, and uses the remaining nodes as the effective allocation set. The policy mode (bind, prefer, or interleave) remains unchanged.

If cgroup-level `cpuset.spread_mem` is enabled, page cache pages are interleaved across non-CPU-less NUMA nodes according to the `spread_mem` policy.

If this parameter is set to **0**, page cache allocation follows the default behavior and is not affected.

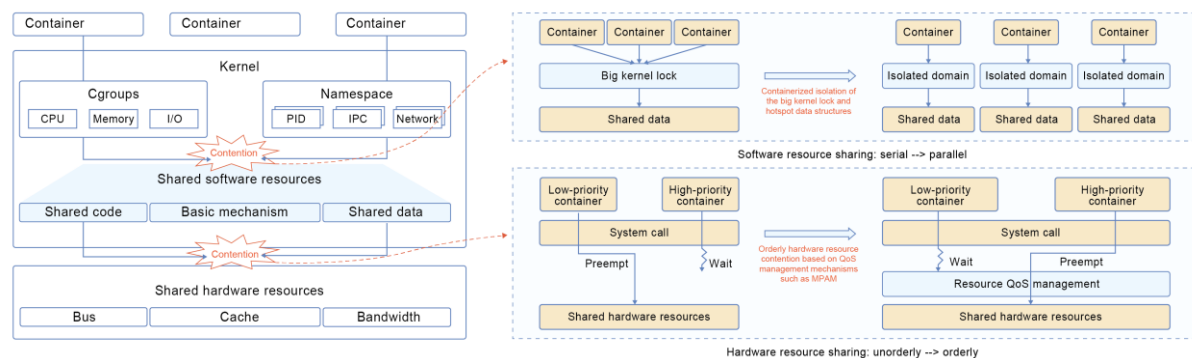
Application Scenarios

In scenarios where files are shared across local and remote nodes, constraining the allocation of page cache to specific NUMA nodes can effectively prevent faults in remote page cache from propagating to the local system.

Cloud-Native

High-Density Many-Core Container Isolation

Server chips have evolved from multi-core to many-core architectures (typically exceeding 256 cores), posing new challenges to OSs. To boost rack-level computing density and reduce data center TCO, many-core servers have become the mainstream in the Internet industry. As cloud technologies and service scales advance, containerized deployment has also become the dominant model. Against this backdrop, serialization and synchronization overheads hinder system scalability, while interference and low resource utilization become increasingly prominent. These scalability issues in container deployments arise from contention for shared hardware and software resources.



Feature Description

Lightweight virtualization is used to partition resources by NUMA domain and enforce container-level resource isolation within each domain. This approach minimizes performance interference caused by hardware and software resource contention and enhances the container deployment scalability.

- VM memory QoS control:** When multiple tenants' VMs are deployed on the same physical host, memory-intensive VMs may consume a large portion of the available memory bandwidth. This can lead to resource contention, preventing other VMs from obtaining sufficient bandwidth to meet their performance requirements. As a result, the overall system QoS may be degraded. Leveraging the MPAM feature of the Kunpeng processor and the OS-level resource control mechanisms, the system can perform fine-grained monitoring and dynamic control of memory bandwidth usage for up to 30 VMs. This capability enables configuration of upper and lower limits, as well as priority policies, to establish a memory bandwidth isolation and assurance framework in a multi-tenant environment.
 - Memory bandwidth upper limit:** A maximum memory bandwidth threshold can be configured for each VM to prevent any single VM from consuming excessive bandwidth resources, thereby avoiding performance interference with other tenant VMs.
 - Memory bandwidth lower limit:** A minimum bandwidth guarantee can be configured to maintain a baseline allocation even for lightly loaded VMs, enabling dynamic resource optimization and efficient bandwidth utilization.
 - Priority-based scheduling policy:** The memory bandwidth priority of each VM can be configured based on service importance, ensuring stable bandwidth allocation for critical workloads and improving the availability and service quality of high-priority VMs.
- NUMA affinity of virtual devices:** PCI devices possess NUMA affinity and can be directly accessed on the host. The OS scheduling mechanism optimizes task placement based on device affinity to minimize performance loss resulting from cross-NUMA access to PCI devices. In VM passthrough scenarios, PCI devices do not expose their NUMA node affinity to guest VMs. This feature expands the PCI device topology of VMs based on the PCI Expander Bridge (PXB). Within a VM, it displays the NUMA node where each virtual device is located. This helps the system optimize scheduling and allows users to deploy service applications according to the NUMA nodes of their virtual devices, thereby reducing performance loss caused by cross-NUMA resource access and improving the overall performance of VM service applications.
- Lightweight virtualization:** Virtual device interrupt offload allows VirtIO device interrupts to be offloaded to hardware injection, reducing storage virtualization overhead. PMD queue load balancing distributes virtual drive I/O queues from a single reactor to multiple reactors, eliminating CPU bottlenecks.

- **CPU scheduling by domain:** CPUs are divided into domains by cluster for container deployment. Each container operates within an independent scheduling domain. This design isolates interference between containers, reduces cross-cluster cache synchronizations, and eases contention for hardware resources like cache and NUMA memory. It improves performance by more than 10% in high-concurrency Redis scenarios.
- **Interference isolation in file system block allocation:** Optimizations to the group lock and s_md_lock in the EXT4 block allocation and release processes enhance the scalability of EXT4 block allocation. When the target block group is occupied, allocation can switch to an idle block group to reduce CPU overhead caused by multiple containers competing for the same group. Leveraging multiple EXT4 block groups helps ease group lock contention. Apart from that, the global target of the streaming allocation is split to multiple targets, so that the contention for the global lock s_md_lock is alleviated and the file data is more aggregated. In a 64-container concurrency scenario, the OPS increases by over 5 times in mixed block allocation and release workloads and by over 10 times in single-block allocation workloads.
- **Efficient slab reclamation:** In this openEuler release, the read and write locks used for slab memory reclamation are replaced with the read-copy-update (RCU) lock-free mechanism. Memory reclamation across different slabs operates independently, significantly improving reclamation efficiency. In multi-container concurrency scenarios, system calls are accelerated.
- **TCP hash interference isolation:** In high-concurrency scenarios, lock contention in tcp_hashinfo bash and ehash and frequent ehash calculations lead to reduced bandwidth and increased latency. The spin lock of tcp_hashinfo bash and ehash can be replaced with read-copy update (RCU), and the ehash calculation method can be changed to lport increment. These changes reduce the query time and calculations and reduce the lock contention in the TCP connection hash.
- **Enhanced control group (cgroup) isolation:** Original atomic operations are replaced with percpu counters to avoid parent node contention across namespaces and eliminate rlimit count interference between containers. This mechanism addresses the linearity issue in the **will-it-scale/signal1** test case and triples concurrent throughput performance in a 64-container deployment. Memory cgroups are released in batches to avoid contention for the same parent node's counter caused by frequent small memory releases, enhancing memory count scalability. In the **tlb-flush2** test case, this improves throughput by 1.5 times with 64 containers. Leveraging eBPF's programmable kernel capabilities, a host information isolation and filtering approach is provided to present container-specific resource views. Compared with the peer LXCFS solution, this openEuler solution avoids the overhead of switching between the kernel space and user space, and eliminates the performance and reliability bottlenecks associated with the LXCFS process. It doubles the resource view throughput in a single container and achieves a 10-fold increase in a 64-container deployment.
- **Interference monitoring:** Interference falls into three categories by result: instruction execution failure, instruction slowdown, and increased instruction execution. Interference is monitored from the kernel perspective, with statistics collected on each typical category during runtime. The system supports online monitoring of schedule latency, throttling, softirq, hardirq, spin lock, mutual exclusion (mutex), and simultaneous multi-threading (SMT) interference while incurring less than 5% performance overhead.
- **Kunpeng memory and cache QoS control:** The memory bandwidth traffic and cache usage at each level can be configured based on the upper limit, lower limit, or priority-based policy. Each thread is assigned a specific isolation policy to suit specific service requirements. The usage of shared resources is monitored in real time at both service and thread levels, and reported to the control policy to enable feedback-driven control. In addition, the MPAM feature and the SMMU combine to enhance peripherals' I/O QoS. They support bandwidth isolation for peripherals and heterogeneous accelerators and allow for resource monitoring at the device level.

- **Dynamic QoS policy configuration:** This openEuler release provides a cluster-level MPAM QoS management plugin. Based on the QoS interfaces provided by MPAM, the plugin automatically applies priority levels to all nodes according to user-defined policies and configures MPAM QoS priorities for online and offline tasks based on user settings. This maximizes resource utilization in co-location scenarios. When online workloads are busy, the last-level cache (LLC) and memory bandwidth resources assigned to offline tasks are automatically reclaimed. When online workloads are idle, these resources are automatically released to improve offline workload performance.
- **I/O QoS controller IOInflight for I/O co-location scenarios:** In high-density co-location scenarios, I/O resource contention can cause online services to experience interference rates exceeding 40%. Traditional blk-throttle static I/O throttling can reduce interference but significantly wastes drive bandwidth. To address this issue, the IOInflight controller is introduced. It uses a dynamic throttling mechanism based on latency monitoring and queue depth to enable precise I/O resource preemption for online workloads. The controller keeps interference and memory overhead within 5%, effectively ensuring online service latency while doubling the bandwidth available to offline workloads compared with traditional hard-limit approaches.

Application Scenarios

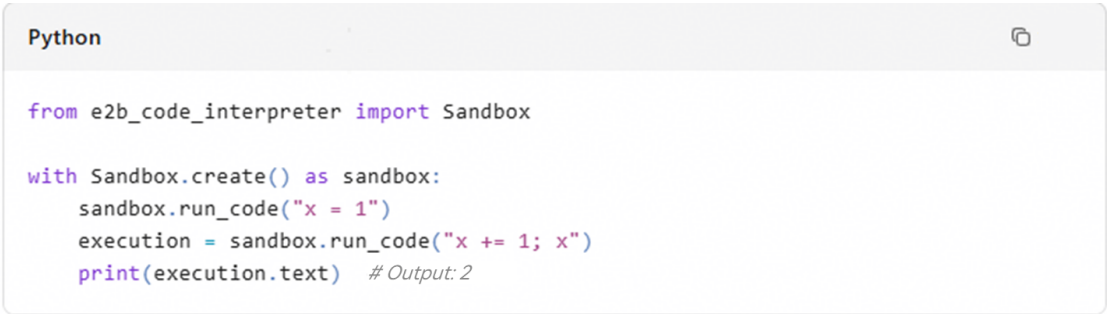
In a many-core server environment, service containers are deployed in a high-density configuration. The openEuler solution minimizes interference between containers, increases deployment density, and enhances resource utilization.

In memory-intensive scenarios, this feature mitigates bandwidth contention among multiple VMs. It monitors and controls the memory bandwidth usage of each VM, ensuring that performance-critical services run reliably.

Agent Sandbox

Feature Description

- Fast startup and elastic scalability
 - **Sub-100 ms cold start:** Achieves startup speeds hundreds of times faster than traditional VMs, powered by Firecracker microVM technology.
 - **Elastic scaling within a single cluster:** Seamlessly scales from a single sandbox to hundreds of concurrent instances, with automatic elasticity to match workload demands.
- Multi-language code execution
 - Supports multiple programming languages, including Python and JavaScript.



```
Python
from e2b_code_interpreter import Sandbox

with Sandbox.create() as sandbox:
    sandbox.run_code("x = 1")
    execution = sandbox.run_code("x += 1; x")
    print(execution.text)  # Output: 2
```

- Enterprise-grade security isolation

- **Hardware-level isolation:** Runs each sandbox in an independent Firecracker microVM with a dedicated kernel.
- **Layered security mechanisms:** Combines the Jailer process, cgroups and namespaces, and network access controls to enforce strong isolation.
- **Resource governance:** Applies CPU, memory, and storage quotas to prevent resource exhaustion attacks.
- Comprehensive environment control
 - **File system access:** Provides full file read/write capabilities, including upload and download operations.
 - **Process management:** Executes arbitrary system commands and installs software packages.
 - **Network access:** Initiates HTTP requests and invokes external APIs.
 - **Dynamic dependencies:** Installs Python, Node.js, and other dependencies at runtime.
 - **State persistence:** Preserves variables and runtime state across multiple executions.
- Long sessions and advanced lifecycle management
 - **Advanced lifecycle operations:** Supports pause, resume, checkpoint, and fork.
 - **Automatic cleanup:** Terminates sessions on timeout and supports manual teardown.
 - **Configurable session duration:** Allows session lifetimes to be configured, supporting long-running sessions of up to several hours.
 - **Graceful shutdown:** Enables automatic VM stop and restart with efficient resource utilization.
- Custom templates
 - Builds custom sandboxes with preconfigured environments using Dockerfiles.

```
bash

e2b template init      # Initialize the template.
# Edit the Dockerfile to add dependencies.
e2b template build     # Build
e2b template deploy    # Deploy
```

Application Scenarios

- **AI agent code execution:** Enable LLMs to securely run code, invoke tools, and process data.
- **Automated workflows:** Build and run batch processing workflows, including scheduled tasks, data processing, and ETL.
- **Cloud IDE:** Develop in a lightweight and scalable cloud-based development environment.

5 Kernel Innovations

openEuler 24.03 LTS SP4 runs on Linux kernel 6.6 and inherits the competitive advantages of community versions and innovative features released in the openEuler community.

- **64 KB kernel support:** openEuler has released a kernel with 64 KB page size. The Arm image supports optional installation of either a 4 KB or 64 KB kernel. While keeping the

default Arm installation behavior of the 4 KB kernel unchanged, it introduces the option to install the 64 KB kernel. By default, compatibility with upstream features is maintained. The 64 KB kernel improves OS performance in common scenarios. For compatibility constraints of the 64 KB kernel, see Appendix 3.

- **Programmable page cache for file systems:** A programmable page cache framework is implemented to address inefficient I/O during model loading in model inference scenarios. The framework is stacked on an existing file system in a transparent manner and forwards file system page faults to user space. This allows applications to implement custom page cache based on load characteristics in user space, significantly improving I/O efficiency for loading different models.
- **Cross-process zero-copy data transfer:** It provides efficient inter-process data transfer within a node. Applications can map pages from a specified virtual memory space in the source process to the virtual memory space of the destination process. It supports both page middle directory (PMD) hugepage mapping and page table entry (PTE) smallpage mapping. The mapped destination addresses are accessible with the same permissions as their corresponding source addresses.
- **Filesystem in Userspace (FUSE) over io_uring:** In the current FUSE architecture, communication between the user-space daemon and the kernel-space driver module via the character device `/dev/fuse` suffers from performance bottlenecks: (1) In multi-threaded environments, FUSE requests must be obtained through `/dev/fuse`, leading to lock contention. (2) I/O requests are transferred one by one, without support for batch processing, resulting in low efficiency for I/O-intensive tasks. (3) The user program initiating read/write requests and the backend FUSE thread handling file I/O often run on different CPU cores, causing frequent core switches and reduced efficiency. To address these bottlenecks, the FUSE over io_uring technology is employed, with the following solutions: (1) Replace `/dev/fuse` with the io_uring communication interface to improve concurrency. (2) Allow the backend FUSE service to flexibly use either io_uring or the traditional `/dev/fuse` character device for communication.
- **I/O QoS controller IOInflight for I/O co-location scenarios:** For details, see High-Density Many-Core Container Isolation.
- **epoll_wait asynchronous prefetch based on Xcall:** In performance optimization scenarios targeting specific system calls, dynamic Xcall leverages an interception mechanism to execute customized system calls. This allows users to intercept system calls at the granularity of application ELF files, without intrusive kernel modifications. Based on this mechanism, a kernel module for epoll-based asynchronous prefetch is implemented. It can achieve notable performance benefits in scenarios such as Redis.
- **Sockmap acceleration for same-host TCP flows:** In scenarios such as Redis and Nginx, a large amount of loopback TCP traffic traverses the full TCP protocol stack, resulting in high processing latency. With this feature, a BPF program replaces the ops of a socket pair, connects the send and receive queues of the two sockets, and directly redirects packets to the peer socket, bypassing the protocol stack. This reduces the packet forwarding path and lowers latency.
- **HiSock capability enhancement:** Based on the existing HiSock capabilities, this feature enhances HiSock for Internet scenarios, including local acceleration, enhanced packet parsing logic, packet capture mechanisms, address translation, and neighbor processing. It improves the performance and throughput of Kunpeng products when handling high-concurrency and low-latency network workloads.
- **copy_from_user optimization:** This capability temporarily disables PAN to allow LDP instructions to access user memory. It replaces the two-instruction LDTR load sequence with a single 16-byte LDP instruction. PAN is restored immediately after the copy operation completes, significantly reducing the number of instructions executed and improving copy throughput.

- **Dynamic SMT:** To improve overall CPU utilization, latency-sensitive tasks and best-effort tasks are deployed on the two SMT threads of the same physical core. However, best-effort and latency-sensitive tasks compete for underlying physical core resources, causing significant latency degradation for latency-sensitive tasks. To address this pain point, this solution balances SMT interference mitigation and overall CPU utilization improvement. It prioritizes latency-sensitive task execution, while automatically throttling best-effort tasks to release microarchitectural resources. Meanwhile, best-effort tasks can opportunistically run on the primary SMT thread and idle sibling threads to improve CPU utilization.
- **Network multipath RPS enhancement:** The network multipath feature binds NIC queue interrupts to CPUs on different NUMA nodes based on configured policies. By identifying traffic patterns of service processes, it enables the network traffic of a specified service to be preferentially received by the NIC queue on the NUMA node hosting the process. The enhanced RPS feature steers packets received from loopback and physical NICs to CPU cores within the same NUMA node or cluster as the service process based on the configured RPS policy. The former distributes interrupts across NIC queues on different NUMA nodes, while the latter further offload packet processing within the same NUMA node or cluster. Together, these two features achieve multi-core CPU load balancing while preserving NUMA and cluster affinity, effectively reducing memory access latency and improving network throughput.
- **NetKit:** It is a passthrough virtual networking device designed to eliminate latency spikes and tail latency effects caused by traditional veth pairs in high-concurrency and high-throughput scenarios. It addresses the overhead introduced by cross-namespace packet memory copies, frequent soft interrupts causing multi-core context switching, and unnecessary MAC address resolution and Layer 2 header encapsulation, which have historically contributed to network latency. The mechanism natively supports eBPF-based kernel-level traffic control and dynamic chained execution of multiple customized eBPF programs.
- **cgroup v2 enhancement:** This feature supports open source cgroup v2 to better meet the cgroup compatibility requirements of open source ecosystems such as Kubernetes. In addition, it introduces cgroup-level asynchronous memory reclamation based on the existing capabilities. This feature reduces the performance overhead caused by synchronous cgroup-level memory reclamation, mitigates performance jitter caused by short-lived memory spikes, and improves workload performance according to scenario requirements.
- **sched_ext programmable scheduling framework:** Based on eBPF technology, sched_ext enables dynamic loading and unloading of customized scheduling algorithms at runtime without requiring kernel recompilation. It reduces the scheduler development cycle and entry barrier, enabling highly optimized scheduling for specific workloads. The built-in security protection mechanism automatically switches back to the default system scheduler when a BPF program encounters an exception, preventing system hangs or failures.

6 Enhanced Features

Compilers

LLVM for openEuler

LLVM for openEuler is a high-performance compiler based on open source LLVM software. It is crafted for compute-intensive scenarios such as server and Internet industries, cutting-edge data center applications, AI computing systems, and video encoding and decoding. In addition, it establishes an LLVM baseline within the openEuler community, providing a stable downstream LLVM distribution that is secure, reliable, and easily innovated upon. It currently supports mainstream languages (C and C++) and chip architectures (x86, AArch64, RISC-V, and LoongArch).

Feature Description

The LLVM for openEuler compiler introduces key compilation features in openEuler 24.03 LTS SP4, enhancing build efficiency, reducing debug information bloat, and enabling full support for FlagGems operators via the Triton-CPU compiler.

Dwarfutils enhancement and debuginfo reduction: Thin Link-Time Optimization (ThinLTO) is a widely adopted optimization technique in modern compilers. In conventional compilation flows, each source file is compiled independently into an object file, and inter-file context is lost prior to linking, leaving significant room for further optimization. Under ThinLTO, source files are first compiled into LLVM Intermediate Representation (LLVM IR) files, which are then integrated and deeply optimized at the link time as if all code originated from a single monolithic file. However, standard ThinLTO drastically increases compile-time overhead, particularly when building large-scale applications. The ThinLTO Split technology resolves this by partitioning compilation units into smaller chunks for parallel compilation, significantly accelerating build speeds. To mitigate the side effect where enabling ThinLTO Split inflates debuginfo size several-fold—potentially causing address space overflow errors during linking—the dwarfutils tool has been integrated to strip redundant debuginfo, effectively reducing binary footprint while preserving debuggability.

Full support for FlagGems operators in Triton-CPU: Triton is a parallel programming language and compiler framework tailored for GPUs, allowing developers to write high-performance kernels using Python-like syntax. Built on top of the LLVM MLIR framework, Triton leverages LLVM's advanced optimization capabilities. Triton-CPU extends Triton to CPU backends, broadening its domain of application. Prior implementations, however, faced limited adaptation to AArch64 vector extensions (such as SVE and SME) and lacked support for commonly used FlagGems operators. This update delivers full compatibility with FlagGems operators and provides native affinity optimizations for AArch64 SVE/SME instruction sets, substantially enhancing Triton's practical deployment value on AArch64 architectures.

Application Scenarios

By pairing the ThinLTO Split technology of LLVM for openEuler with dwarfutils, development teams can drastically accelerate application compilation without sacrificing debugging capabilities, speeding up overall development and CI/CD pipelines. Triton-CPU enables applications originally developed for GPU ecosystems using Triton to seamlessly migrate to CPU environments. This satisfies developers' requirements for heterogeneous computing architectures and diverse CPU-based AI inference workloads.

Go for openEuler

Go for openEuler is a high-performance Go distribution designed for reliability and developer friendliness. It features an optimized compiler that offers wide ecosystem compatibility, openEuler affinity, and superior user experience. Primarily targeted at container cloud scenarios that require agile development and high performance, such as cloud native and microservice applications, Go for openEuler addresses native Go performance limitations to enhance efficiency for mainstream Go workloads. The compiler adapts to hardware platforms such as LoongArch and Kunpeng to unleash their computing power.

Feature Description

Key feature 1: LoopRotate optimization enhancement

The legacy LoopRotate optimization often introduces redundant jumps, resulting in negligible performance gains. By enhancing the LoopRotate pass, the compiler rearranges code block layouts to make them more compact. This eliminates redundant branch jumps, significantly improving execution efficiency for loop-heavy code.

Original loop structure ---> Legacy LoopRotate optimization ---> Enhanced LoopRotate



Key feature 2: Register-based parameter passing for bytealg functions

Certain assembly functions on the AArch64 platform previously relied on the ABI0 calling convention, causing extra execution overhead due to stack-based parameter passing. Upgrading from ABI0 to ABIInternal allows input and output parameters to be passed directly through CPU registers instead of stack pushes.

Key feature 3: LDP/STP instruction optimization

Replacing standard single-register load/store instructions (LDR/STR) with paired load/store instructions (LDP/STP) supported by the Kunpeng architecture significantly boosts data transfer throughput during sequential memory access operations.

Before

--->

After

str x1, [x0, #16]

stp x1, x2, [x0, #16]

str x2, [x0, #24]

Key feature 4: Memory block pre-clearing optimization

Previously, when Go objects were freed, the runtime merely modified their flags and deferred memory clearing until individual objects were allocated later, incurring frequent, fragmented clearing overhead. The enhanced runtime inspects and directly zeros out the entire **mspan** within the **nextFree** function while clearing its flags. Combining multiple small-scale zeroing operations into a single bulk-clearing operation delivers the following benefits:

- **Optimized caching mechanism:** Go's memory allocator manages memory across multi-level caches (**mcache** and **mcentral**). Coarsening the clearing granularity reduces cache allocation frequency and lock contention.
- **Reduced garbage collection overhead:** The Garbage Collector (GC) scans heap memory to identify unused blocks. Processing larger consolidated memory spans reduces the total block count during GC sweeps, lowering overall collection overhead.

Application Scenarios

Go for openEuler is an optimized Go core runtime, tailored for openEuler and other Linux environments. It is widely adopted across cloud-native infrastructure, software-defined storage (SDS), and cloud gaming services.

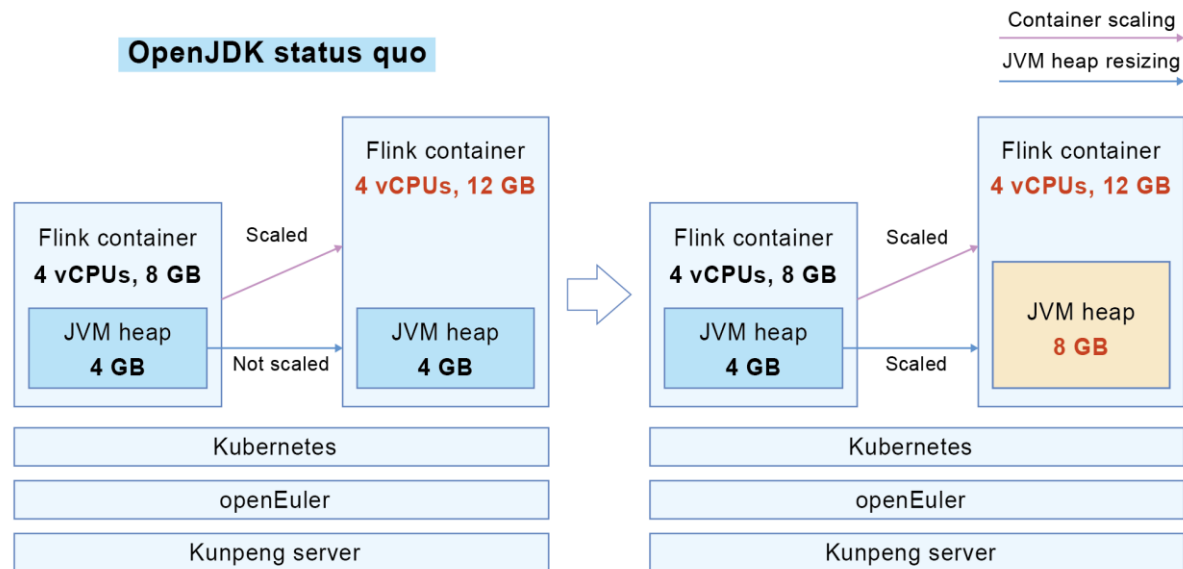
BiSheng JDK

Heap Resizing

Feature Description

In modern containerized deployments, most container environments support resource scale-up. However, OpenJDK 8 limits maximum heap size configuration to the startup stage only. This

restriction prevents Java applications from utilizing additional memory provided by container scaling without a process restart. To address this issue, BiSheng JDK 8 and 21 introduce online heap memory resizing for the Garbage-First Garbage Collector (G1GC) and Parallel Scavenge Garbage Collector (PSGC). This feature allows users to dynamically update the Java heap memory limit during runtime, eliminating the need for a JVM restart.



Application Scenarios

This capability is crucial for container-based deployments that require Java application heap memory to resize online in response to container resource scale-up. It has been implemented in JD big data scenarios.

BiSheng JDK 8 Enhancements

Compact Strings

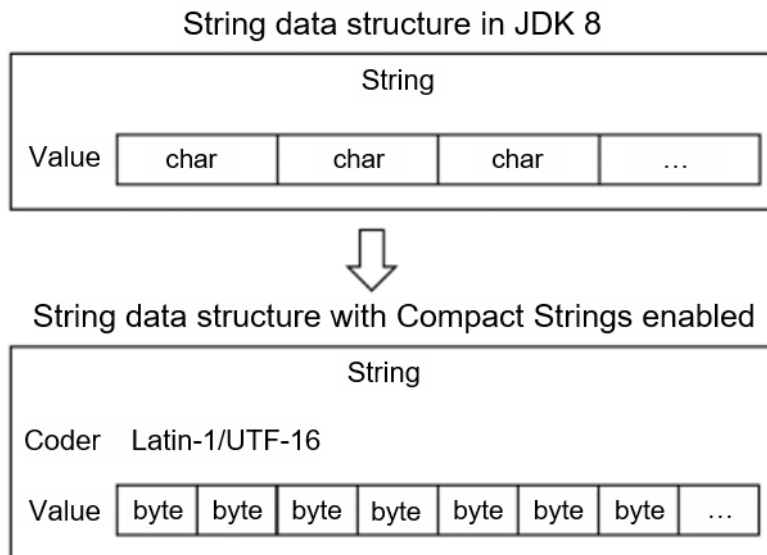
Compact Strings is a significant memory optimization feature introduced in Java 9, designed to improve string storage and reduce memory footprint. In Java 8 and earlier versions, strings were internally stored using a char array, where each character occupied 2 bytes (UTF-16 encoding), regardless of the actual content. This led to inefficient memory utilization when processing large volumes of strings containing only ASCII characters. Currently, Compact Strings has been backported from JDK 17 to BiSheng JDK 8, further enhancing the overall performance of BiSheng JDK 8.

Feature Description

Compact Strings replaces the internal representation of the string object from a UTF-16 char array to a byte array coupled with a coder field. This updated string class encodes characters as either Latin-1 (1 byte per character) or UTF-16 (2 bytes per character) based on the content of the string. The coder field indicates the specific encoding used for the string, as detailed below:

- Strings that contain only Latin-1 characters are stored in byte arrays. Each character occupies only one byte.

- Strings that contain non-Latin-1 characters (such as Chinese and Japanese) are still stored in char arrays. Each character occupies two bytes.
- Each string object contains a coder field, which indicates the encoding of this string. The coder field value can be:
 - **0** (Latin-1): Contains only Latin-1 characters.
 - **1** (UTF-16): Contains non-Latin-1 characters.



Application Scenarios

Compact Strings enables significant memory savings in string storage. For strings containing only Latin-1 characters, memory consumption is reduced by approximately 50%. Therefore, Compact Strings is highly effective for string-intensive applications, which further reduces GC frequency and enhances overall system performance.

JProfileCache

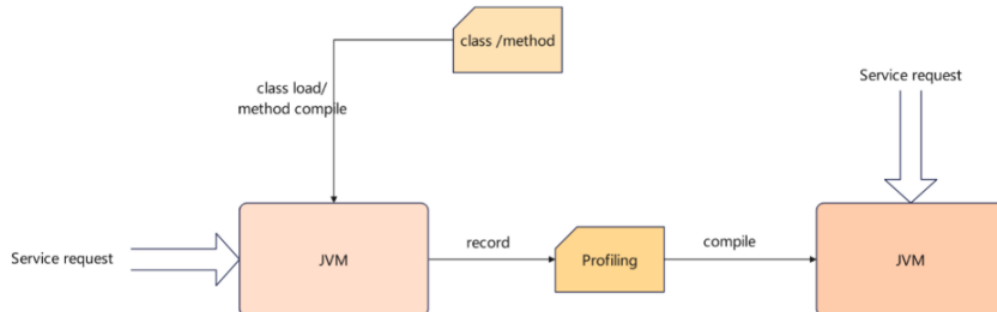
During the startup phase of Java applications, concurrent execution of Just-In-Time (JIT) compilation for hot methods and service request processing creates intense CPU resource contention. This compilation lag often leads to a prolonged "warm-up" period, slowing down initial system performance ramp-up. The JProfileCache feature in BiSheng JDK addresses this by collecting profiling data from prior application runs. Upon subsequent restarts, it preemptively compiles hot methods before user traffic arrives, allowing the Java process to reach peak performance almost instantly.

Feature Description

The BiSheng JDK JProfileCache workflow operates across two distinct phases: profiling and pre-compilation.

- **Profiling phase:** During live production runs, the JVM accepts incoming requests while tracking methods compiled by the JIT compiler. This profiling data is exported and saved to an external profile file.
- **Pre-compilation phase:** On subsequent application launches, the JVM ingests the recorded profile file and proactively triggers JIT compilation for the identified hot methods.

By translating hot methods into optimized native code prior to receiving user traffic, JProfileCache eliminates compilation overhead during traffic spikes, lowering CPU usage and accelerating application ramp-up.



Application Scenarios

JProfileCache is designed for latency-sensitive and throughput-critical environments where fast startup and minimal warm-up times are essential, such as big data analytics with Spark.

AutoCDS

Conventional Application Class Data Sharing (AppCDS) requires a multi-step manual workflow, which is error-prone and complex to integrate into modern deployment pipelines. However, AutoCDS in this new release automates its configuration process. By specifying a single configuration path, the JVM automatically detects its execution stage, dynamically applies the necessary parameters, and executes the appropriate workflow. This simplifies the conventional three-step AppCDS setup into a single step, eliminating manual intervention and configuration errors.

Feature Description

When a Virtual Machine (VM) instance is created, AutoCDS checks for the **-XX:AutoSharedArchivePath** parameter:

- **Disabled:** The JVM follows standard startup logic without any behavior change.
- **Enabled:** The JVM saves the generated **appcds.lst** and **appcds.jsa** files to the specified path (**base_path**).

The automated execution follows a three-step logic based on file presence:

1. **Check for the .lst file:** The JVM checks if **appcds.lst** exists in **base_path**. If missing, the application launches with **-XX:+UseAppCDS -XX:DumpLoadedClassList=base_path/appcds.lst -Xshare:off**. Upon application exit, **appcds.lst** is saved automatically.
2. **Check for the .jsa file:** If **appcds.lst** exists but **appcds.jsa** is missing, the JVM triggers archive creation by forking a child process to compile **appcds.lst** into **appcds.jsa**.
3. **Enable AppCDS:** If **appcds.jsa** is present, the JVM directly enables AppCDS using this pre-generated archive, completing the AppCDS enablement process.

Application Scenarios

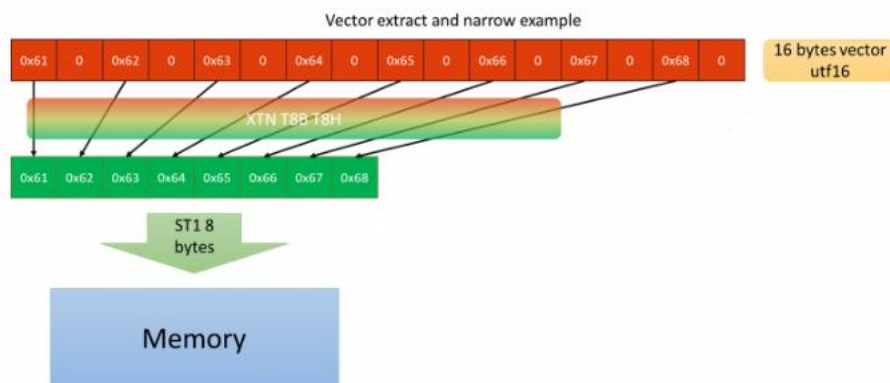
AutoCDS is designed for scenarios sensitive to JVM startup overhead, accelerating JVM startup and class loading, such as in big data Spark workloads.

UTF-8/16 Encoding and Decoding Vectorization

Currently exclusive to the AArch64 architecture, this feature harnesses the Single Instruction, Multiple Data (SIMD) vectorization capabilities of the Kunpeng platform to parallelize byte loading, conversion, and write-back. By accelerating execution along hot paths, it drastically improves processing efficiency. Inputs meeting vectorization criteria follow the high-performance intrinsic path, while non-conforming scenarios (such as malformed surrogates) gracefully fall back to the existing scalar implementation to ensure functional correctness and full compatibility.

Feature Description

Drawing inspiration from the native simdutf library, this feature intrinsifies UTF-8 and UTF-16 conversion pipelines for batch processing. It leverages SIMD instructions to process multiple bytes or 16-bit code units in a single cycle. Utilizing Kunpeng vector instructions such as **ld1**, **st1**, **xtn**, **sli**, and **usra**, it performs parallel loading, conversion, and write-back, significantly reducing loop iterations and conditional branches compared to conventional character-by-character processing.



Application Scenarios

This feature is designed for big data, microservices, and other workloads involving heavy string serialization and deserialization (SerDe). During network transmission, strings are commonly encoded as UTF-8 to reduce bandwidth consumption. Upon arrival, the receiving end decodes these UTF-8 streams into UTF-16 for efficient in-memory indexing.

BiSheng JDK 17 Enhancements

De-optimization Observability

The JDK Flight Recorder (JFR) Streaming API of JDK 17 is a key feature that enables JFR to move from post-event static analysis to real-time monitoring.

In conventional JFR usage, a user must start recording, stop recording, dump contents into a **.jfr** file, and finally use Java Mission Control (JMC) tools for offline analysis. This is a post-event analysis mode, which is effective for troubleshooting problems that have occurred.

The Streaming API introduces a new mode, which allows a Java application to continuously subscribe to and consume JFR event streams from the JVM in real time without interrupting JFR recording or generating a complete **.jfr** file.

Feature Description

When using the Streaming API, a user can obtain JFR events such as de-optimization events before the current time from positions marked as *********.

// 1. Create a recording stream.

```
RecordingStream rs = new RecordingStream();
```

// 2. Enable events of interest and configure related settings.

```
rs.enable("jdk.GCPhasePause").withPeriod(Duration.ofSeconds(1));
```

```
rs.enable("jdk.Deoptimization").withPeriod(Duration.ofSeconds(1));
```

// 3. Subscribe to an event and set corresponding event handler (callback function).

```
rs.onEvent("jdk.GCPhasePause", event -> {
```

```
// Read the following fields from the event.
```

```
Duration = event.getDuration("duration");
```

```
String name = event.getString("name"); // For example, "GC Pause"
```

```
*****
```

```
shell: jcmd JFR.start delay=-1 filename=xxx.jfr
```

```
*****
```

```
});
```

// 4. Start the stream (non-blocking call).

```
rs.startAsync();
```

Application Scenarios

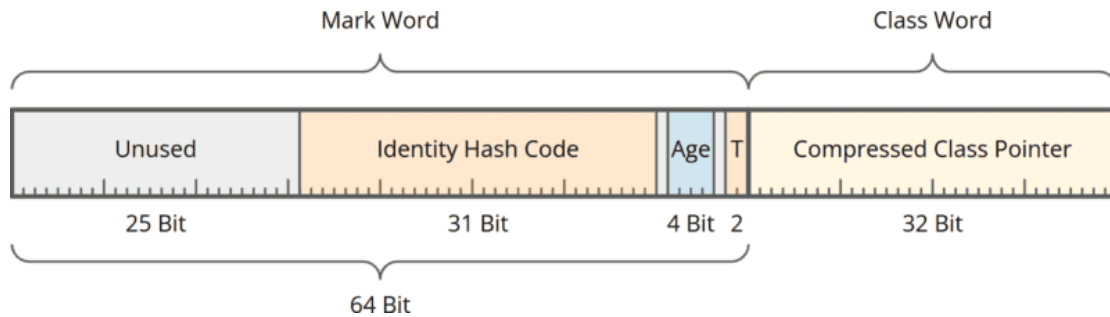
The combination of online monitoring and post-event analysis ensures that the JFR file captures all events within a specific period preceding the current moment.

Metadata Compression

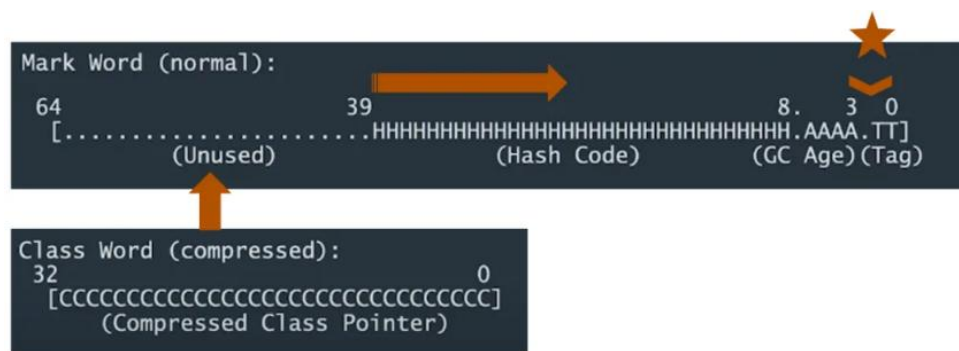
In the JVM, every object incurs a storage overhead known as the object header, which houses critical runtime metadata. As the number of live objects grows, particularly in workloads dominated by small objects, this header overhead becomes a major contributor to memory bloat. To address this challenge, the Compressed Object Headers feature, derived from OpenJDK's Project Lilliput, reduces the HotSpot JVM object header size from 128 or 96 bits down to 64 bits. This reduction noticeably shrinks Java heap memory footprints while delivering a performance boost across application workloads.

Feature Description

A standard Java object header comprises a 64-bit Mark Word and a 32-bit or 64-bit Class Word (plus array length for array objects). The Mark Word stores dynamic runtime state, including GC age, identity hash code, and locking status. The original Mark Word bit layout is as follows.



By exploiting the fact that the high-order bits of the Mark Word are typically unused, this feature packs the Class Word into the high-order bits of the Mark Word. Combined with a shortened identity hash code and reconfigured header mark bits, the entire header is compacted into a single 64-bit word.



Key modifications include:

- Relocates the Class Word into the high-order bits of the Mark Word. This design relies on compressed class pointers (**-XX:+UseCompressedClassPointers**), a prerequisite enabled by default in the vast majority of Java applications.
- Reduces the identity hash code field to 25 bits. Applications relying on extremely large hash tables may experience minor performance degradation.
- Introduces a 1-bit self-forwarded tag to represent GC copy failure, replacing the legacy mechanism of overwriting the Mark Word with a self-referencing pointer.



Legacy array headers consume 96 or 128 bits (depending on the compressed class pointer state) with a 16-byte alignment. Under compressed object headers, reference and primitive arrays (including **boolean**, **byte**, **char**, **short**, **int**, and **float**) combine the Mark Word, Class Word, and array length into 12 bytes, shifting the alignment baseline to 12 bytes to minimize padding waste. For example, a 1-element **int** array that previously required 24 bytes now consumes only 16 bytes, achieving a 33% reduction in memory footprint. For workloads creating large volumes of such small arrays, total memory footprint reduction is substantial.

Application Scenarios

Ideal for memory-intensive applications with high allocation volumes of small objects. The higher the proportion of small objects in the heap, the more pronounced the memory savings achieved by this optimization.

JProfileCache

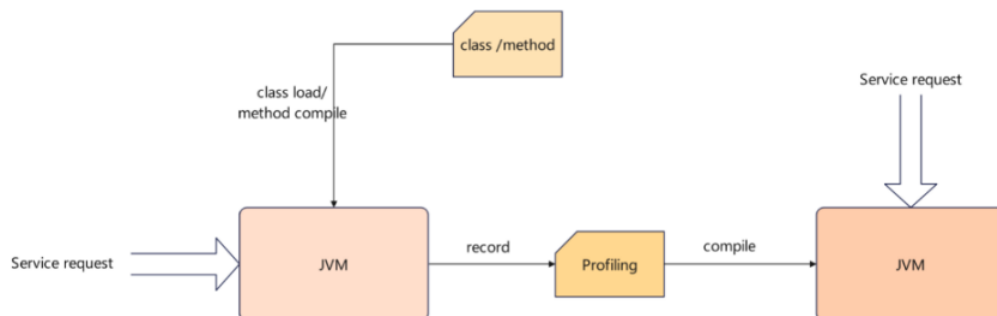
During the startup phase of Java applications, concurrent execution of Just-In-Time (JIT) compilation for hot methods and service request processing creates intense CPU resource contention. This compilation lag often leads to a prolonged "warm-up" period, slowing down initial system performance ramp-up. The JProfileCache feature in BiSheng JDK addresses this by collecting profiling data from prior application runs. Upon subsequent restarts, it preemptively compiles hot methods before user traffic arrives, allowing the Java process to reach peak performance almost instantly.

Feature Description

The BiSheng JDK JProfileCache workflow operates across two distinct phases: profiling and pre-compilation.

- **Profiling phase:** During live production runs, the JVM accepts incoming requests while tracking methods compiled by the JIT compiler. This profiling data is exported and saved to an external profile file.
- **Pre-compilation phase:** On subsequent application launches, the JVM ingests the recorded profile file and proactively triggers JIT compilation for the identified hot methods.

By translating hot methods into optimized native code prior to receiving user traffic, JProfileCache eliminates compilation overhead during traffic spikes, lowering CPU usage and accelerating application ramp-up.



Application Scenarios

JProfileCache is designed for latency-sensitive and throughput-critical environments where fast startup and minimal warm-up times are essential, such as big data analytics with Spark.

AutoCDS

Conventional Application Class Data Sharing (AppCDS) requires a multi-step manual workflow, which is error-prone and complex to integrate into modern deployment pipelines. However, AutoCDS in this new release automates its configuration process. By specifying a single configuration path, the JVM automatically detects its execution stage, dynamically applies the necessary parameters, and executes the appropriate workflow. This simplifies the conventional three-step AppCDS setup into a single step, eliminating manual intervention and configuration errors.

Feature Description

When a Virtual Machine (VM) instance is created, AutoCDS checks for the **-XX:AutoSharedArchivePath** parameter:

- **Disabled:** The JVM follows standard startup logic without any behavior change.
- **Enabled:** The JVM saves the generated **appcds.lst** and **appcds.jsa** files to the specified path (**base_path**).

The automated execution follows a three-step logic based on file presence:

1. **Check for the .lst file:** The JVM checks if **appcds.lst** exists in **base_path**. If missing, the application launches with **-XX:+UseAppCDS -XX:DumpLoadedClassList=base_path/appcds.lst -Xshare:off**. Upon application exit, **appcds.lst** is saved automatically.
2. **Check for the .jsa file:** If **appcds.lst** exists but **appcds.jsa** is missing, the JVM triggers archive creation by forking a child process to compile **appcds.lst** into **appcds.jsa**.
3. **Enable AppCDS:** If **appcds.jsa** is present, the JVM directly enables AppCDS using this pre-generated archive, completing the AppCDS enablement process.

Application Scenarios

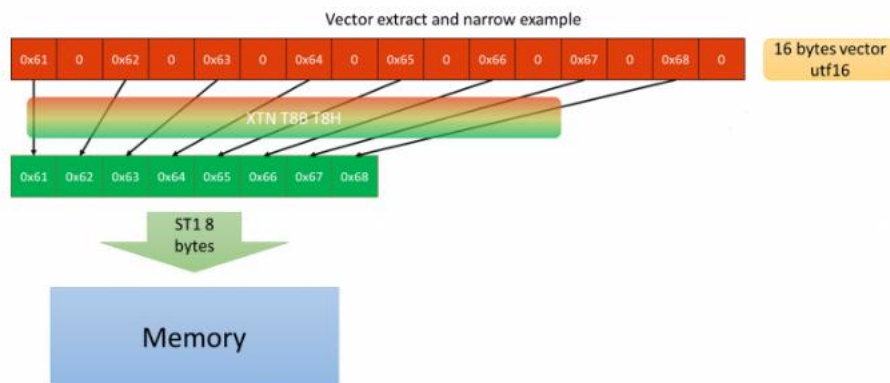
AutoCDS is designed for scenarios sensitive to JVM startup overhead, accelerating JVM startup and class loading, such as in big data Spark workloads.

UTF-8/16 Encoding and Decoding Vectorization

Currently exclusive to the AArch64 architecture, this feature harnesses the Single Instruction, Multiple Data (SIMD) vectorization capabilities of the Kunpeng platform to parallelize byte loading, conversion, and write-back. By accelerating execution along hot paths, it drastically improves processing efficiency. Inputs meeting vectorization criteria follow the high-performance intrinsic path, while non-conforming scenarios (such as malformed surrogates) gracefully fall back to the existing scalar implementation to ensure functional correctness and full compatibility.

Feature Description

Drawing inspiration from the native `simdutf` library, this feature intrinsifies UTF-8 and UTF-16 conversion pipelines for batch processing. It leverages SIMD instructions to process multiple bytes or 16-bit code units in a single cycle. Utilizing Kunpeng vector instructions such as **ld1**, **st1**, **xtn**, **sli**, and **usra**, it performs parallel loading, conversion, and write-back, significantly reducing loop iterations and conditional branches compared to conventional character-by-character processing.



Application Scenarios

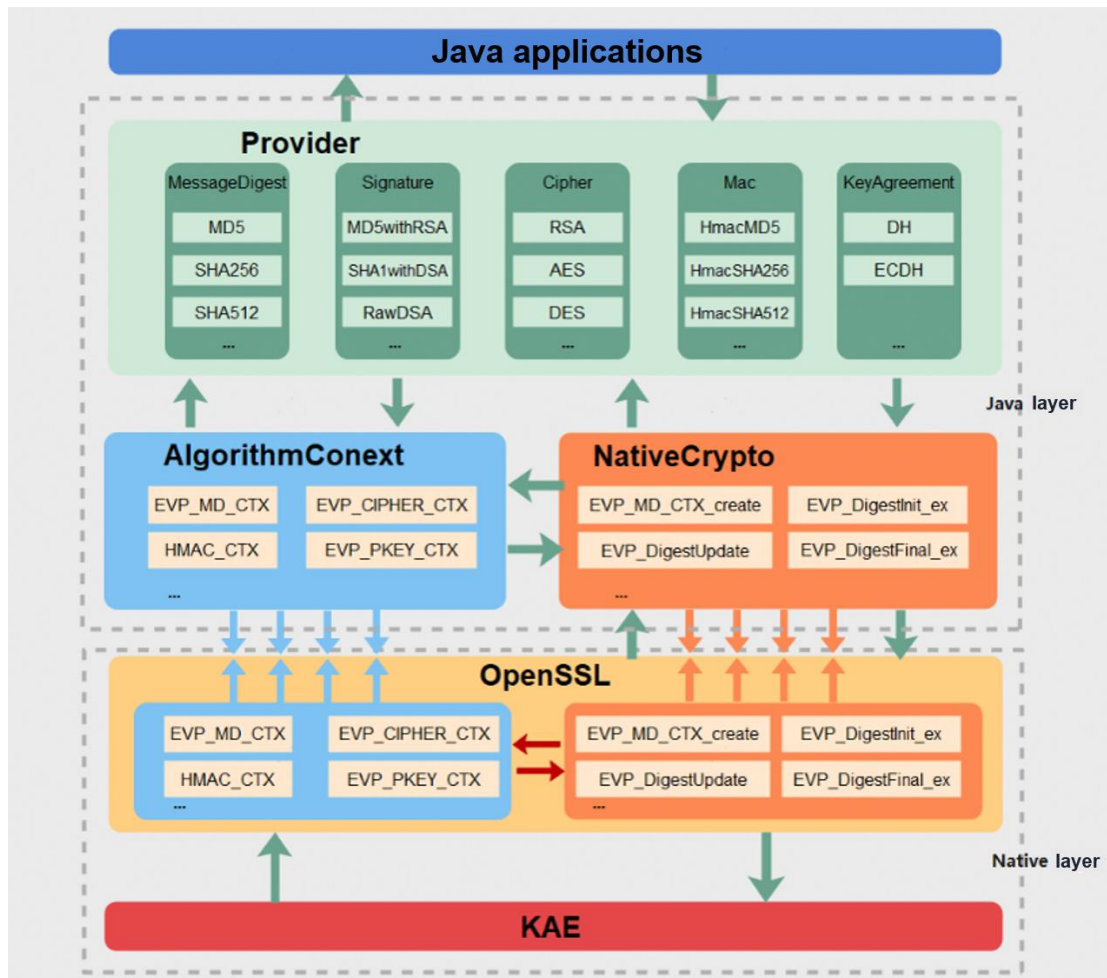
This feature is designed for big data, microservices, and other workloads involving heavy string serialization and deserialization (SerDe). During network transmission, strings are commonly encoded as UTF-8 to reduce bandwidth consumption. Upon arrival, the receiving end decodes these UTF-8 streams into UTF-16 for efficient in-memory indexing.

BiSheng JDK 21 Enhancements

KAE Support

KAE encryption and decryption is a module of the Kunpeng Accelerator Engine (KAE). It invokes the underlying KAE hardware by modifying the OpenSSL library.

Therefore, application code only needs to call the OpenSSL library, and KAE offers acceleration capabilities as the underlying engine of OpenSSL. KAEProvider is located between the application and OpenSSL and provides a convenient way to use the OpenSSL library and enable KAE acceleration.



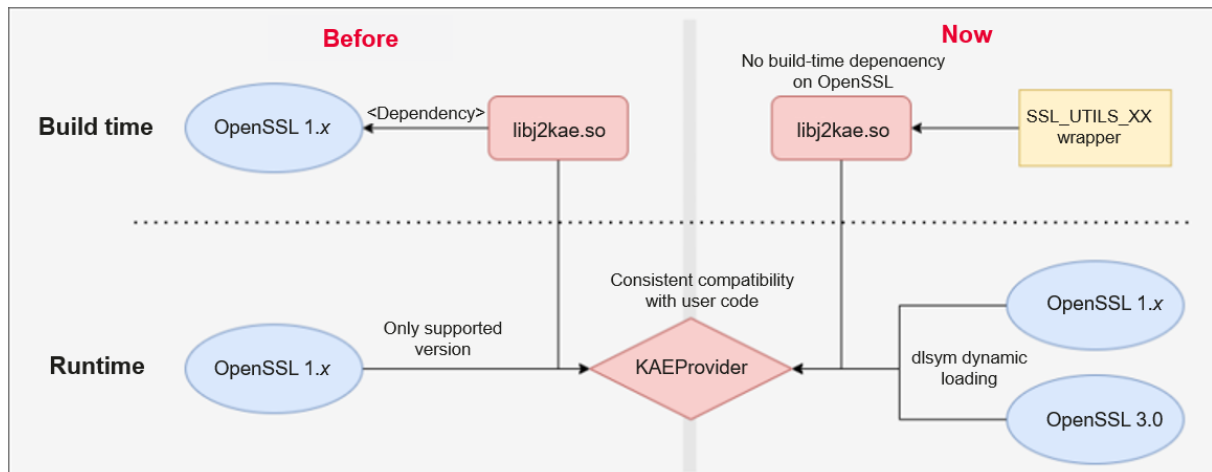
The KAE compression module supports compression of zlib and gzip data formats, increasing the compression bandwidth. In Java applications, compression and decompression features are extensively utilized. Especially in data storage and HTTP services, the CPU usage for compression can exceed 70%, making the compression speed of open source zlib a significant bottleneck. If KAEZlib is enabled for Java applications, service performance will be greatly improved.

Feature Description

KAESProvider supports OpenSSL 1.x. In 2021, OpenSSL 3.0 is released, with significant enhancements made to its architecture, scalability, and security, making it a better choice for secure communication applications. However, OpenSSL 3.0 has deprecated, added, or modified several APIs. KAESProvider cannot directly support OpenSSL 3.0 and code adaptation is required.

After this feature is added, the KAESProvider build result remains unchanged, but the build process does not depend on a local OpenSSL library in the build environment (previously, building the target `libj2kae.so` required a local OpenSSL 1.x environment). Instead, the specific OpenSSL version to be used is dynamically determined and loaded at runtime.

The usage of KAESProvider remains unchanged. Users need to manually enable the feature by modifying the **OPENSSL_ENGINES** environment variable and the **kae.useOpenSSLVersion** configuration. The original encryption and decryption logic remains unchanged, and the original functions and usage patterns of the JDK are not affected.



For KAEZip support, the **libzip.so** dynamic library is split into **libzip.so** and **libz.so** in the JDK, which contain the JNI and zlib code, respectively. To enable KAEZlib, use the **-DGZIP_USE_KAE** parameter. During compression and decompression, the Java side does not process the file header and tail. The generation and parsing of the compressed data structure depend on the zlib and gzip compression algorithms of the underlying zlib library.

Application Scenarios

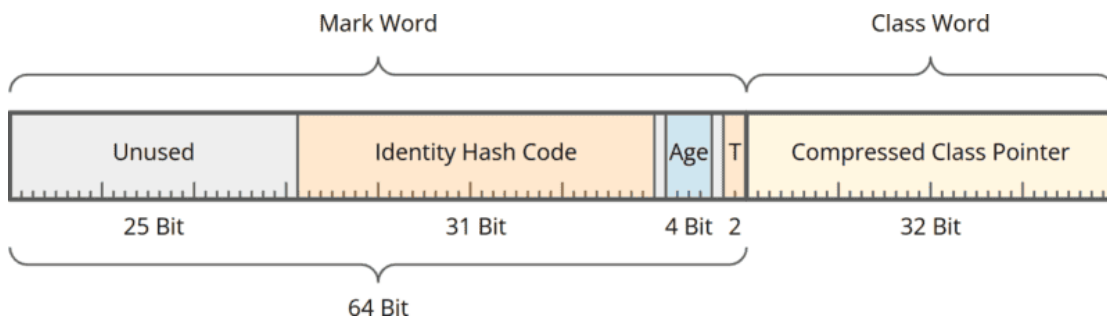
KAEProvider and KAEZip can effectively improve algorithm performance in encryption, decryption, compression, and decompression scenarios.

Metadata Compression

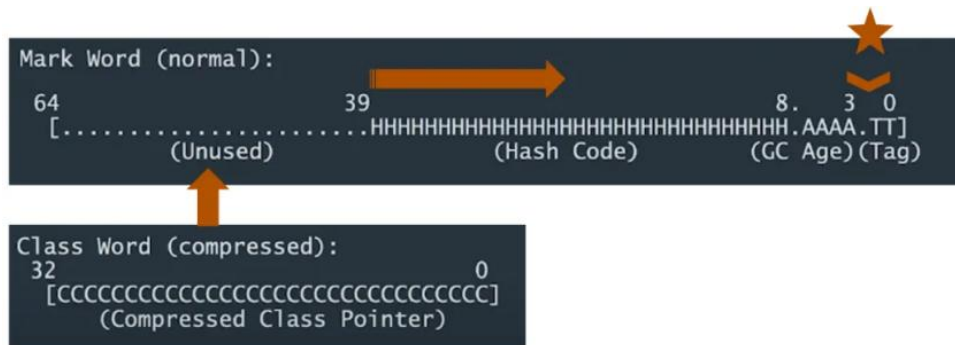
In the JVM, every object incurs a storage overhead known as the object header, which houses critical runtime metadata. As the number of live objects grows, particularly in workloads dominated by small objects, this header overhead becomes a major contributor to memory bloat. To address this challenge, the Compressed Object Headers feature, derived from OpenJDK's Project Lilliput, reduces the HotSpot JVM object header size from 128 or 96 bits down to 64 bits. This reduction noticeably shrinks Java heap memory footprints while delivering a performance boost across application workloads.

Feature Description

A standard Java object header comprises a 64-bit Mark Word and a 32-bit or 64-bit Class Word (plus array length for array objects). The Mark Word stores dynamic runtime state, including GC age, identity hash code, and locking status. The original Mark Word bit layout is as follows.



By exploiting the fact that the high-order bits of the Mark Word are typically unused, this feature packs the Class Word into the high-order bits of the Mark Word. Combined with a shortened identity hash code and reconfigured header mark bits, the entire header is compacted into a single 64-bit word.



Key modifications include:

- Relocates the Class Word into the high-order bits of the Mark Word. This design relies on compressed class pointers (**-XX:+UseCompressedClassPointers**), a prerequisite enabled by default in the vast majority of Java applications.
- Reduces the identity hash code field to 25 bits. Applications relying on extremely large hash tables may experience minor performance degradation.
- Introduces a 1-bit self-forwarded tag to represent GC copy failure, replacing the legacy mechanism of overwriting the Mark Word with a self-referencing pointer.



Legacy array headers consume 96 or 128 bits (depending on the compressed class pointer state) with a 16-byte alignment. Under compressed object headers, reference and primitive arrays (including **boolean**, **byte**, **char**, **short**, **int**, and **float**) combine the Mark Word, Class Word, and array length into 12 bytes, shifting the alignment baseline to 12 bytes to minimize padding waste. For example, a 1-element **int** array that previously required 24 bytes now consumes only 16 bytes, achieving a 33% reduction in memory footprint. For workloads creating large volumes of such small arrays, total memory footprint reduction is substantial.

Application Scenarios

Ideal for memory-intensive applications with high allocation volumes of small objects. The higher the proportion of small objects in the heap, the more pronounced the memory savings achieved by this optimization.

JIT Warmup

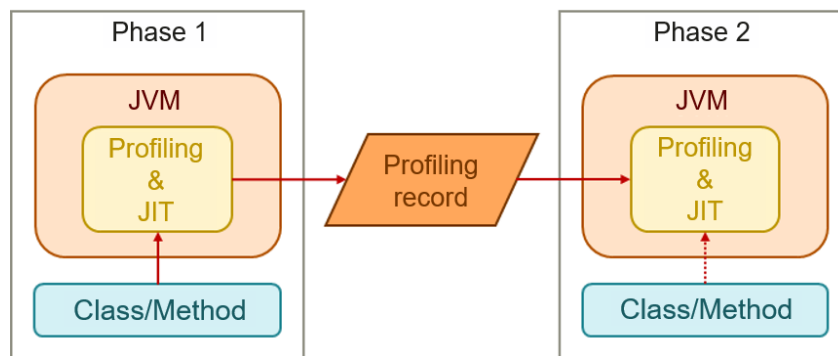
This feature enables Java processes to reach peak performance rapidly after startup. The transition from process initiation to peak performance is traditionally a lengthy procedure,

involving interpreter execution, profiling, C1 compilation, further profiling, and C2 compilation. This workflow integrates profiling and online just-in-time (JIT) compilation, and constitutes a critical part of application warmup. This feature effectively accelerates VM startup and warmup by streamlining this workflow, helping applications reach peak performance faster.

Feature Description

The JProfileCachesolution of the BiSheng JDK divides application release into two phases:

- **Recording:** At the conclusion of program execution, the profiling information of hotspot methods—primarily method invocation counts and back-edge counts—along with the associated classes (including parent classes) are exported to a specified file.
- **Pre-compilation:** In the next startup, the JVM reads the classes containing the hotspot methods and performs pre-loading. Simultaneously, these hotspot methods are added to the compilation queue for ahead-of-time (AOT) compilation. This approach bypasses the profiling stage and compiles the hotspot methods into high-performance native code before a user request arrives, thereby reducing the warmup overhead.



Application Scenarios

This feature is particularly suited for cloud-native scenarios with stringent startup speed requirements, as well as Spark big data environments involving the repeated initiation of driver and executor processes. In these contexts, this feature effectively accelerates VM startup and application warmup, enabling applications to reach their peak performance rapidly.

ANNC

Accelerated Neural Network Compiler (ANNC) is an AI compiler designed for accelerating neural network computations. It focuses on computation graph optimization, high-performance fused operator generation, and efficient code generation and optimization to boost the inference performance of models like recommendation systems and LLMs. It can integrate with mainstream open source inference frameworks and various hardware backends to enhance software extensibility.

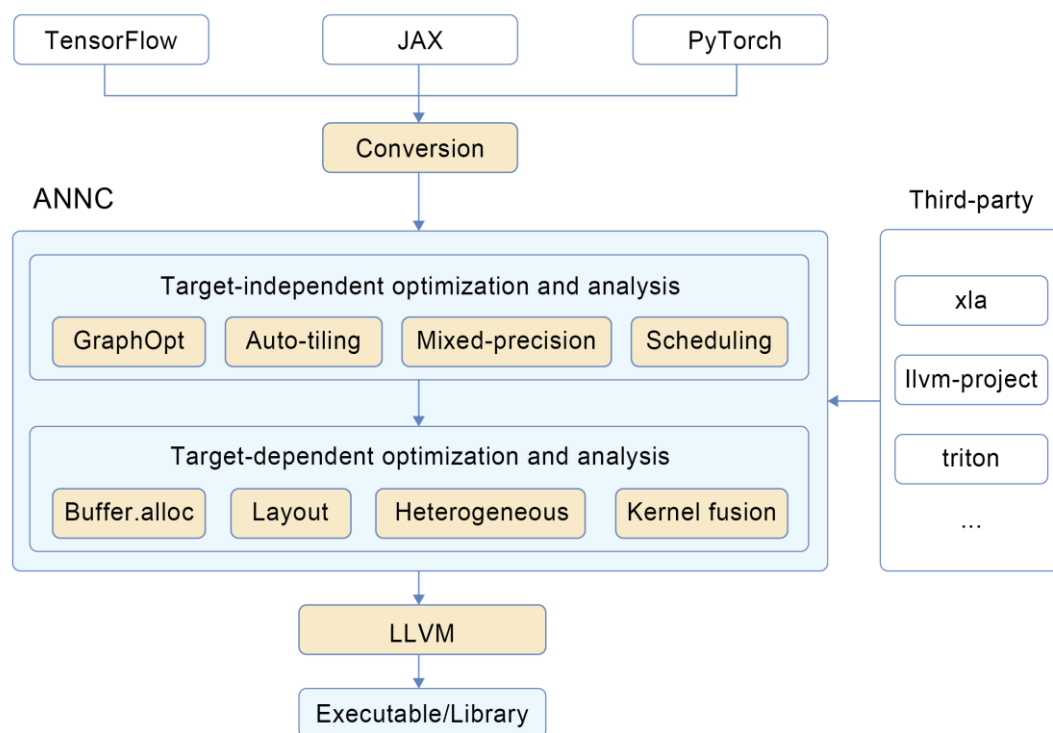
Feature Description

Computation graph optimization refines the neural network's computational flow. From an algorithmic perspective, it reduces redundant operations, performs mixed-precision rewriting, and automatically schedules subgraphs to lower the computational load and improve cache utilization. From a hardware architecture perspective, it optimizes tensor data layouts, operator fusion and conversion, and subgraph partitioning and scheduling to further lower the load and fully utilize hardware resources.

Tailored to the severe bottleneck and high hot-data ratio of embedding layers in recommendation models, ANNC introduces operator fusion and constant folding. These features eliminate redundant memory transfers and accesses caused by standalone operator executions, significantly accelerating inference performance.

Generating and integrating a high-performance fused operator library comprises three parts: front-end computation graph pattern recognition and conversion, high-performance operator library query and integration, and automated operator library generation. At the assembly instruction level, it reduces memory access and accelerate parallel computations through optimization techniques like data prefetching, model parallelism, and new instruction sets.

ANNC aims to accelerate AI inference and reduce power consumption through graph compilation optimization and high-performance operator generation and integration, thereby improving inference efficiency per unit cost. In addition, its software compatibility and ease-of-use design reduce operational costs and environmental impact.



Application Scenarios

ANNC now focuses on graph compilation optimization and operator library selection and invocation. It yields significant benefits for coarse- and fine-ranking models in mainstream search and recommendation systems. Graph compilation optimization achieves even better results for complex embedding layer networks that handle intricate feature processing.

Moreover, ANNC has also laid the groundwork for performance optimization in future scenarios, such as LLMs. By integrating with LLM inference frameworks and combining existing graph-operator fusion and memory layout optimization techniques with core performance optimization methods like GEMM, it aims to reduce inference latency and increase inference throughput.

Security

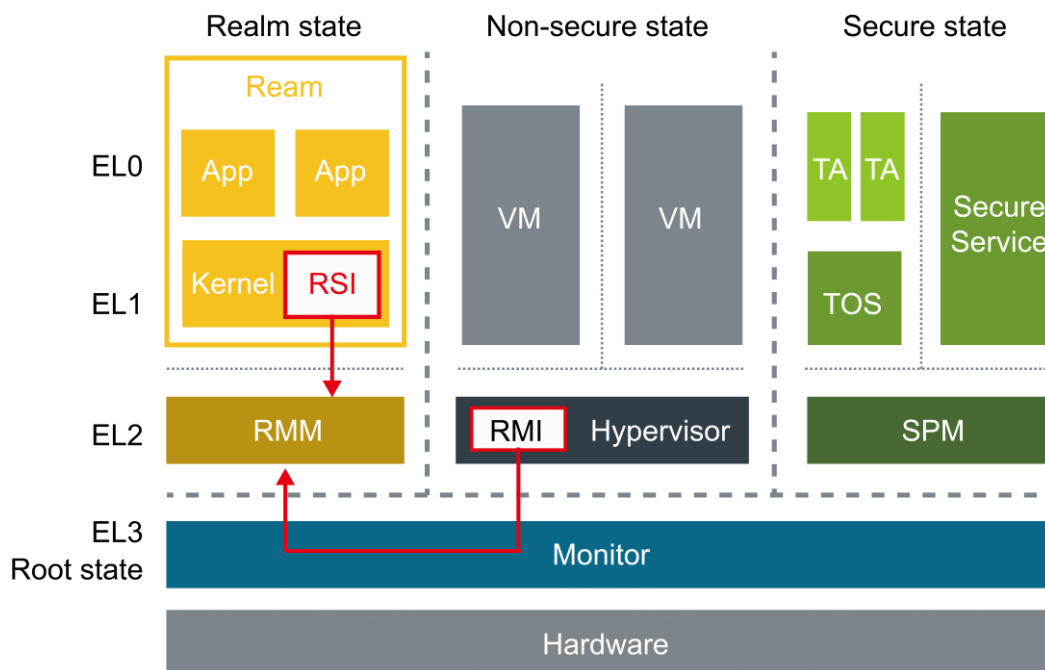
CCA

Arm Confidential Compute Architecture (CCA) is a specification newly introduced in Armv9. It is designed to define a standard confidential computing solution for next-gen computing devices. CCA establishes realms as TEEs to protect the confidentiality and integrity of data and code in use, ensuring effective protection even against privileged infrastructure software or cloud service providers.

Based on the Arm CCA specification, openEuler has implemented support for CCA across its relevant OS components, including KVM, QEMU, libvirt, and the guest kernel. The openEuler community edition provides native support for realm cVMs, fulfilling the security requirement to protect data in use. Crucially, this implementation also offers the ease-of-use and compatibility necessary for seamless integration with traditional application ecosystems and existing VM management software.

Feature Description

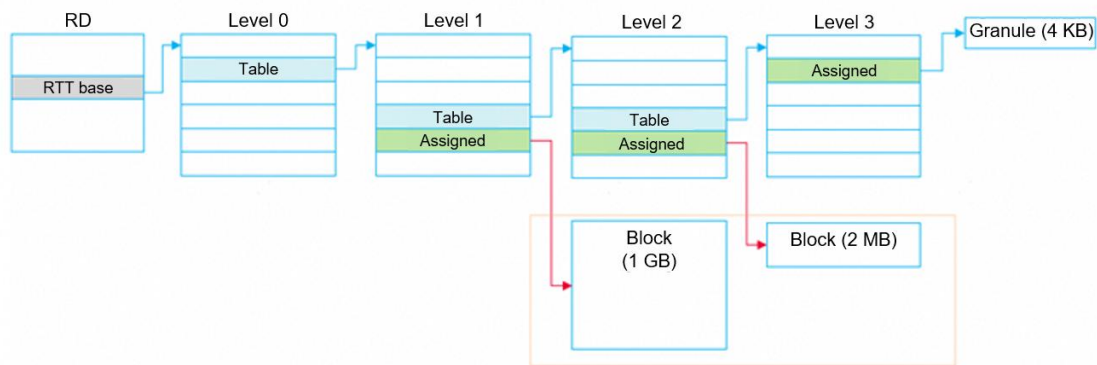
Arm CCA utilizes the following core components working in synergy to construct a realm, an isolated and protected execution space. The realm is completely segregated from the normal world in terms of code execution and data access.



- **Realm:** A realm is the core abstraction of CCA. It is a new execution environment that runs parallel to the normal world and secure world (TrustZone). It is hardware-isolated and designed to host sensitive code and data. It is independent of the host OS and hypervisor, which can manage a realm but cannot access the content within.
- **Dynamic management:** The hypervisor can dynamically create realms and allocate memory and CPU resources to them as required. However, once a realm is initialized, the hypervisor hands over control to the Realm Management Monitor (RMM), a protected secure virtualization module, and can no longer access the data within the realm.

- **Memory management:** CCA extends the system memory management unit (MMU) to identify and isolate realm memory. Any access attempt from outside the realm (including the hypervisor) is blocked by the hardware, ensuring data confidentiality.
- **Remote attestation:** Each CCA-enabled processor has a unique, hardware-based identity. When a realm starts, it can generate an attestation token that is cryptographically signed by the hardware. Users can obtain this report, verify its signature, and check the component measurements to ensure that their workloads are running in a genuine, unaltered Arm CCA environment.

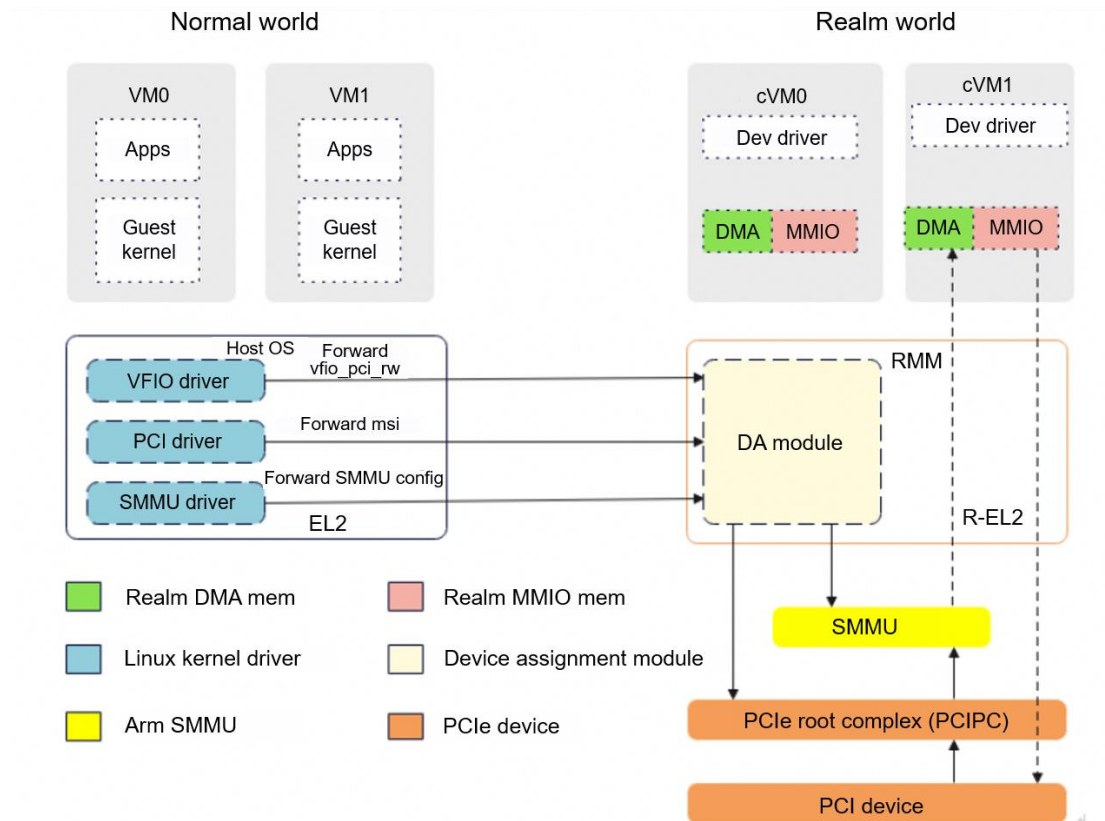
During the default boot process of a CCA cVM, the RMM typically manages the state of protected memory at a fixed 4 KB granularity and builds Stage-2 page tables based on this granularity. This process introduces key performance overheads, including high page table traversal costs, heavy TLB pressure, and frequent RMM interface calls. Building Stage-2 page tables for CCA cVMs based on 1 GB or 2 MB huge pages can significantly mitigate these performance bottlenecks. openEuler supports choosing between 1 GB or 2 MB page mappings based on the intermediate physical address (IPA) alignment and the physical page continuity of CCA cVMs.



CCA features the following advantages when building Stage-2 page tables based on 1 GB or 2 MB huge pages:

- **Shortened traversal paths:** Huge pages (1 GB or 2 MB) allow mapping to be completed at a higher level, which shortens the Stage-2 page table traversal paths of CCA cVMs and reduces page table access latency.
- **Improved TLB hit rate:** Huge pages occupy fewer TLB entries, thereby reducing TLB misses and page table lookup overhead for CCA cVMs.
- **Accelerated realm startup and memory initialization:** Mapping large blocks of physical memory in batches significantly shortens the creation time of CCA cVMs.

CCA cVMs rely on Kunpeng processors to achieve confidential passthrough for PCIe devices via a PCIe protection controller (PCIPC) embedded in the PCIe root complex. In confidential computing scenarios, PCIPC-protected PCIe devices can be passed through into the realm domain. Data is transferred without intermediate relays or copying, thereby ensuring the confidentiality of the device data path. The architecture of CCA confidential device passthrough is shown below.



CCA PCIe device confidential passthrough has the following advantages:

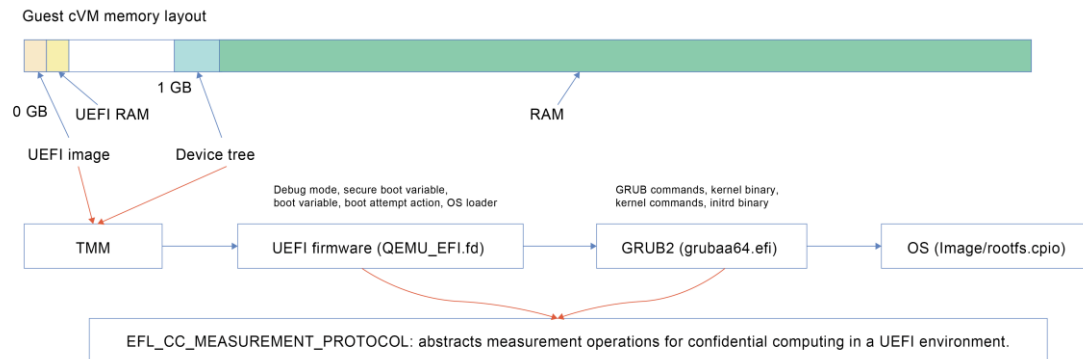
- **Secure isolation**
Once the CCA capability is enabled, PCIPC-protected secure devices can only be accessed from the realm side, and are inaccessible to host-side software.
- **High performance**
Device passthrough for realm cVMs eliminates performance loss on the data plane compared to industry encryption and decryption solutions.
- **Ease of use**
Compatibility with existing open source OSs removes the requirement for kernel driver modifications.

Application Scenarios

cVMs are mainly used in cloud computing environments to provide high-level security isolation for core workloads. They enable the processing of sensitive data (such as financial records, healthcare information, and intellectual property) on untrusted public clouds while ensuring the confidentiality and integrity of the data. These cVMs can block any unauthorized access, even from cloud service providers. This effectively meets the key requirements of data sovereignty, regulatory compliance, and cross-agency privacy data collaboration (such as joint modeling and analysis).

virtCCA

Feature Description



The current virtCCA architecture has this constraint: it only supports a boot mode where the kernel and rootfs are mounted separately. However, in mainstream cloud platforms, VMs typically rely on a GRUB bootloader. This requires integrating the Unified Extensible Firmware Interface (UEFI) firmware (like EDK2), kernel, and initial RAM file system (initramfs) into a single image, such as a QCOW2 file. The enhanced virtCCA addresses this by providing the following functions:

- Single image encapsulation
 - **Unified boot stack:** virtCCA integrates the EDK2 firmware, GRUB bootloader, kernel, and initramfs into a QCOW2 image, creating a complete boot stack.
 - **Streamlined boot process:** GRUB uses a configuration file (**grub.cfg**) to locate the kernel path, which requires the kernel and initramfs to reside on the same file system, for example, ext4 or XFS.
- Secure boot chain
 - **Secure boot:** EDK2 verifies the digital signatures of GRUB and the kernel, ensuring that the boot components have not been tampered with.
 - **Hardware resource collaboration:** virtCCA leverages UEFI runtime services to enumerate hardware devices, providing a virtualized resource pool for VM monitors like KVM.
- Cloud native optimization

virtCCA supports snapshot cloning and dynamic rootfs expansion (depending on cloud-init in initramfs).

Application Scenarios

The adoption of UEFI as a boot mode in cloud environments is a core technology for modern virtualization. virtCCA's new UEFI support expands its use cases for cVMs:

- Fast instance boot and auto scaling

UEFI uses parallel hardware initialization (such as simultaneous detection of CPUs, memory, and storage devices) to significantly shorten the boot time.
- Large-capacity drive support

By leveraging the GUID Partition Table (GPT), UEFI breaks the traditional master boot record limit of 2 TB. This allows virtCCA to support cloud drives of 100 TB or more (like Alibaba Cloud's ESSDs), which is essential for big data storage (such as HDFS) and AI training.

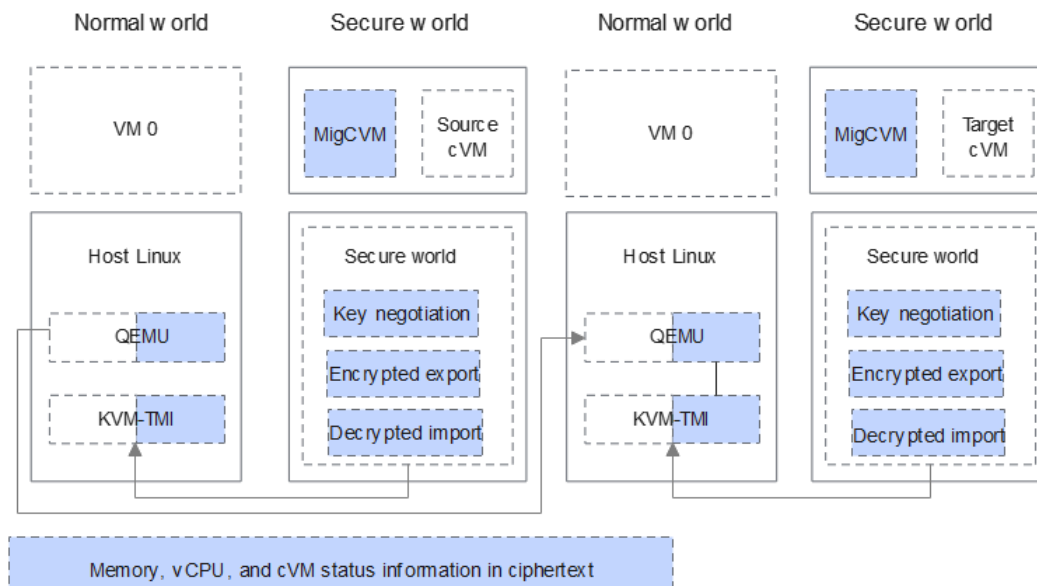
- Automated O&M and batch deployment

Image standardization: UEFI-compatible images provided by cloud service providers streamline user deployment by enabling GPT partitioning by default.

virtCCA cVM Live Migration

Feature Description

cVM live migration refers to the secure transfer of a running cVM from one confidential computing environment to another verified confidential environment without interrupting services. This process ensures that sensitive data within the cVM remains encrypted or isolated at all stages—before, during, and after the migration.



This feature ensures the confidentiality and integrity of a cVM's state in live migration. This is achieved through the MigCVM component (migration management) and the TMM component (secure world hypervisor). The functional highlights include:

- The source platform authenticates the target platform, and the TMM component performs trustworthiness measurement on MigCVM.
- After the identity authentication is successful, the source platform completes migration key negotiation with the target platform MigCVM.
- An encrypted session is established between the source and target platforms to ensure the security of data migration.
- MigCVM traces the migration status to secure the migration process and handle abnormal status.
- TMM is responsible for exporting and importing the memory status and vCPU status of the virtCCA cVM.
- TMM encrypts the cVM's status data based on the migration key and verifies the data integrity.

Application Scenarios

Mainstream VM management platforms require VMs to support live migration. This enables the transfer of VMs from one physical server to another in server maintenance scenarios, ensuring service continuity throughout the migration process. The virtCCA cVM live migration feature is primarily applicable to:

- Hardware and system maintenance and upgrade
When a physical server undergoes firmware upgrades, hardware replacement, or system updates, cVMs can be migrated to other hosts securely and in real time, ensuring zero service interruption.
- Load balancing and performance optimization
When a physical host experiences high CPU or memory load, cVMs with high resource consumption can be migrated to hosts with lower loads. This enables flexible scheduling to ensure optimal overall or partial performance.

Kuasar Confidential Container

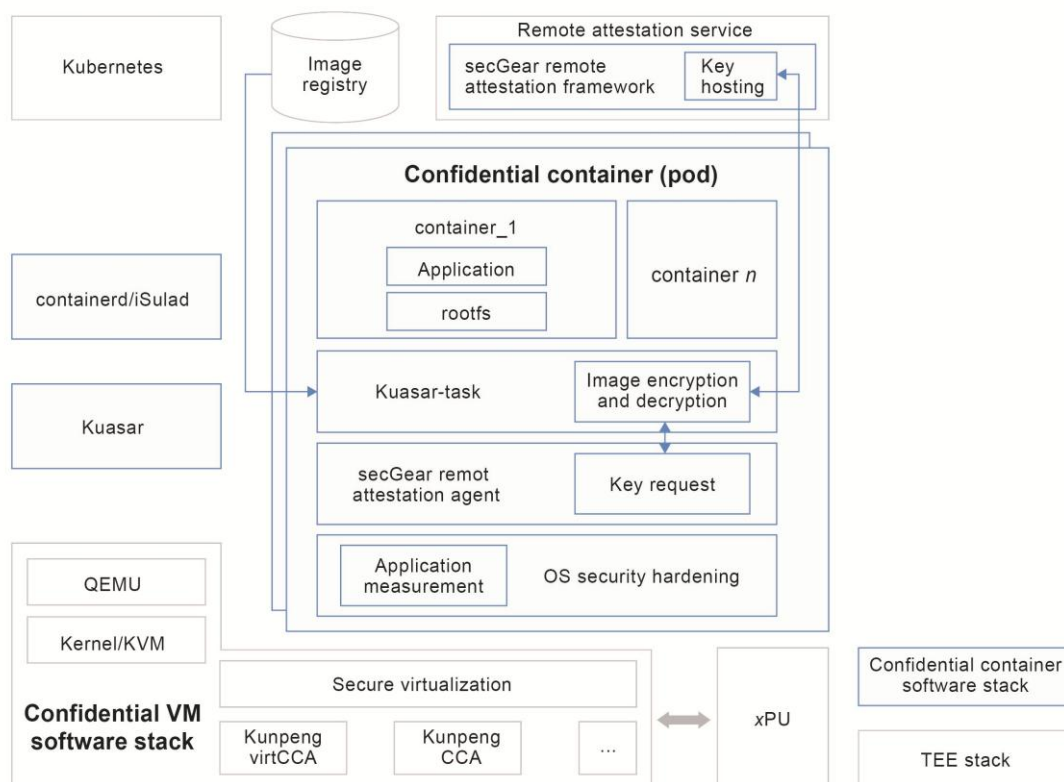
Feature Description

Kuasar has expanded its capabilities to include confidential container support while maintaining its existing secure container functionality. This support can be enabled through iSulad runtime configuration.

The current Kuasar confidential container implementation leverages the iSulad+Kuasar solution to significantly accelerate boot and drastically reduce memory overhead. On the one hand, the Sandbox API eliminates the need to create a separate pause container during container creation, saving time spent on preparing the pause container image snapshot. On the other hand, the 1:N management model allows the Sandboxer process to be persistent. This avoids the cold-start time of the Shim process, greatly accelerating container boot and bringing memory benefits proportional to the number of pods. Finally, Kuasar is implemented in Rust. Compared to Go, Rust provides inherent advantages in memory safety and contributes to overall memory efficiency.

Key functions include:

- Native integration with the iSulad container engine preserves Kubernetes ecosystem compatibility.
- Hardware-level protection via Kunpeng virtCCA technology ensures confidential workloads are deployed in trusted environments.
- The secGear remote attestation framework, which complies with the remote attestation procedures (RATS) (RFC9334), allows containers running in a confidential computing environment to prove their trustworthiness to external trusted services.
- Container images can be pulled and decrypted in confidential containers to protect their confidentiality and integrity.



Application Scenarios

Kuasar addresses data security concerns while seamlessly integrating with cloud native ecosystems. It benefits confidential applications from cloud native advantages including high availability, auto scaling, and rapid delivery. The solution finds broad application in confidential computing scenarios spanning AI security, trusted data circulation, and privacy protection.

Global Trust Authority for Remote Attestation

Feature Description

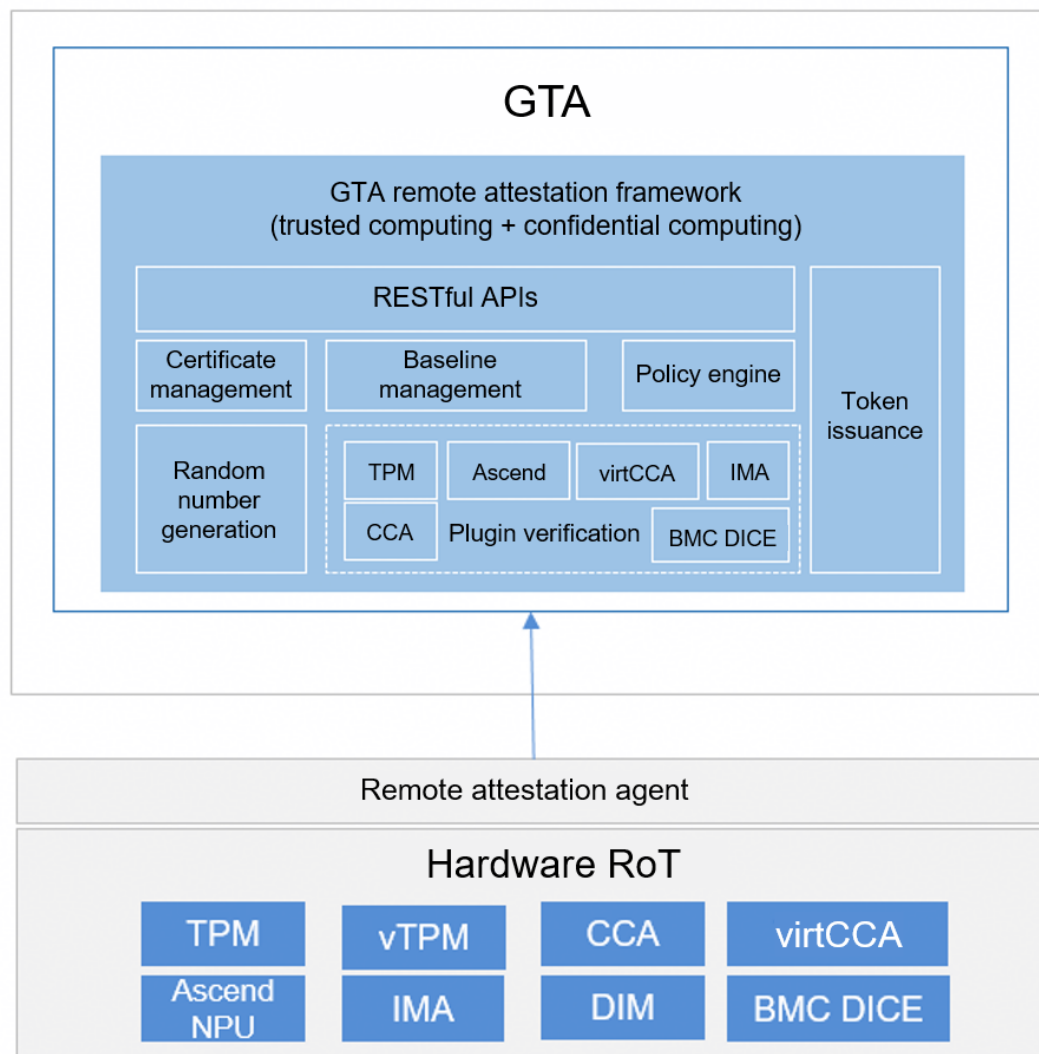
The Global Trust Authority (GTA) remote attestation component adopts a client-server architecture, supporting remote attestation of TPM/vTPM, BMC DICE, Ascend NPU, iTrustee, virtCCA, CCA, and IMA/DIM.

- The server provides the remote attestation service framework, which is compatible with trusted computing and confidential computing. It supports the addition, deletion, modification, and query of certificates and policies, quote verification, random number generation, and JWT token generation.
- The client collects local TPM evidence and interacts with the server to verify quotes.

This component provides various capabilities in terms of security and usability.

GTA provides differentiated security competitiveness such as database integrity protection, data link encryption, anti-replay, SQL injection prevention, user isolation, and key rotation.

The passport and background-check models are available. The client supports multiple verification modes, such as scheduled reporting and challenge response. Both the client and server can be deployed using RPM packages and within Docker containers.



Application Scenarios

Remote attestation is a prerequisite for enabling confidential computing and trusted computing. Subsequent computations are allowed only when the operating environment is strictly proven to be secure and trustworthy in the cryptographic sense. Remote attestation should be included as a core component of all end-to-end solutions involving confidential computing. If the operating environment is untrusted, the subsequent tasks must be stopped. This service is widely used in AI model protection, user privacy protection, and key management.

Global Trust Authority – Resource Broker Service

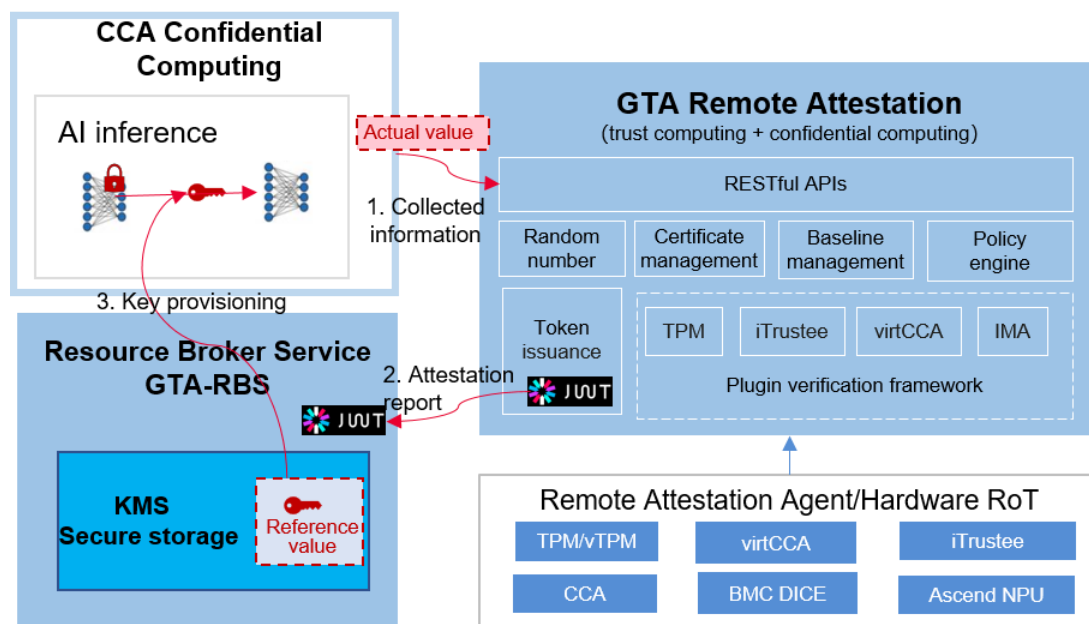
Feature Description

The Global Trust Authority (GTA) resource broker service (RBS) is a supporting service tightly coupled with the GTA remote attestation interface. RBS provides capabilities of importing, exporting, and securely storing resources (such as keys), and can bind resources to a security environment reference value (for example, mapping the resource of ID 1 to the CCA confidential

environment reference value RIM of 0x1234...). In practice, when a confidential environment requests a resource, RBS verifies the JWT report provided by the GTA remote attestation component. It delivers the resource to the confidential environment only if the actual value of this environment matches the predefined reference value. The combination of GTA remote attestation, confidential computing/trusted computing TEE, and RBS aligns with the definitions of Verifier, Attester, and Relying Party in RFC 9334, supporting both the background-check model and the passport model. This combination supports most confidential computing use cases and conforms to the general industry practices for confidential computing.

Application Scenarios

In the AI inference scenario, the model owner generates a key to encrypt the model locally, deploys the model in a confidential environment such as a CCA cVM, and imports the key into RBS for secure storage. The key is bound to a security environment reference value; for example, the reference values for CCA registers such as REMs 0–3 and RIM can be defined. At runtime, GTA remote attestation is used to verify in real time whether the confidential environment matches the predefined reference value of the REM or other registers. Once RBS passes the verification, it securely transmits the key to the cVM for subsequent model decryption. Combining RBS with remote attestation in confidential computing securely provisions keys required within a cVM. These keys are most frequently applied to data-at-rest encryption and decryption, and can also be utilized in other scenarios such as secure transmission.

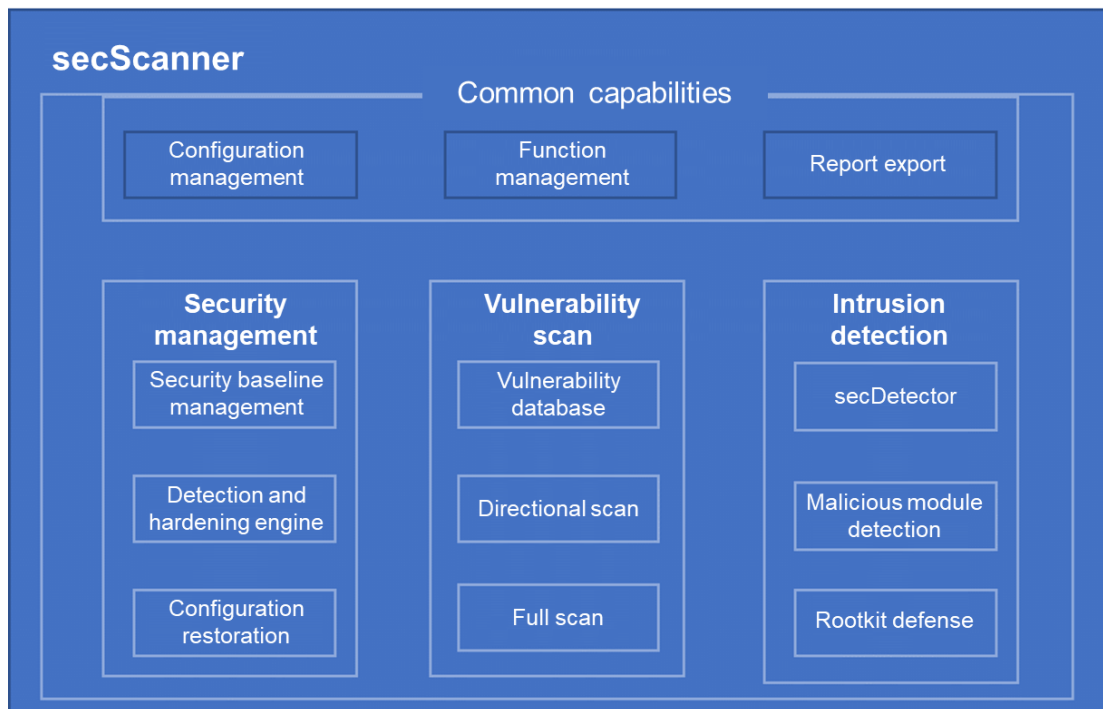


secScanner

secScanner is a Linux security scanner that provides system management, vulnerability scan, and rootkit detection to keep your OS protected.

Feature Description

Built on a "3 core + 3 common capabilities" architecture, secScanner provides comprehensive, automated, and flexible proactive protection.



- **Security management:** Addresses complex configurations and varying security needs across different scenarios. It supports multiple or custom security baselines with fine-grained parameter control. Key features include one-click security detection, system hardening, and configuration restoration to ensure robust protection.
- **Vulnerability scan:** Provides comprehensive and timely protection against known CVE threats in OS components. It downloads security advisories from openEuler Security Center to maintain an up-to-date local vulnerability database. It supports both full system scans and lightweight, targeted scans of key components. It identifies risky components and provides specific upgrade recommendations to block potential attack paths.
- **Intrusion detection:** Provides generalized protection against unknown threats. To defend against advanced intrusion methods, it utilizes secDetector to identify potential malicious rootkit modules and offers specific cleanup recommendations to ensure system security.

Application Scenarios

secScanner is primarily designed for server security O&M. It enables automated security O&M by detecting weak configurations with a single click and hardening them based on preset baselines. By automating these tasks, it minimizes risks caused by human error, reduces manual workload, and ensures robust system security.

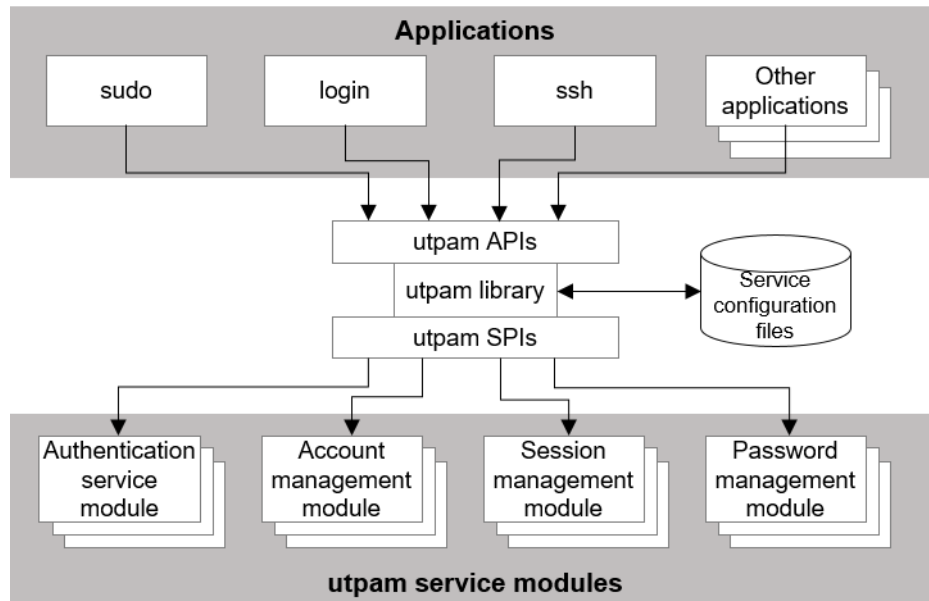
utpam

utpam is a Linux authentication framework. It allows system administrators to define and combine authentication methods for various services across the system as needed. By providing a unified interface, utpam streamlines user authentication for applications.

Feature Description

libutpam is the core library of utpam. It initializes authentication based on configuration files provided by the application. By default, these files are located in `/etc/utpam` and named after their respective services. utpam provides two types of interfaces:

- **APIs (northbound):** Enable applications to select from four authentication types: user authentication, account management, session management, and password management.
- **SPIs (southbound):** Allow developers to implement the specific authentication logic corresponding to each API.



Application Scenarios

utpam is a Rust-based extension suite for Linux PAM. It provides pluggable authentication modules and libraries that can be loaded on demand during login, authorization, and session management. Typical scenarios include:

- **Security hardening:** Enforces strict validation or default denials at specific stages to block non-compliant authentication paths.
- **Privilege control:** Implements fast-track approvals or extra checks for root and high-privilege accounts to balance security with operational efficiency.
- **Audit and prompts:** Outputs contextual data (user, TTY, service, and source) during the authentication process to improve observability.
- **Heterogeneous integration:** Accesses conventional PAM/C systems via stable C interfaces and headers while leveraging the safety of Rust shared libraries.

utpam is designed for teams and systems requiring granular control over authentication stages, enhanced auditing, and specialized policies for privileged accounts. It is also ideal for those looking to introduce reliable Rust implementations into existing PAM/C environments.

cu-concrete

Feature Description

cu-concrete is an enterprise-grade security hardening platform. It is used to automatically detect and remediate system-level security risks. The platform resolves a wide range of security issues, including low batch hardening efficiency, difficulties in adapting to security baselines, high coupling in traditional hardening scripts, hard-coded parameters, and high O&M costs. Designed with a layered and modular architecture, the platform integrates three interaction interfaces: CLI, TUI, and WebUI. It has 55 built-in standard hardening items and supports parameter-based configuration. This allows the platform to adapt to different security baselines without any code modifications. Furthermore, it offers an open framework capability, enabling users to develop and integrate new hardening modules based on specifications for continuous security evolution. The platform also supports batch delivery of hardening tasks by cloud pools or lists, while monitoring task execution status in real time. It automates configuration generation, allows script fine-tuning, and retains execution logs, ensuring end-to-end traceability from task delivery to result viewing. Overall, the platform adopts a configuration-based and open design concept. All hardening strategies are managed parametrically through YAML files. It ensures module extensibility through unified development and directory specifications, and achieves persistent hardening status via file-based, lightweight storage.

Application Scenarios

Designed for enterprise operations with large-scale heterogeneous server clusters, cu-concrete provides standard and batch security hardening and compliance rectification across diverse server environments, and easily adapts to various security baselines. It effectively addresses traditional challenges such as long remediation period, high manual maintenance costs, and baseline alignment difficulties. Furthermore, it empowers enterprises to continuously expand their hardening capabilities based on evolving security needs. Overall, the platform provides a stable, compliant, and auditable defense architecture for large-scale heterogeneous server clusters. It fully meets enterprise requirements for routine security hardening, batch maintenance and remediation, and compliance audit traceability.

cu-scanner

Feature Description

cu-scanner is a dedicated cybersecurity tool designed for scanning vulnerability information. It collects, organizes, and deduplicates vulnerability information from CVE and security advisory files or data interfaces. It then quickly converts such information into XML files in the Open Vulnerability and Assessment Language (OVAL) format and JSON files in the Security Content Automation Framework (SCAF) format.

The tool helps security personnel process vulnerability information efficiently and provides standardized data for vulnerability assessment and security scan. This enhances both the accuracy and timeliness of cybersecurity defense. Whether handling large volumes of local files or real-time data interfaces, cu-scanner reliably and quickly performs data processing and format conversion to meet security requirements across diverse scenarios.

Application Scenarios

- **Vulnerability assessment and disclosure:** Security tools use OVAL definitions to automatically evaluate systems for known vulnerabilities, helping organizations identify potential security risks.

- **Compliance check:** OVAL definitions help verify whether systems comply with specific security standards and regulatory requirements, such as PCI-DSS and HIPAA.
- **Configuration management:** By leveraging OVAL definitions, organizations can assess system configurations against best practices, reducing the risk of security loopholes.
- **Security audit:** Auditors use OVAL to systematically inspect and record system security states, ensuring that all security measures are effectively implemented.
- **Patch management:** OVAL helps identify systems that require patches, ensuring environments remain up-to-date and secure.
- **Automated response:** During security incident responses, OVAL enables rapid assessment of affected systems, helping security teams take prompt remediation actions.
- **Security tool integration:** Many security tools—such as vulnerability scanners and compliance checkers—support OVAL standards, enabling automated security assessment and reporting through OVAL definitions.

safeguard

Feature Description

safeguard is a Linux security audit and access control tool built on kernel runtime security instrumentation (KRSI) (eBPF+LSM). Designed for host and container environments, it monitors, audits, and blocks critical operations, including file access, network activity, storage mounting, and process execution. The tool supports security policy configuration based on the context, such as the process name, parent process name, UID, GID, and container scope. It can record risky behaviors in monitoring mode or enforce access control in blocking mode. In addition, safeguard supports trustlist policy generation, helping users build fine-grained, least-privilege runtime environments.

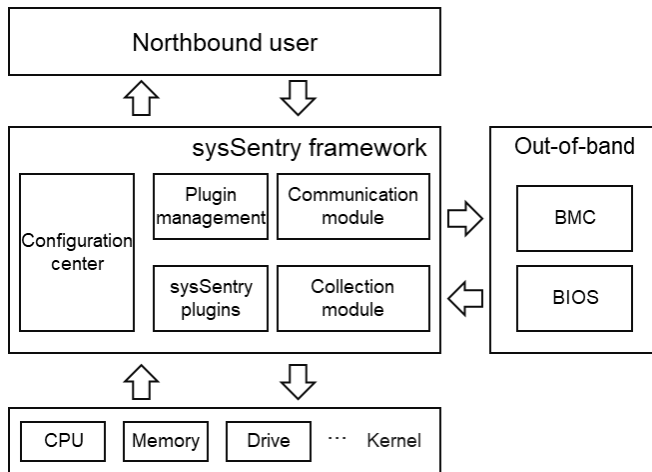
Application Scenarios

safeguard is suitable for security hardening and runtime protection across servers, cloud-native nodes, and container environments. Typical scenarios include: protecting sensitive file access, blocking abnormal network access, restricting container mount operations, enforcing process execution trustlists, auditing service runtime behavior, hardening supply chain build environments, and detecting and intercepting risks such as server-side request forgery (SSRF), container escape, and unauthorized access. Through these applications, safeguard enhances system security observability and proactive defense capabilities.

Reliability

sysSentry

sysSentry provides a fault inspection framework. By offering a unified northbound fault reporting interface and southbound plugins that support various inspection and diagnostic capabilities, the framework enables the inspection and diagnosis of hardware faults across CPUs, memory, drives, NPUs, and other components.

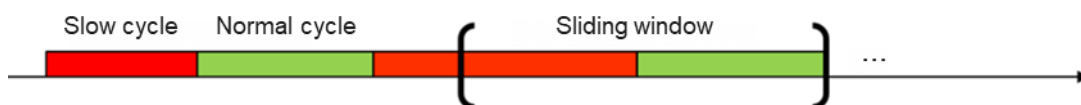


sysSentry features the following functions:

- **Unified alarm/event notification service:** Receives fault information reported by various plugins and forwards the information in a unified manner. This enables different subscriber services to receive specific fault notifications tailored to their operational needs.
- **Unified log service:** Aggregates and records fault information from all plugins, improving fault location efficiency.
- **Fault diagnosis/inspection framework:** Supports the development and configuration of inspection and diagnostic tasks using a pluggable approach. It allows plugins written in C/C++, Python, or Shell to be managed independently, including start/stop operations and status/result queries.
- **Lightweight data collection service:** Retrieves hardware status information via kernel, BIOS, and BMC interfaces for analysis and use by various plugins. It is designed for high adaptability, supporting a wide range of underlying architectures, software versions, and data collection requirements.

Slow I/O or Drive Detection Plugin

Feature Description



Slow I/O detection utilizes a sliding window to analyze the I/O latency data of various drives. A slow I/O event is triggered for a drive when the number of abnormal cycles within the sliding window exceeds a predefined threshold.

Currently, two slow I/O detection plugins are supported: **avg_block_io**, which calculates abnormal thresholds based on average I/O latency, and **ai_block_io**, which utilizes AI clustering algorithms to determine thresholds. Both plugins support monitoring for up to 10 I/O phases: blk-throttle, wbt, iocost, get_tag, plug, deadline, bfq, kyber, hctx, and driver.

Supported drive types include NVMe SSDs, SATA SSDs, and SATA HDDs.

Application Scenarios

- **Hardware faults:** slow I/O caused by drive damage, aging, or other physical hardware issues.
- **I/O stack exceptions:** slow I/O caused by errors in kernel I/O stack configuration or processing.
- **High service loads:** I/O resource over-utilization caused by heavy business demands.

Drive Lifespan Detection and Health Monitoring Plugin

Feature Description

The drive lifespan detection and health monitoring plugin leverages the `ipmitool` utility to poll and retrieve active drive alarms from the Baseboard Management Controller (BMC), filtering reported issues according to user-defined configuration parameters. To facilitate rapid troubleshooting and early warning of drive failures, the plugin resolves mapping relationships between logical device names and physical serial numbers (SNs):

- **Pass-through (JBOD) mode:** Queries the IPMI interface directly to retrieve device names and their corresponding physical drive SNs.
- **RAID mode:** Utilizes vendor-specific RAID management tools to establish the exact mapping between logical device names and physical drive SNs.

Application Scenarios

Currently exclusive to the Kunpeng platform, this feature targets Huawei-developed NVMe drives. It delivers timely proactive alerts for potential hardware failures, thermal threshold breaches, I/O performance degradation, and low remaining endurance (lifespan), ensuring high system reliability and availability.

Zone Reclaim Optimization for HPC Memory

Feature Description

When Zone Reclaim is enabled, new logic is introduced to wake up `kswapd` prior to triggering direct memory reclamation. When free memory is scarce, the system synchronously wakes up `kswapd` to perform asynchronous background memory reclamation, restoring free memory levels up to the high watermark. This prevents frequent direct memory reclamation during page fault handling, thereby improving memory allocation efficiency and boosting overall Zone Reclaim performance.

Application Scenarios

In typical dual-socket (2P) server architectures, each compute node encompasses multiple DDR memory nodes, with each node provisioned with a fixed memory capacity. In memory-intensive workloads, this architecture—characterized by a large number of memory nodes with limited per-node capacity—is prone to triggering the out-of-memory (OOM) killer when a single node exhausts its memory, leading to forced termination of user-space processes.

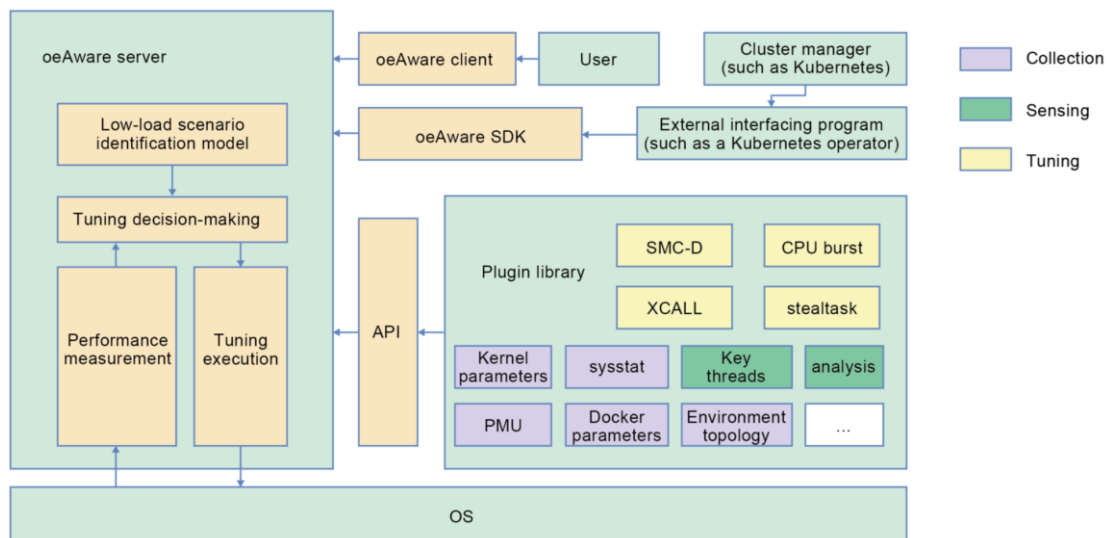
To mitigate this issue, enabling the Zone Reclaim mechanism is generally recommended. When a single node encounters memory pressure, Zone Reclaim prioritizes reclaiming local file page caches over allocating hard-to-reclaim anonymous pages across remote nodes. This localized reclamation strategy effectively relieves single-node memory shortages and enhances NUMA memory locality.

By introducing asynchronous background memory reclamation, this feature eliminates the latency overhead associated with synchronous direct reclamation. It streamlines memory allocation, ultimately elevating overall system performance.

Performance Tuning

oeAware Enhancements

oeAware is a framework that provides low-load collection, sensing, and tuning upon detecting defined system actions on openEuler. The framework divides the tuning process into three layers: collection, sensing, and tuning. Each layer is developed as plugins and associated through subscription, overcoming the limitations of traditional tuning techniques that run independently and are statically enabled or disabled.



Feature Description

Each oeAware plugin is a dynamic library built on standard interfaces. A plugin consists of several instances, each acting as an independent functional set for data collection, sensing, or tuning. Within these instances are topics, which output the processed data. This data is then shared with other plugins or external applications for further analysis and optimization.

- The SDK enables subscription to plugin topics, with a callback function handling data from oeAware. This allows external applications to create tailored functionalities, such as cross-node information collection or local node analysis.
- The performance monitoring unit (PMU) information collection plugin gathers performance records from the system PMU.
- The Docker information collection plugin retrieves specific parameter details about Docker containers in the environment.
- The system information collection plugin captures kernel parameters, thread details, and resource information (CPUs, memory, I/Os, and network) from the current environment.
- The thread sensing plugin monitors key information about threads.
- The evaluation plugin examines NUMA and network information during service operations, suggesting optimal tuning methods.

- The system tuning plugins comprise (1) `stealtask` for enhanced CPU tuning, (2) `smc_tune` (SMC-D) which leverages shared memory communication in the kernel space to boost network throughput and reduce latency, (3) `xcall_tune` (XCALL) which offers code paths that bypass non-critical processes to minimize system call processing overhead, and (4) `soft_domain_tune` for task scheduling by domain to isolate multiple instances into separate domains for optimal affinity.
- The Docker tuning plugin addresses CPU performance issues during sudden load spikes by utilizing the CPU burst feature.

Constraints

- **SMC-D:** SMC acceleration must be enabled before the server-client connection is established. This plugin is most effective in scenarios with numerous persistent connections.
- **Docker tuning:** This plugin is not compatible with Kubernetes containers.
- **xcall_tune:** The **FAST_SYSCALL** kernel configuration option must be activated.
- **realtime_tune:** This plugin must be used together with the Preempt-RT kernel.
- **net_hard_irq_tune:** This plugin applies only to network communication over TCP.
- **soft_domain_tune:** The **SOFT_DOMAIN** kernel configuration option must be enabled, and the container size cannot exceed the size of a NUMA node.

Application Scenarios

`stealtask` is ideal for scenarios aiming to boost CPU utilization, such as in Doris. This tuning instance effectively increases CPU utilization and prevents idle CPU cycles.

XCALL is designed for applications with substantial system call overhead. It offers code paths that bypass non-critical processes, optimizing system call handling and reducing overhead. However, this approach may compromise some maintenance and security capabilities.

SMC-D excels in environments demanding high throughput and low latency, including high-performance computing (HPC), big data processing, and cloud platforms. By leveraging direct memory access (DMA), SMC-D significantly reduces CPU load and accelerates interactive workloads.

`soft_domain_tune` is applicable to multi-instance workloads, where the instance size does not exceed the size of a single NUMA node. These instances run independently of each other. By partitioning the scheduling scope into domains, this feature reduces interference between different workloads.

CPU burst is tailored for high-load container environments like Doris, addressing performance bottlenecks caused by CPU constraints.

`realtime_tune` is applicable to scenarios that demand low latency, minimal jitter, and strict latency restrictions, such as in the industrial manufacturing sector.

`net_hard_irq_tune` is best suited for services whose performance is heavily impacted by TCP networking, including applications like Redis and Nginx.

NUMA-aware scheduling applies to workloads, like Spark, where cross-NUMA memory access significantly degrades service performance and fixed core pinning is difficult to maintain.

Traffic-based container scheduling is ideal for containerized services, such as containerized MySQL. When the service load is low, it satisfies QoS requirements while using the minimum necessary CPU resources. This reduces CPU migration, idle switching, cache misses, and cross-NUMA access, thereby enhancing resource locality. When the service load is high, it can bypass

core pinning constraints to utilize more CPU resources, improving QoS and maximizing overall CPU resource utilization.

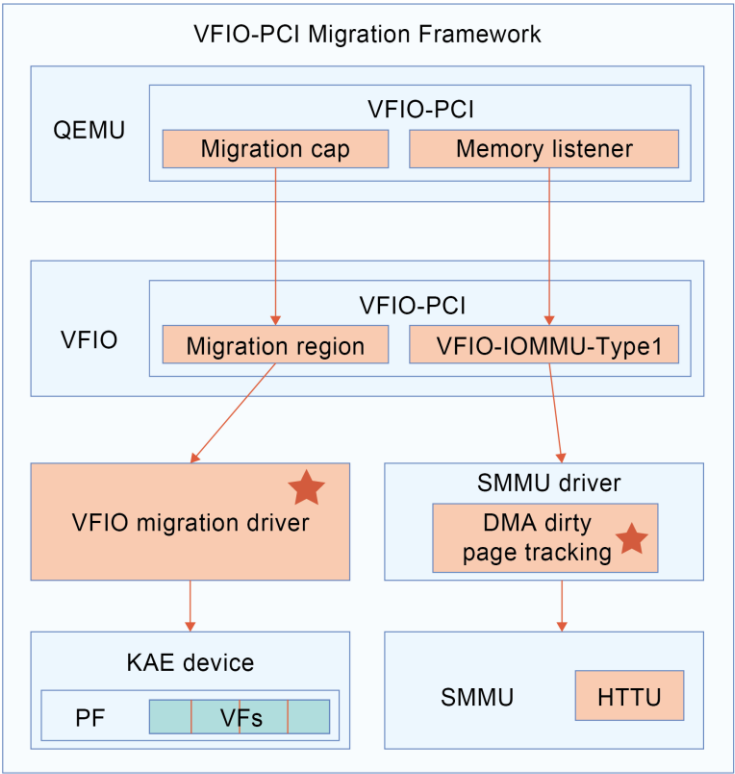
Virtualization

vKAE Passthrough Live Migration

Feature Description

The Kunpeng Accelerator Engine (KAE) is a hardware-based acceleration solution provided by the new Kunpeng 920 processor model, featuring HPRE, SEC, and ZIP devices for encryption, decryption, compression, and decompression. This enables KAE to significantly reduce processor overhead and boost efficiency. KAE passthrough live migration addresses the scenario where VMs are configured with KAE passthrough devices, offering enhanced operational flexibility and continuous service availability.

SMMU dirty page tracking is a key technology for efficient and reliable live migration of passthrough devices. In the Arm architecture, a purely software-based approach to dirty page tracking incurs significant performance overhead. Hardware Translation Table Update (HTTU) solves this by allowing the hardware to automatically update the SMMU page table status. During a write operation, the write permission bit of the corresponding page table entry is automatically set. During a live migration, the write permission bit of the page table is scanned to collect statistics on dirty pages.



Application Scenarios

This feature empowers live migration for VMs using KAE passthrough devices, making it ideal for fields with high requirements for data security and processing performance, such as finance, cloud computing, and big data processing. Ultimately, it enhances business continuity and operational stability.

VMAnalyzer

VMAnalyzer is a lightweight virtualization monitoring and analysis tool designed with two core objectives:

Virtualization environment monitoring: real-time tracking of CPUs, memory, and drive I/Os to identify performance bottlenecks.

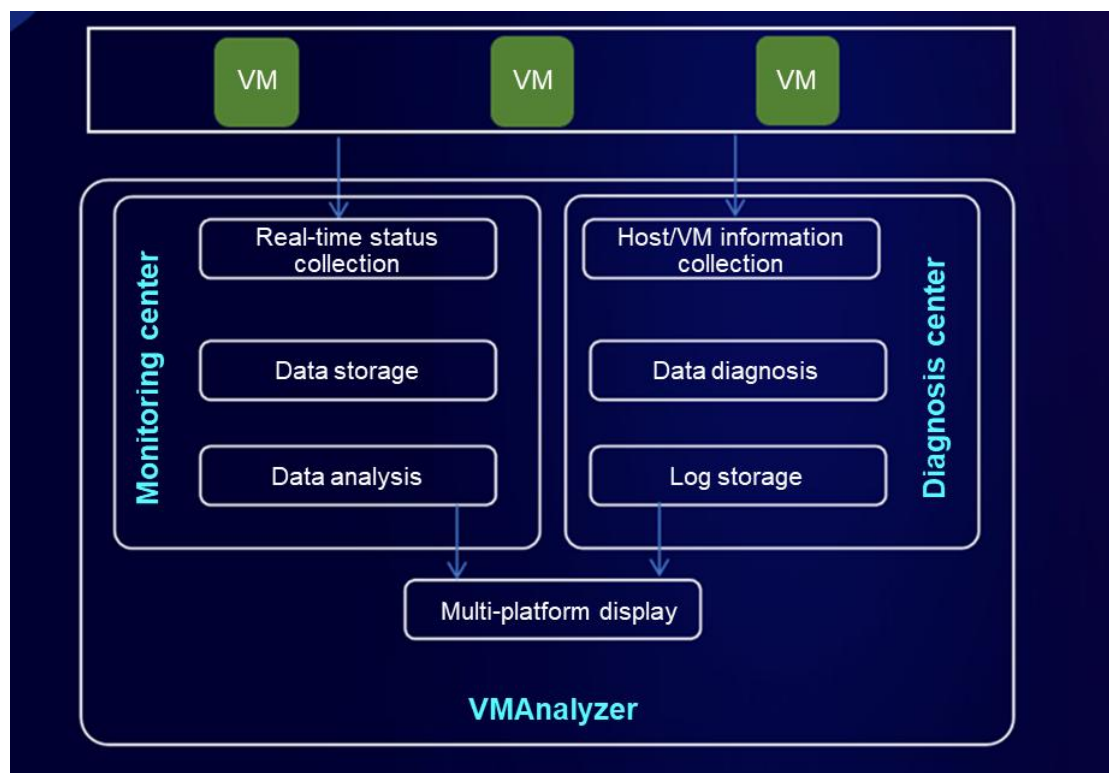
High reliability assurance: analyzing QEMU processes and physical configurations to predict risks and deliver proactive maintenance policies for failure prevention.

Feature Description

VMAnalyzer consists of two primary modules: the monitoring center and the diagnosis center.

Monitoring center: Provides real-time data collection and granular analysis of VM health. It offers flexible visualization of performance metrics across all cloud hosts via integration with platforms such as the Console, OPS, and DW.

Diagnosis center: Conducts deep-dive diagnostics of the virtualization layer. By executing targeted commands to analyze configurations, system status, and VM resource consumption, it identifies complex issues and exports detailed findings into comprehensive log files.



Application Scenarios

VMAnalyzer is optimized for cloud computing environments. It provides granular insights into VM health and performance, enabling you to easily identify bottlenecks. By streamlining maintenance, VMAnalyzer ensures that VMs operate with high performance and maximum reliability.

GIC Oversubscription Optimization

Feature Description

Generic interrupt controller v4.1 (GICv4.1) inherently requires multiple VMOVP commands to be executed serially with synchronization. Due to this limitation, performance deteriorates in scenarios involving oversubscription and range-based core pinning, where vCPUs migrate frequently.

However, oversubscription is widely used to maximize computing power. As a standard virtualization I/O optimization, GICv4.1 suffers from performance degradation in 1:2 oversubscription scenarios. This forces users to disable GICv4.1, leaving them unable to resolve the critical pain point of virtualization overhead in such I/O-intensive oversubscription scenarios. To improve performance, this feature is designed to reduce the number of VMOVP commands, alleviating this performance bottleneck and further boosting overall performance.

Usage:

A new parameter, `/sys/module/kvm/parameters/enable_vmovp_elision`, is introduced to toggle this feature. To enable it, set this parameter to **Y**.

Application Scenarios

On the new Kunpeng 920 processor model, this feature optimizes performance for scenarios where GICv4.1 is enabled and VMs frequently migrate between cores within the same affinity group.

vTimer Passthrough

Feature Description

In traditional VMs, a timer interrupt triggers a VM exit upon expiration. The hypervisor then injects the interrupt before execution switches back to the guest. This process can significantly impact latency-sensitive workloads that rely heavily on timers. The vTimer passthrough feature eliminates VM exits and entries during timer interrupt injection. By bypassing these traps, it reduces interrupt injection latency and improves performance in relevant scenarios.

Application Scenarios

On new models of Kunpeng 920 and later processors with GICv4.1 enabled, this feature significantly improves timer latency accuracy for guest OS workloads that demand ultra-low timer latency.

Hardware

Raspberry Pi

Raspberry Pi is a series of single-board computers developed by the Raspberry Pi Foundation and Broadcom. They are widely used in industrial automation, robotics, Internet of Things (IoT),

education, and enthusiast projects due to their low price, small size, low power consumption, high programmability, and abundant ecosystem. Raspberry Pi 4B and Raspberry Pi 5 are classic products, both using Arm processors. Raspberry Pi 4B is a cost-effective entry-level computer, and Raspberry Pi 5 is an innovative product with significant performance breakthroughs and extended capabilities, making it competitive in high-performance edge computing.

Feature Description

As typical open source hardware products, Raspberry Pi 4B and Raspberry Pi 5 support multiple Linux distributions such as Raspberry Pi OS, Ubuntu, and openEuler. They have extensive peripherals, powerful video encoding and decoding capabilities, LAN on motherboard (LOM), and can be used as independent computer systems.

Application Scenarios

Raspberry Pi 4B and Raspberry Pi 5 are widely used in many fields:

- **Education:** programming language learning such as Python, and electronic experiments using the peripheral interfaces
- **Multimedia and entertainment:** media center or game console
- **IoT and smart home:** sensor nodes or smart home hubs for environment monitoring, automation control, and edge computing
- **Server and network applications:** home servers, lightweight web services, and containerized applications
- **DIY projects:** robot control, 3D printing management, and drone flight control
- **Research and development:** AI experiments and prototype validation in embedded development
- **Industrial automation:** device monitoring, man-machine interface, and machine vision

Enhanced Hygon Features

Hygon CCP Driver Support for SM4-XTS and SM4-GCM

Feature Description

This feature updates the kernel driver of the Hygon cryptographic co-processor (CCP) to support advanced SM4 algorithm modes, including SM4-XTS and SM4-GCM. SM4-XTS uses a tweakable block cipher mode, making it suitable for block device and drive encryption. It requires a 32-byte key and a 16-byte initialization vector (IV). SM4-GCM features a Galois/Counter Mode that supports additional authenticated data (AAD) with an authentication tag, ensuring both data confidentiality and integrity. The algorithms are registered with the Linux kernel crypto API as `xts-sm4-ccp` and `gcm-sm4-ccp`, while retaining the `xts-sm4-cis` path for CIS extensions. For testing, the **ccp_test.ko** kernel module directly invokes the `crypto_skcipher` and `crypto_aead` APIs. Additionally, user-space `AF_ALG` interface testing is provided via `libkcapi`. Test vectors reference GB/T 17964-2021 for SM4-XTS and RFC 8998 for SM4-GCM.

Application Scenarios

- **Block device and drive encryption:** Provides SM4-XTS hardware-accelerated encryption for server local drive storage and cloud storage, satisfying data-at-rest security requirements.

- **Databases and core financial services:** Leverages the authenticated encryption with associated data (AEAD) capability of SM4-GCM to provide confidentiality and integrity protection for transaction data, logs, and message queues.
- **Container and cloud native environments:** Delivers SM algorithm support for Kubernetes persistent volumes and container image layer encryption.
- **Regulatory compliance:** Meets compliance requirements for the SM4 algorithm across public, finance, and telecommunications sectors.

SDS

fastblock

Feature Description

fastblock is an open-source distributed block storage system engineered for high performance and ultra-low latency. It overcomes the bottlenecks of conventional SDS systems (such as Ceph) in NVMe SSD environments, including high CPU overhead, high single-volume latency, and limited availability.

- **Ultra-fast I/O path:** Built on the Storage Performance Development Kit (SPDK), fastblock employs user-space NVMe drivers and lock-free queues to bypass the kernel, achieving extreme single-volume read/write performance and ultra-low latency.
- **Zero-copy networking:** Leverages Remote Direct Memory Access (RDMA) for data transmission, supporting kernel bypass and zero-copy operations to minimize CPU involvement during network communications.
- **Strongly consistent replication:** Employs the Multi-Raft consensus protocol for data replication. This ensures strong data consistency and high reliability while avoiding I/O blocking during cluster jitter commonly seen in conventional primary-secondary synchronous replication.
- **Efficient metadata management:** Utilizes Go and etcd to build a lightweight and easily customizable Monitor cluster, ensuring high consistency of metadata views across clients and Object Storage Daemons (OSDs).

Application Scenarios

fastblock is ideal for modern data centers and cloud infrastructure with stringent requirements for storage throughput, IOPS, and response time:

- **High-performance virtualization and cloud platforms:** Seamlessly integrates with QEMU/KVM virtual machines via the SPDK vhost-blk interface, delivering sub-millisecond low-latency, high-concurrency system and data volume services for public and private cloud platforms.
- **Containerized cloud-native databases:** Connects with Kubernetes via the Container Storage Interface (CSI) to provide high-performance, highly available persistent volumes for I/O-sensitive databases and StatefulSets, including MySQL, PostgreSQL, and Redis.
- **HPC and bare metal services:** Maps remote block devices directly to bare-metal servers using Network Block Devices (NBDs), powering compute-storage disaggregated workloads such as AI model training and big data analytics that demand massive throughput.
- **High-availability financial-grade core applications:** Capitalizes on Multi-Raft strong consistency and RDMA low-latency networking to meet the rigorous zero-data-loss and ultra-fast response demands of core systems, including financial trading and online payment platforms.

7 Copyright Statement

All materials or contents contained in this document are protected by the copyright law, and all copyrights are owned by openEuler, except for the content cited by other parties. Without a prior written permission of the openEuler community or other parties concerned, no person or organization shall reproduce, distribute, reprint, or publicize any content of this document in any form; link to or transmit the content through hyperlinks; upload the content to other servers using the "method of images"; store the content in information retrieval systems; or use the content for any other commercial purposes. For non-commercial and personal use, the content of the website may be downloaded or printed on the condition that the content is not modified and all rights statements are reserved.

8 Trademarks

All trademarks and logos used and displayed on this document are all owned by the openEuler community, except for trademarks, logos, and trade names that are owned by other parties. Without the written permission of the openEuler community or other parties, any content in this document shall not be deemed as granting the permission or right to use any of the aforementioned trademarks and logos by implication, no objection, or other means. Without prior written consent, no one is allowed to use the name, trademark, or logo of the openEuler community in any form.

9 Appendixes

Appendix 1: Setting Up the Development Environment

Environment Setup	URL
Downloading and installing openEuler	https://www.openeuler.org/en/
Preparing the development environment	https://gitee.com/openeuler/community/blob/master/en/contributors/prepare-environment.md
Building a software package	https://gitee.com/openeuler/community/blob/master/en/contributors/package-install.md

Appendix 2: Security Handling Process and Disclosure

Security Issue Disclosure	URL
Security handling process	https://gitee.com/openeuler/security-committee/blob/master/docs/en/vulnerability-management-process/security-process-en.md
Security disclosure	https://gitee.com/openeuler/security-committee/blob/master/docs/en/vulnerability-management-process/security-disclosure-en.md
Security strategy overview	https://gitee.com/openeuler/security-committee/blob/master/docs/en/vulnerability-management-process/security-strategy-overview-en.md

Appendix 3: Specifications and Compatibility for openEuler-Incubated Features on 64K Page-Size Kernels

Component	Requirement Description	64K Compatibility
kernel	ext4: Use iomap for buffered I/O paths of regular files and enable large folios.	Incompatible. Restricted via mount parameters. Disabled by default.
	clear_free_list_pages sysctl	Incompatible. Disabled in the default kernel config.
	Dirty page logging	Incompatible. Disabled in the default kernel config.
	Tiered memory reliability	Incompatible. Disabled in the default kernel config.
	Per-memcg swap control	Incompatible. Disabled in the default kernel config.
	Shared memory mapping tools supporting huge pages	Incompatible. Disabled in the default kernel config.
Virtualization	VM tiered memory compression	Incompatible. Unavailable on 64K page size.
	Kernel-space vDPA system drives and online data drive hot-expansion for DPUs	Incompatible. Guarded by default.
	vDPA	Incompatible. Guarded by default.
secDetector	In-kernel intrusion detection system designed for OSs to provide intrusion detection and response capabilities for critical information infrastructure	Incompatible. Installation blocked by default.
Dynamic Integrity Measurement (DIM)	Change detection in memory code segments caused by attacks such as runtime tampering and injection	Incompatible. Installation blocked by default.
sysMonitor	OS execution fault monitoring and logging	Incompatible. Installation blocked by default.
sysMaster	Innovative Process 1 scheme, enabling second-level fault self-healing to maintain 24/7 system availability	Incompatible. Installation blocked by default.